Anisa / Iran Linux House

MLOps Fundamentals On-Prem

واژه‌نامه‌ی MLOps

MLOps Fundamentals On-Prem — Glossary

لغت‌نامه‌ی ساده‌ی بیش از ۲۶۰ واژه‌ی دوره برای کسی که هنوز هیچ‌کدام را نمی‌شناسد

مدرس

سعید رضا سالاری

Saeed Reza Salari

نسخه‌ی ۱٫۰ · تاریخ: مهر ۱۴۰۵

فهرست مطالب

Table of Contents

چطور از این کتابچه استفاده کنیم 3

نقشه‌ی دوره Course map 4

1 پایه‌های خط فرمان و شبکه Command Line & Networking Basics 5

2 پایتون و محیط اجرا Python & Environments 12

3 مبانی یادگیری ماشین Machine Learning Basics 15

4 مفاهیم MLOps MLOps Concepts 21

5 Git و کار تیمی Git & Teamwork 26

6 کیفیت کد Code Quality 32

7 Docker Docker & Containers 35

8 ذخیره‌ساز شیء و S3 Object Storage & S3 39

9 DVC: نسخه‌بندی داده و pipeline DVC: Data Versioning & Pipelines 42

10 MLflow و مخزن مدل MLflow & the Model Registry 46

11 سرویس‌دهی مدل با FastAPI Model Serving with FastAPI 50

12 بسته‌بندی و امنیت image Containerization & Image Security 53

13 استقرار روی VM و Infrastructure as Code VM Deployment & IaC 55

14 Kubernetes Kubernetes 58

15 Helm و Ingress Ingress & Helm 63

16 CI/CD CI/CD 65

17 پایش و Drift Monitoring & Drift 67

جفت‌های اشتباه‌برانگیز Commonly confused pairs 70

اختصارها Abbreviations 75

نمایه‌ی انگلیسی English index 77

برای رفتن به هر بخش روی عنوان کلیک کنید. شماره‌ی صفحه‌ها با نسخه‌ی PDF همین کتابچه هماهنگ است.

چطور از این کتابچه استفاده کنیم

How to use this booklet

این کتابچه برای کسی نوشته شده که هیچ‌کدام از این فناوری‌ها را نمی‌شناسد. لازم نیست از اول تا آخر بخوانید؛ آن را کنار دستتان نگه دارید و هر وقت واژه‌ای در کلاس یا آزمایشگاه برایتان ناآشنا بود، سراغش بروید.

1. واژه را پیدا کنید: در «نمایه‌ی انگلیسی» (پایان کتابچه) به ترتیب حروف الفبای انگلیسی دنبالش بگردید، یا از «نقشه‌ی دوره» ببینید در کدام جلسه آمده. شماره‌ی جلسه و شماره‌ی صفحه هر دو لینک‌دار هستند و با کلیک روی آن‌ها در نسخه‌ی دیجیتال به جای درست می‌روید.
2. ساختار هر واژه: عنوان (واژه‌ی انگلیسی، معادل فارسی و در صورت لزوم تلفظ)، تعریف ساده، سطر «در این دوره» (نمونه‌ی واقعی از دستور، فایل یا عدد همین دوره) و سطر «همچنین ببینید» برای واژه‌های مرتبط.
3. فصل‌ها به ترتیب دوره چیده شده‌اند و واژه‌های هر فصل از ساده به پیچیده مرتب شده‌اند.
4. واژه‌های مشابه که زیاد با هم اشتباه گرفته می‌شوند (مثل image و container یا CI و CD) در بخش «جفت‌های اشتباه‌برانگیز» کنار هم مقایسه شده‌اند و در خود واژه هم با علامت مشخص شده‌اند.
5. دستورها و نام فایل‌ها همیشه با خط ثابت و از چپ به راست نوشته شده‌اند. شماره‌ی جلسه‌ها طبق نقشه‌ی ۱۴ جلسه‌ای دوره است؛ مثال‌های جلسه‌های ۵ تا ۱۴ ممکن است در نسخه‌ی نهایی تمرین‌ها جزئی تفاوت داشته باشند.
6. هیچ رمز یا کلیدی در این کتابچه نیامده است. «کلید آزمایشگاه» را از مدرس بگیرید و فقط در فایل `.env` بگذارید.

نقشه‌ی دوره

Course map

هر جلسه‌ی ۳ ساعته چند واژه‌ی کلیدی تازه دارد. جلسه‌ی ۰ پیش‌کار است و پروژه‌ی تجمعی churn تا جلسه‌ی ۱۴ ادامه پیدا می‌کند. روی هر واژه کلیک کنید تا به تعریفش بروید.

جلسه	موضوع	واژه‌های کلیدی
۰	**پیش‌کار: Bash و نصب از بسته‌ی آفلاین**	Terminal، Bash، PATH، Environment variable، Pipe، Exit code، SSH، curl، Air-gapped، Wheelhouse
۱	**شروع دوره و مبانی MLOps**	MLOps، ML lifecycle، The 7 MLOps components، MLOps maturity levels، Reproducibility، uv، Lockfile، Baseline، ROC-AUC، F1 score
۲	**Git برای ML و کیفیت کد**	Commit، Branch، Pull Request، Tag، SemVer، Conventional Commits، ruff، pytest، pre-commit، Git hook، Gitea
۳	**Docker و ذخیره‌ساز شیء**	Container، Image، Docker Compose، Volume، Bucket، S3، SeaweedFS، Access key، Authentication
۴	**نسخه‌بندی داده با DVC**	DVC، Pointer file، DVC cache، Remote، Hash (md5)، dvc checkout، Data versioning، Data lake، Data drift
۵	**pipeline تکرارپذیر با DVC**	DVC pipeline، dvc.yaml، DAG، dvc repro، Params، DVC metrics، dvc.lock، Orchestration
۶	**ردیابی آزمایش با MLflow**	MLflow، Run، Experiment، Tracking server، Artifact، Experiment tracking
۷	**مخزن مدل و promotion**	Model registry، Registered model، Alias، Champion، Promotion، Evaluation gate، Model signature، Model card، SHAP
۸	**سرویس‌دهی با FastAPI**	Model serving، FastAPI، uvicorn، pydantic، Endpoint، Request، Health check، /metrics endpoint
۹	**بسته‌بندی و امنیت image**	Dockerfile، Multi-stage build، Non-root user، Container registry، Trivy، CVE، SBOM، Syft
۱۰	**استقرار روی VM و IaC**	Virtual machine، Multipass، systemd، Reverse proxy، Infrastructure as Code، Ansible، Playbook، Inventory، Idempotent، Rollback
۱۱	**Kubernetes**	Kubernetes، Cluster، kind، kubectl، Pod، Deployment، Service، ConfigMap، Secret، Probes: liveness، Rollout
۱۲	**Ingress و Helm**	Ingress، Ingress controller، Gateway API، Helm، Chart، Values، Release
۱۳	**CI آگاه از ML، پایش و drift**	CI، Gitea Actions، Runner، ML-aware CI، Scheduled workflow، Prometheus، Grafana، RED method، Evidently، Retraining
۱۴	**پروژه‌ی نهایی و ارائه**	Clean clone، Definition of Done، Production-readiness checklist، Runbook، Golden repo، MLOps maturity levels، SLO

1 پایه‌های خط فرمان و شبکه

Command Line & Networking Basics

تقریباً همه‌ی آزمایشگاه‌های این دوره با تایپ دستور در ترمینال انجام می‌شود. این فصل واژه‌هایی را توضیح می‌دهد که در «جلسه‌ی ۰» (پیش‌کار Bash) و هر جلسه‌ی بعد بدون توضیح دوباره به گوش می‌خورند.

جلسه‌ها: جلسه ۰ تا ۱۴ · **تعداد واژه‌ها:** ۳۲

1.1 Terminal · ترمینال

ترمینال پنجره‌ای متنی است که در آن به‌جای کلیک کردن، دستور تایپ می‌کنید و جواب را به‌صورت متن می‌گیرید. مثل گفت‌وگوی پیامکی با کامپیوتر است: شما یک جمله می‌نویسید و کامپیوتر پاسخ می‌دهد.

> **در این دوره:** همه‌ی دستورهای دوره (`git`، `uv`، `docker`، `kubectl`) در ترمینال اجرا می‌شوند؛ فونت ترمینال کلاس حداقل ۱۸ پوینت است (جلسه‌ی ۰ و ۱).

همچنین ببینید: Shell، Bash، Command

1.2 Shell · پوسته

Shell برنامه‌ای است که داخل ترمینال اجرا می‌شود، متنی را که تایپ کرده‌اید می‌خواند و آن را به سیستم‌عامل می‌دهد تا اجرا کند. ترمینال فقط «پنجره» است و Shell «مترجمی» است که پشت آن پنجره کار می‌کند.

> **در این دوره:** در این دوره Shell همان Bash است؛ کاربران ویندوز آن را داخل WSL2 اجرا می‌کنند (جلسه‌ی ۰).

همچنین ببینید: Terminal، Bash، WSL2

1.3 Bash · بش

Bash رایج‌ترین Shell در لینوکس و پیش‌فرض بسیاری از توزیع‌هاست. علاوه بر اجرای تک‌دستور، یک زبان اسکریپت‌نویسی ساده هم دارد: می‌توانید چند دستور را در یک فایل بنویسید و یک‌جا اجرا کنید.

> **در این دوره:** «جلسه‌ی ۰» به Bash اختصاص دارد؛ اسکریپت‌هایی مثل `preflight.sh` و `install-from-bundle.sh` با Bash نوشته شده‌اند.

همچنین ببینید: Shell، Shell script، Command

1.4 Command · دستور

Command یک سطر متن است که به کامپیوتر می‌گوید چه کاری انجام دهد؛ معمولاً با نام یک برنامه شروع می‌شود و بعد از آن گزینه‌ها و ورودی‌ها می‌آیند. بعد از زدن Enter دستور اجرا و نتیجه چاپ می‌شود.

> **در این دوره:** نمونه‌ی ساده‌ی دوره برای دیدن نسخه‌ی ابزار: `uv --version`. بقیه‌ی دستورها هم همین الگو را دارند.

همچنین ببینید: Option / Flag / Argument، Terminal، Exit code

1.5 Option / Flag / Argument · گزینه، پرچم، آرگومان

Argument هر مقدار ورودی است که بعد از نام دستور می‌دهید (مثلاً نام فایل). Option یا Flag رفتار دستور را تغییر می‌دهد و معمولاً با یک یا دو خط تیره شروع می‌شود، مثل `ls -l` یا `ls --help`. مثل سفارش غذا: «پیتزا» دستور است، «بدون پنیر» گزینه و «شماره‌ی میز ۵» آرگومان.

> **در این دوره:** در `mkdir -p a/b` گزینه‌ی p یعنی «اگر پوشه وجود داشت خطا نده»؛ در `git commit -m "msg"` گزینه‌ی m پیام را می‌گیرد (جلسه‌ی ۰ و ۲).

همچنین ببینید: Command، Shell script

1.6 Path · مسیر

Path آدرس یک فایل یا پوشه در سیستم فایل است. مسیر مطلق (absolute) از ریشه شروع می‌شود، مانند `/home/student/churn`، و مسیر نسبی (relative) از پوشه‌ای که الان در آن هستید (پوشه‌ی کاری یا working directory) حساب می‌شود، مانند `src/churn/train.py`.

> **در این دوره:** بیشتر خطاهای «فایل پیدا نشد» در دوره به این برمی‌گردد که دستور از ریشه‌ی Repo اجرا نشده است؛ مثلاً `uv run python -m churn.train --config configs/params.yaml` فقط از ریشه‌ی پروژه کار می‌کند (جلسه‌ی ۱).

همچنین ببینید: Environment variable، PATH

1.7 Environment variable · متغیر محیطی

متغیر محیطی یک جفت «نام و مقدار» است که Shell در اختیار همه‌ی برنامه‌هایی که اجرا می‌کند می‌گذارد. برنامه‌ها از آن برای تنظیمات و اطلاعات حساس استفاده می‌کنند تا لازم نباشد آن‌ها داخل کد نوشته شوند.

```
set -a; source .env; set +a
```

> **در این دوره:** آدرس و کلید S3 آزمایشگاه در فایل `.env` و به شکل متغیرهایی مانند `S3_ENDPOINT_URL` و `AWS_ACCESS_KEY_ID` نگه‌داری می‌شود، نه داخل Git (جلسه‌ی ۳ و ۴).

همچنین ببینید: .env file، Secret، PATH

1.8 .env file · فایل تنظیمات محیطی (دات‌اِنو)

فایل متنی ساده با سطرهایی به شکل `NAME=value` که تنظیمات یک پروژه را در یک‌جا نگه می‌دارد و معمولاً به متغیرهای محیطی تبدیل می‌شود. چون ممکن است رمز و کلید داشته باشد، هرگز commit نمی‌شود؛ به‌جای آن یک نمونه‌ی بدون رمز به نام `.env.example` در Git می‌گذارند.

> **در این دوره:** در Repo دوره `.env` در `.gitignore` است و `make s3-ls` مقدارهای آن را برای آدرس و کلید S3 می‌خواند (جلسه‌ی ۲ و ۳).

همچنین ببینید: Environment variable، Secret، .gitignore.

1.9 PATH · متغیر PATH

PATH یک متغیر محیطی خاص است که فهرست پوشه‌هایی را دارد که Shell برای پیدا کردن برنامه‌ی هر دستور در آن‌ها می‌گردد. اگر برنامه‌ای در هیچ‌کدام از این پوشه‌ها نباشد، با پیام «command not found» روبه‌رو می‌شوید.

در این دوره: نصب‌کننده‌ی آفلاین ابزارها را در `~/.local/bin` می‌گذارد؛ اگر `uv: command not found` دیدید، ترمینال جدید باز کنید یا `source ~/.bashrc` بزنید تا PATH به‌روز شود (جلسه‌ی ۰ و ۱).

همچنین ببینید: Environment variable، Command

1.10 sudo · — (سودو)

`sudo` یک دستور را با اختیارات مدیر سیستم (root) اجرا می‌کند. کارهایی مثل نصب برنامه‌ی سیستمی یا تغییر تنظیمات کل ماشین به این اختیار نیاز دارند، و چون اشتباه با آن آسیب جدی می‌زند باید فقط وقتی لازم است استفاده شود.

در این دوره: نصب Docker و Multipass روی لپ‌تاپ و ساخت VM در جلسه‌ی ۱۰ به دسترسی admin/sudo نیاز دارند؛ همین را قبل از دوره با `preflight.sh` بررسی می‌کنیم (جلسه‌ی ۰ و ۱۰).

همچنین ببینید: File permission، SSH

1.11 File permission · مجوز فایل

در لینوکس هر فایل مشخص می‌کند چه کسی اجازه‌ی خواندن (read)، نوشتن (write) و اجرا (execute) دارد. اگر یک اسکریپت مجوز اجرا نداشته باشد، تا آن را با `chmod +x` ندهید اجرا نمی‌شود.

در این دوره: فایل‌هایی مثل `scripts/preflight.sh` باید مجوز اجرا داشته باشند؛ در جلسه‌ی ۹ هم کانتینر را با کاربر بدون اختیار (non-root) اجرا می‌کنیم.

همچنین ببینید: sudo، Shell script، Non-root user

1.12 Pipe · لوله

Pipe با نماد `|` خروجی یک دستور را مستقیم به ورودی دستور بعدی می‌دهد. مثل خط تولید: خروجی ایستگاه اول بدون واسطه به ایستگاه دوم می‌رود.

```
docker ps | grep seaweedfs
```

در این دوره: برای فیلتر کردن خروجی طولانی، مثلاً `docker ps | grep seaweedfs` (جلسه‌ی ۰ و ۳).

همچنین ببینید: Redirect، stdout / stderr

1.13 Redirect · تغییر مسیر خروجی

Redirect به شما اجازه می‌دهد به‌جای چاپ خروجی دستور روی صفحه، آن را داخل یک فایل بنویسید (`>` برای جایگزین کردن و `>>` برای افزودن به انتهای فایل). مثل اینکه به‌جای خواندن نتیجه‌ی یک آزمایش از روی صفحه، آن را در دفترچه یادداشت کنید.

در این دوره: در Bash پیش‌کار برای ذخیره‌ی خروجی `preflight.sh` و فرستادن آن برای مدرس از آن استفاده می‌شود (جلسه‌ی ۰).

همچنین ببینید: Pipe، stdout / stderr

1.14 stdout / stderr · خروجی استاندارد / خطای استاندارد (اِس‌تی‌دی‌اوت / اِس‌تی‌دی‌اِر)

هر برنامه دو مجرای خروجی دارد: `stdout` برای نتیجه‌ی عادی و `stderr` برای پیام‌های خطا و هشدار. هر دو به‌طور پیش‌فرض روی صفحه چاپ می‌شوند، ولی می‌توان آن‌ها را جدا از هم به فایل فرستاد.

> **در این دوره:** لاگ‌های برنامه‌های داخل کانتینر هم از همین دو مجرا می‌آیند و با `docker logs` یا `kubectl logs` دیده می‌شوند (جلسه‌ی ۳ و ۱۱).

همچنین ببینید: Redirect، Exit code، Logging (structured)

1.15 Exit code · کد خروج

هر برنامه در پایان یک عدد به Shell برمی‌گرداند: ۰ یعنی موفق و هر عدد دیگر یعنی خطا. ابزارهای خودکار مانند Git hookها و CI فقط با همین عدد می‌فهمند که مرحله موفق بوده یا نه.

```
ruff check . ; echo $?
```

> **در این دوره:** اگر hook قبل از commit با کد غیر صفر تمام شود، Git کل commit را لغو می‌کند؛ در Bash آخرین کد با `echo $?` دیده می‌شود (جلسه‌ی ۰ و ۲).

همچنین ببینید: Git hook، CI (Continuous Integration)، stdout / stderr

1.16 Shell script · اسکریپت شل

فایلی متنی که چند دستور Shell را پشت سر هم نگه می‌دارد تا بتوان همه را با یک فرمان اجرا کرد. هر کاری که بیش از یک‌بار تکرار شود، کاندید اسکریپت شدن است.

> **در این دوره:** `preflight.sh` سلامت لپ‌تاپ را بررسی می‌کند و اگر همه‌چیز درست باشد کلمه‌ی `READY` را چاپ می‌کند؛ `kind-up.sh` کلاستر Kubernetes را بالا می‌آورد (جلسه‌ی ۰ و ۱۱).

همچنین ببینید: Bash، Makefile / make، Exit code

1.17 SSH — (اِس‌اِس‌اِچ)

SSH روش امن اتصال به یک کامپیوتر دیگر از راه شبکه است: روی ماشین دور یک ترمینال می‌گیرید یا فایل منتقل می‌کنید، و کل ارتباط رمزگذاری شده است. معمولاً با یک جفت کلید (عمومی و خصوصی) به‌جای رمز عبور وارد می‌شوند.

> **در این دوره:** Gitea در لپ‌تاپ روی پورت `2222` هم SSH دارد (برای push امن) و Ansible در جلسه‌ی ۱۰ از همین راه به VM اوبونتو وصل می‌شود.

همچنین ببینید: Ansible، Virtual machine (VM)، Port

1.18 curl — (کِرل)

`curl` ابزار خط فرمانی برای فرستادن درخواست HTTP به یک آدرس و دیدن پاسخ است. برای آزمایش سریع یک API لازم نیست مرورگر باز کنید.

```
curl -i http://localhost:8333
```

> **در این دوره:** برای آزمایش سلامت SeaweedFS روی `localhost:8333` و بعدها صدا زدن `/health` و `/predict` در API استفاده می‌شود (جلسه‌ی ۰، ۴ و ۸).

همچنین ببینید: HTTP، URL، HTTP status code، API

1.19 localhost · میزبان محلی (لوکال‌هاست)

`localhost` نام ویژه‌ای است که همیشه به همان کامپیوتری اشاره می‌کند که در آن هستید (آدرس IP آن `127.0.0.1` است). سرویسی که فقط روی localhost گوش می‌دهد از بیرون ماشین دیده نمی‌شود.

در این دوره: تمام سرویس‌های آزمایشگاه فقط روی `127.0.0.1` باز می‌شوند، مثل Gitea روی `localhost:3000` و MLflow روی `localhost:5050`؛ پس چیزی در شبکه‌ی کلاس در معرض دید نیست (جلسه‌ی ۱ تا ۱۳).

همچنین ببینید: Port، IP address، URL

1.20 Port · پورت (درگاه)

اگر آدرس IP مثل نشانی یک ساختمان باشد، Port شماره‌ی واحد داخل آن ساختمان است: یک ماشین می‌تواند چندین سرویس را روی پورت‌های مختلف اجرا کند. هر سرویس با «آدرس:پورت» پیدا می‌شود.

در این دوره: پورت‌های دوره: Gitea `3000`، S3 در SeaweedFS `8333`، MLflow `5050`، registry `5001`، Ingress `8080`. پورت `5000` عمداً استفاده نشده چون AirPlay مک آن را می‌گیرد (جلسه‌ی ۱ تا ۱۲).

همچنین ببینید: IP address، localhost، Port mapping

1.21 IP address · نشانی IP (آی‌پی)

عددی (مانند `192.168.1.10`) که هر دستگاه را در یک شبکه مشخص می‌کند تا بسته‌های داده به مقصد درست برسند. نشانی‌هایی که با `127.` شروع می‌شوند همیشه به خود ماشین اشاره دارند.

در این دوره: IP کانتینرها و پادها داخل شبکه‌ی خصوصی Docker و Kubernetes تعریف می‌شود؛ ما بیشتر با نام (DNS) کار می‌کنیم، نه IP (جلسه‌ی ۳ و ۱۱).

همچنین ببینید: DNS، Port، localhost

1.22 DNS · سامانه‌ی نام دامنه (دی‌ان‌اس)

DNS مثل دفترچه‌ی تلفن اینترنت است: نام قابل‌فهمی مثل `churn.localhost` را به نشانی IP تبدیل می‌کند. بدون آن باید همه‌ی IPها را حفظ می‌کردیم.

در این دوره: داخل Docker Compose سرویس‌ها با نامشان به هم وصل می‌شوند (مثلاً MLflow به `seaweedfs:8333`)، و Ingress با نام `churn.localhost:8080` آدرس می‌دهد (جلسه‌ی ۳ و ۱۲).

همچنین ببینید: IP address، URL، Ingress

1.23 HTTP · پروتکل انتقال ابرمتن (اچ‌تی‌تی‌پی)

HTTP قرارداد گفت‌وگوی مرورگرها و برنامه‌ها با سرورهاست: مشتری «درخواست» (request) می‌فرستد و سرور «پاسخ» (response) می‌دهد. هر درخواست یک متد (مثل GET یا POST)، یک آدرس و گاهی بدنه دارد.

در این دوره: API مدل با HTTP صدا زده می‌شود: `GET /health` برای سلامت و `POST /predict` برای پیش‌بینی (جلسه‌ی ۸).

همچنین ببینید: HTTP status code، URL، REST، API

1.24 HTTP status code · کد وضعیت HTTP

عددی سه‌رقمی که سرور همراه پاسخ می‌فرستد تا نتیجه را بگوید: خانواده‌ی ۲۰۰ موفق، ۴۰۰ خطای درخواست‌کننده (مثل ۴۰۴ پیدا نشد یا ۴۲۲ ورودی نامعتبر)، و ۵۰۰ خطای خود سرور. نخستین رقم برای فهمیدن اوضاع کافی است: ۲ یعنی خوب، ۴ یعنی اشتباه از سمت شما و ۵ یعنی اشتباه از سمت سرور.

در این دوره: سلامت API با `200` از `/health` تأیید می‌شود و ۴۰۳ از SeaweedFS یعنی سرویس بالاست ولی درخواست بدون اعتبارنامه است (جلسه‌ی ۴ و ۸).

همچنین ببینید: HTTP، Health check، curl

1.25 URL · نشانی وب (یوآرال)

URL آدرس کامل یک منبع است و از چند بخش ساخته می‌شود: طرح (`http`)، میزبان (`localhost`)، پورت (`localhost:8333`) و مسیر (`/dvc/store`). مثل نشانی پستی کامل که شهر، خیابان و پلاک دارد.

در این دوره: `http://localhost:5050` رابط MLflow و `s3://dvc/store` مسیر remote در DVC است (جلسه‌ی ۴ و ۶).

همچنین ببینید: HTTP، Port، DNS

1.26 API · رابط برنامه‌نویسی (ای‌پی‌آی)

API (Application Programming Interface) راهی است که یک برنامه به برنامه‌ی دیگر اجازه می‌دهد خدمتی از آن بخواهد، بدون اینکه درونش را ببیند. مثل منوی رستوران: شما از روی منو سفارش می‌دهید و نمی‌دانید آشپزخانه چطور کار می‌کند.

در این دوره: مدل churn در جلسه‌ی ۸ پشت یک API قرار می‌گیرد؛ S3 هم یک API است که boto3، DVC و MLflow با آن حرف می‌زنند.

همچنین ببینید: REST، HTTP، JSON، Endpoint (API)

1.27 REST · — (رست)

REST سبکی برای طراحی API روی HTTP است که در آن هر «منبع» یک آدرس دارد و عملیات با متدهای HTTP (GET برای خواندن، POST برای ساختن/فرستادن) انجام می‌شود. داده معمولاً به‌شکل JSON رد و بدل می‌شود.

در این دوره: API پیش‌بینی churn یک سرویس REST است: ورودی JSON می‌گیرد و احتمال ریزش را به‌صورت JSON برمی‌گرداند (جلسه‌ی ۸).

همچنین ببینید: HTTP، JSON، API

1.28 JSON · — (جی‌سون)

JSON قالب متنی ساده برای نگه‌داری داده‌ی ساخت‌یافته با کلید و مقدار است، مثلاً `{"tenure": 12}`. هم انسان آن را می‌خواند و هم برنامه‌ها، و رایج‌ترین قالب تبادل داده‌ی API است.

در این دوره: فایل `metrics.json` خروجی آموزش است و شامل `roc_auc` و `f1` می‌شود؛ بدنه‌ی درخواست و پاسخ API هم JSON است (جلسه‌ی ۱ و ۸).

همچنین ببینید: YAML، CSV، REST

1.29 · YAML — (یامل)

YAML قالب متنی برای فایل‌های تنظیمات است که با تورفتگی (indent) ساختار را نشان می‌دهد و از JSON خواناتر است. در تورفتگی فاصله‌ی خالی مهم است و TAB مجاز نیست.

در این دوره: `configs/params.yaml`، `dvc.yaml`، فایل‌های manifest در Kubernetes، playbook آنسیبل و workflow در Gitea Actions همه YAML هستند.

همچنین ببینید: TOML، JSON، Manifest، Playbook

1.30 · TOML — (تامل)

TOML قالب فایل تنظیمات با بخش‌هایی به شکل `[section]` و سطرهای `key = value` است. در Python برای تنظیم پروژه به کار می‌رود.

در این دوره: فایل `pyproject.toml` در Repo، نام و وابستگی‌های پروژه و تنظیمات ruff و pytest را نگه می‌دارد (جلسه‌ی ۱ و ۲).

همچنین ببینید: pyproject.toml، YAML، ruff

1.31 · CSV — (سی‌اس‌وی)

CSV فایل متنی جدول داده است که هر سطر یک رکورد و هر ستون با ویرگول جدا شده است؛ مثل یک جدول Excel ساده بدون ظاهر. داده‌ی دوره هم همین قالب را دارد.

در این دوره: `data/raw/churn.csv` در نسخه‌ی ۱ پنج هزار سطر دارد (۲۰۱۲۰۲ بایت) و در نسخه‌ی ۲ هشت هزار سطر (۳۲۰۷۶۳ بایت) (جلسه‌ی ۴).

همچنین ببینید: Dataset، JSON، Pointer file (.dvc)

1.32 · WSL2 زیرسیستم ویندوز برای لینوکس (دبلیوواس‌اِل‌تو)

WSL2 قابلیتی در ویندوز است که یک لینوکس واقعی (مثلاً اوبونتو) را درون ویندوز اجرا می‌کند. دانشجویان ویندوزی تمام دستورهای دوره را در همین محیط اجرا می‌کنند تا همه یک محیط یکسان داشته باشند.

در این دوره: Repo را در پوشه‌ی خانه‌ی لینوکس (`~/work`) بسازید، نه در `/mnt/c`، چون آنجا بسیار کند است؛ Docker Engine هم داخل WSL نصب می‌شود (جلسه‌ی ۰).

همچنین ببینید: Shell، Bash

2 پایتون و محیط اجرا
Python & Environments

Python زبان اصلی دوره است. بیشتر خطاهای روزهای اول از «محیط» می‌آید، نه از کد؛ این فصل واژه‌هایی را می‌گوید که نصب و اجرای تکرارپذیر پروژه به آن‌ها وابسته است.

جلسه‌ها: جلسه ۱، ۲، ۹ · **تعداد واژه‌ها:** ۱۲

2.1 Python · پایتون

Python زبان برنامه‌نویسی ساده و محبوب در علم داده و یادگیری ماشین است. تقریباً همه‌ی ابزارهای این دوره (scikit-learn، DVC، MLflow، FastAPI، Ansible) با Python نوشته شده‌اند.

در این دوره: نسخه‌ی دوره Python `3.12` است که با uv مدیریت می‌شود؛ خودِ پروژه با `requires-python = ">=3.12"` این را اعلام می‌کند (جلسه‌ی ۱).

همچنین ببینید: uv، pyproject.toml، Virtual environment

2.2 Module / Package · ماژول / بسته‌ی پایتون

Module یک فایل `.py` شامل کد قابل استفاده‌ی دوباره است و Package پوشه‌ای از ماژول‌هاست که می‌توان آن را `import` کرد. نصب یک «بسته از اینترنت» (مثل pandas) و ساختن بسته‌ی خودتان دو چیز متفاوت‌اند، ولی هر دو با همین اسم گفته می‌شوند.

```
uv run python -m churn.train --config configs/params.yaml
```

در این دوره: کد پروژه بسته‌ای به نام `churn` در `src/churn/` است و آموزش با `python -m churn.train` اجرا می‌شود (جلسه‌ی ۱).

همچنین ببینید: Dependency، pyproject.toml

2.3 Dependency · وابستگی

Dependency هر بسته‌ی دیگری است که کد شما برای کار کردن به آن نیاز دارد. هر وابستگی خودش ممکن است وابستگی‌های دیگری داشته باشد (وابستگی غیرمستقیم).

```
uv add scikit-learn pandas pyyaml
```

در این دوره: وابستگی‌های تولید پروژه `pandas`، `pyyaml` و `scikit-learn` هستند؛ ابزارهای توسعه مثل `ruff` و `pytest` در گروه dev جدا نگه داشته می‌شوند (جلسه‌ی ۱ و ۲).

همچنین ببینید: Lockfile (uv.lock)، Virtual environment، pip

2.4 Virtual environment · محیط مجازی

محیط مجازی (venv) یک پوشه‌ی جداگانه برای بسته‌های یک پروژه است تا نسخه‌های پروژه‌ی دیگر با آن قاطی نشود. مثل داشتن یک جعبه‌ابزار مجزا برای هر کار، به‌جای یک کشوی مشترک.

در این دوره: uv محیط پروژه را در پوشه‌ی `.venv` می‌سازد و `uv run` هر دستور را داخل همان اجرا می‌کند (جلسه‌ی ۱).

همچنین ببینید: uv، Dependency، Lockfile (uv.lock)

2.5 pip · — (پیپ)

`pip` ابزار قدیمی و استاندارد نصب بسته‌های Python از PyPI است. در این دوره به‌جای آن از uv استفاده می‌کنیم که سریع‌تر است و lockfile هم می‌سازد.

در این دوره: فایل `lab-kit/requirements.txt` فهرست نسخه‌های دقیق بسته‌هاست که بسته‌ی آفلاین از روی آن wheelhouse می‌سازد (جلسه‌ی ۰ و ۱).

همچنین ببینید: uv، PyPI، Wheelhouse

2.6 uv · — (یووی)

uv یک مدیر سریع Python است که کار نصب خود Python، ساخت محیط مجازی، نصب بسته‌ها و ساخت lockfile را یک‌جا انجام می‌دهد. یک ابزار به‌جای چند ابزار جدا.

```
uv init --package --name churn --python 3.12
```

در این دوره: نسخه‌ی دوره `0.12.15` است؛ دستورهای `uv init`، `uv add`، `uv sync` و `uv run` در همه‌ی جلسه‌ها به کار می‌روند و در کلاس از wheelhouse آفلاین نصب می‌کنند (جلسه‌ی ۱).

همچنین ببینید: Virtual environment، Lockfile (uv.lock)، Wheelhouse

2.7 pyproject.toml · فایل pyproject.toml

فایل استاندارد تعریف یک پروژه‌ی Python: نام، نسخه، وابستگی‌ها و تنظیم ابزارها. با خواندن همین یک فایل می‌توان فهمید پروژه چه نیاز دارد.

در این دوره: در Repo دوره، علاوه بر وابستگی‌ها، بخش‌های `[tool.ruff]` و `[tool.pytest.ini_options]` تنظیمات کیفیت کد را دارند (جلسه‌ی ۱ و ۲).

همچنین ببینید: TOML، Dependency، Lockfile (uv.lock)، ruff

2.8 Lockfile (uv.lock) · فایل قفل

Lockfile نسخه‌ی دقیق همه‌ی بسته‌ها، حتی وابستگی‌های غیرمستقیم، را ثبت می‌کند تا همه‌ی اعضای تیم و سرور CI دقیقاً همان محیط را بسازند. بدون آن «روی لپ‌تاپ من کار می‌کرد» اتفاق می‌افتد.

در این دوره: فایل `uv.lock` همراه کد در Git commit می‌شود؛ بدون commit کردن آن، Repo تکرارپذیر نیست (جلسه‌ی ۱ و ۲).

همچنین ببینید: Reproducibility، Dependency، uv

2.9 Wheel · بسته‌ی از پیش‌ساخته (wheel) (ویل)

Wheel فایل `.whl` است: بسته‌ی Python که قبلاً کامپایل شده و آماده‌ی نصب است، پس نیازی به ساختن روی ماشین شما نیست. بعضی wheelها کد C دارند و به معماری CPU (مثلاً x86 یا ARM) وابسته‌اند.

> **در این دوره:** به همین دلیل بسته‌ی آفلاین دو نسخه‌ی `amd64` و `arm64` دارد و مک‌های Apple Silicon باید نسخه‌ی arm64 را بردارند (جلسه‌ی ۰).

همچنین ببینید: Wheelhouse، PyPI

2.10 Wheelhouse · انبار wheel آفلاین (ویل‌هاوس)

Wheelhouse پوشه‌ای است که همه‌ی wheelهای لازم پروژه در آن جمع شده تا بدون اینترنت نصب شوند. جایگزین محلی PyPI برای شبکه‌ی بسته.

> **در این دوره:** `install-from-bundle.sh` تنظیم uv را در `~/.config/uv/uv.toml` می‌نویسد تا فقط از wheelhouse نصب کند؛ اگر نسخه‌ای بیرون از فهرست بخواهید، خطای `No solution found` می‌گیرید (جلسه‌ی ۰ و ۱).

همچنین ببینید: Wheel، Air-gapped / Offline، PyPI

2.11 PyPI · فهرست بسته‌های پایتون (پای‌پی‌آی)

PyPI مخزن عمومی و رسمی بسته‌های Python است که `pip` و `uv` به‌طور پیش‌فرض از آن دانلود می‌کنند. مثل یک اپ‌استور برای کتابخانه‌های Python.

> **در این دوره:** در کلاس آفلاین، PyPI در دسترس نیست و جایش را wheelhouse گرفته است (جلسه‌ی ۰).

همچنین ببینید: pip، Wheelhouse، Air-gapped / Offline

2.12 Jupyter Notebook · نوت‌بوک (جوپیتر نوت‌بوک)

Notebook محیطی است که کد، خروجی و توضیح را در یک سند تعاملی کنار هم می‌گذارد و برای کشف و آزمایش بسیار راحت است. اشکالش این است که ترتیب اجرای سلول‌ها می‌تواند نتیجه را غیرقابل‌تکرار کند.

> **در این دوره:** سطح صفر بلوغ در جلسه‌ی ۱ معمولاً «نوت‌بوک ← فایل pickle ← استقرار دستی» است؛ کد دوره به‌جای آن در فایل‌های `.py` با تست و pipeline نوشته می‌شود.

همچنین ببینید: MLOps maturity levels، Baseline

3 مبانی یادگیری ماشین
Machine Learning Basics

دوره یک کلاس علم داده نیست؛ مدل «موضوع» کار است نه «هدف». ولی برای فهمیدن اینکه چه چیزی را نسخه‌بندی، ثبت و پایش می‌کنیم، به همین واژه‌های پایه نیاز دارید. مثال همه‌جا پیش‌بینی ریزش مشتری (churn) است.

جلسه‌ها: جلسه ۱، ۴، ۶، ۷، ۱۳ · **تعداد واژه‌ها:** ۲۸

3.1 Model · مدل

Model برنامه‌ای است که از روی داده‌ی گذشته یاد گرفته است ورودی‌های جدید را به یک خروجی (مثل احتمال ریزش) تبدیل کند. در عمل یک فایل است که شامل ساختار و اعداد یادگرفته‌شده است و می‌توان آن را ذخیره و جابه‌جا کرد.

در این دوره: مدل پایه‌ی دوره یک logistic regression است که در فایل `models/model.joblib` ذخیره می‌شود (جلسه‌ی ۱).

همچنین ببینید: Training، Inference / Prediction، Logistic regression، Serialization (joblib / pickle)

3.2 Training · آموزش (مدل)

Training فرایندی است که طی آن الگوریتم با دیدن داده‌های مثال، اعداد داخلی مدل را طوری تنظیم می‌کند که خطایش کم شود. مثل دانش‌آموزی که با حل کردن تمرین‌های زیاد یاد می‌گیرد.

```
make train
```

در این دوره: `make train` مدل را آموزش می‌دهد و چاپ می‌کند: `{'roc_auc': 0.802, 'f1': 0.5969}` برای داده‌ی نسخه‌ی ۱ (جلسه‌ی ۱ و ۴).

همچنین ببینید: Inference / Prediction، Parameter (weight)، Hyperparameter، Retraining

3.3 Inference / Prediction · استنتاج / پیش‌بینی

Inference یعنی استفاده از مدلِ آموزش‌دیده برای گرفتن خروجی روی داده‌ی جدید. آموزش یک‌بار و سنگین است، ولی inference بارها و معمولاً سریع و آنی انجام می‌شود.

در این دوره: سرویس FastAPI در جلسه‌ی ۸ فقط inference انجام می‌دهد: ورودی یک مشتری و خروجی احتمال ریزش او است.

همچنین ببینید: Training، Endpoint (API)، Model serving

3.4 Feature · ویژگی (فیچر)

Feature یک ستون ورودی مدل است؛ یعنی یک مشخصه‌ی قابل‌اندازه‌گیری از هر نمونه. برای هر مشتری مثلاً مدت اشتراک یا مبلغ ماهانه یک feature است.

در این دوره: featureهای داده‌ی churn شامل `tenure` و `monthly_charges` هستند که در شبیه‌سازی drift آن‌ها را جابه‌جا می‌کنیم (جلسه‌ی ۴ و ۱۳).

همچنین ببینید: Label / Target، Dataset، Data drift

3.5 Label / Target · برچسب / هدف

Label مقداری است که مدل باید پیش‌بینی کند و در داده‌ی آموزش از قبل معلوم است. در هر نمونه Feature ها «ورودی» و Label «جواب درست» هستند.

> **در این دوره:** ستون هدف در داده‌ی دوره `churn` است (۱ یعنی مشتری رفت، ۰ یعنی ماند)؛ در `train.py` با `TARGET = "churn"` جدا می‌شود (جلسه‌ی ۱).

همچنین ببینید: Feature، Dataset، Churn

3.6 Dataset · مجموعه‌داده (دیتاست)

Dataset جدولی از نمونه‌هاست که هر سطر یک مورد (مثلاً یک مشتری) و هر ستون یک feature یا label است. کیفیت و نسخه‌ی داده به‌اندازه‌ی کد در نتیجه اثر دارد.

> **در این دوره:** دو نسخه‌ی داده دارد: v1 با ۵۰۰۰ سطر و v2 با ۸۰۰۰ سطر و جمعیت تغییریافته که با `make data` و `make data-v2` ساخته می‌شوند (جلسه‌ی ۴).

همچنین ببینید: Feature، Label / Target، Pointer file (.dvc)، CSV

3.7 Train/test split · تقسیم داده به آموزش و آزمون

قبل از آموزش، بخشی از داده کنار گذاشته می‌شود (مجموعه‌ی test) و مدل هرگز آن را نمی‌بیند. بعداً کیفیت مدل روی همین بخش سنجیده می‌شود، مثل امتحان از سؤال‌هایی که در تمرین‌ها نبوده‌اند.

> **در این دوره:** در `configs/params.yaml` مقدار `test_size: 0.2` است، یعنی ۲۰٪ داده برای آزمون کنار می‌رود (جلسه‌ی ۱).

همچنین ببینید: Overfitting، Seed، Metric

3.8 Overfitting · بیش‌برازش (اُوِرفیتینگ)

وقتی مدل داده‌ی آموزش را «حفظ» می‌کند به‌جای اینکه الگوی کلی را یاد بگیرد، روی همان داده عالی ولی روی داده‌ی جدید ضعیف عمل می‌کند. مثل دانش‌آموزی که جواب سؤال‌های تمرین را حفظ کرده و در امتحان با سؤال جدید می‌ماند.

> **در این دوره:** برای تشخیص آن، متریک را روی مجموعه‌ی test می‌گیریم، نه روی داده‌ی آموزش (جلسه‌ی ۱).

همچنین ببینید: Train/test split، Metric، Hyperparameter

3.9 Parameter (weight) · پارامتر / وزن

Parameterهای مدل اعدادی هستند که خود الگوریتم هنگام آموزش از روی داده پیدا می‌کند؛ در logistic regression به آن‌ها وزن (weight) هم می‌گویند. شما آن‌ها را دستی تعیین نمی‌کنید و نتیجه‌ی آموزش‌اند. این با hyperparameter متفاوت است.

> **در این دوره:** وزن‌ها داخل فایل مدل ذخیره می‌شوند؛ ولی در `params.yaml` نه، چون آن‌جا فقط مقدارهایی هستند که شما انتخاب می‌کنید (جلسه‌ی ۱ و ۵).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Parameter (weight) و Hyperparameter

همچنین ببینید: Hyperparameter، Model، Training

3.10 Hyperparameter · ابرپارامتر (هایپرپارامتر)

Hyperparameter تنظیمی است که قبل از آموزش خودتان انتخاب می‌کنید و بر چگونگی یادگیری اثر دارد. مثل تنظیم شعله‌ی اجاق: روی پخت اثر می‌گذارد ولی خودش از غذا یاد گرفته نمی‌شود.

در این دوره: در `params.yaml` بخش `train` مقدارهای `C: 1.0` و `max_iter: 500` را دارد؛ در جلسه‌ی ۵ آن‌ها پارامتر مرحله‌ی train در DVC می‌شوند و با `dvc params diff` مقایسه می‌شوند.

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Parameter (weight) و Hyperparameter

همچنین ببینید: Parameter (weight)، Params (params.yaml)، Experiment tracking

3.11 Seed · بذر تصادفی (سید)

بسیاری از مراحل ML (مثل جداکردن داده‌ی test) تصادفی‌اند. Seed عددی ثابت است که تصادف را «قابل‌تکرار» می‌کند؛ با همان seed همیشه همان تقسیم و همان نتیجه را می‌گیرید.

در این دوره: `seed: 42` در `params.yaml` به `train_test_split(random_state=...)` می‌رود و ساخت داده با `make data` هر بار دقیقاً همان فایل را می‌دهد (جلسه‌ی ۱ و ۴).

همچنین ببینید: Reproducibility، Train/test split

3.12 Baseline · مدل پایه (بیس‌لاین)

Baseline ساده‌ترین مدل قابل‌قبول است که مدل‌های بعدی با آن مقایسه می‌شوند. اگر مدل پیچیده‌تر از آن بهتر نباشد، پیچیدگی اضافی ارزشی ندارد.

در این دوره: baseline دوره logistic regression با `roc_auc 0.802` و `f1 0.5969` است که در تگ `v0.1.0` ثبت شده (جلسه‌ی ۱ و ۲).

همچنین ببینید: Logistic regression، Metric، ROC-AUC

3.13 Metric · معیار ارزیابی (متریک)

Metric یک عدد است که کیفیت مدل را خلاصه می‌کند (مثل ROC-AUC یا F1). بدون متریک نمی‌توان گفت مدل جدید بهتر از قبلی است یا نه.

در این دوره: خروجی آموزش در `metrics.json` ذخیره می‌شود و در جلسه‌ی ۵ تا ۷ با `dvc metrics diff` و MLflow مقایسه و در promotion به‌عنوان آستانه استفاده می‌شود. (متریک در پایش سیستم معنای دیگری دارد؛ به «Metric (Prometheus)» نگاه کنید.)

همچنین ببینید: ROC-AUC، F1 score، Evaluation gate، Metric (Prometheus)

3.14 ROC-AUC · سطح زیر منحنی ROC (آراوسی ای‌یوسی)

ROC-AUC نشان می‌دهد مدل تا چه حد مشتریان پرخطر را بالاتر از کم‌خطر رتبه‌بندی می‌کند، بدون وابستگی به یک آستانه‌ی خاص. عدد ۰٫۵ یعنی حدس شانسی و ۱ یعنی رتبه‌بندی کامل.

در این دوره: baseline نسخه‌ی ۱ داده `0.802` گرفت؛ روی داده‌ی v2 با drift به `0.7493` افت کرد (جلسه‌ی ۴).

همچنین ببینید: Metric، F1 score، Threshold

3.15 Precision · دقت (precision) (پرسیژن)

از میان مشتریانی که مدل «ریزش می‌کند» گفته، چند درصد واقعاً رفته‌اند؟ Precision بالا یعنی هشدارهای مدل کمتر بی‌جاست. مثل اینکه از ده هشدار آتش، چند تا واقعاً آتش بوده.

> **در این دوره:** در F1 ترکیب می‌شود و با تغییر threshold اثر می‌پذیرد (جلسه‌ی ۱).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Precision و Recall

همچنین ببینید: Recall، F1 score، Confusion matrix

3.16 Recall · بازیابی (recall) (ریکال)

از میان مشتریانی که واقعاً رفته‌اند، چند درصد را مدل پیدا کرد؟ Recall بالا یعنی کمتر ریزش را از دست می‌دهیم. مثل اینکه از همه‌ی آتش‌سوزی‌های واقعی، چند تا را آلارم تشخیص داد.

> **در این دوره:** در مسئله‌ی churn معمولاً از دست دادن مشتری گران‌تر از یک هشدار اضافه است، پس recall مهم است (جلسه‌ی ۱).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Precision و Recall

همچنین ببینید: Precision، F1 score، Threshold

3.17 F1 score · امتیاز F1 (اِف‌وان)

F1 میانگین هارمونیک precision و recall است و فقط وقتی بالاست که هر دو با هم بالا باشند. با یک threshold ثابت محاسبه می‌شود.

> **در این دوره:** baseline با threshold `0.5` عدد `f1 = 0.5969` را گرفته است (جلسه‌ی ۱).

همچنین ببینید: Precision، Recall، Threshold

3.18 Threshold · آستانه

مدل احتمال می‌دهد (مثلاً ۰٫۷)، ولی برای تصمیم «ریزش/عدم ریزش» باید خطی گذاشت. آستانه همین خط است: احتمال بالاتر از آن یعنی «ریزش». بالا بردن آن precision را بالا و recall را پایین می‌آورد و برعکس.

> **در این دوره:** `eval.threshold: 0.5` در `params.yaml` برای محاسبه‌ی F1 استفاده می‌شود؛ در جلسه‌ی ۷ مفهوم متفاوتی هم دارد: آستانه‌ی متریک برای اجازه‌ی promotion (evaluation gate).

همچنین ببینید: Precision، Recall، Evaluation gate

3.19 Confusion matrix · ماتریس درهم‌ریختگی

جدولی ۲×۲ که چهار حالت را می‌شمارد: درست مثبت، نادرست مثبت، درست منفی و نادرست منفی. precision و recall هر دو از همین چهار عدد حساب می‌شوند و جدول نشان می‌دهد مدل کجا اشتباه می‌کند.

> **در این دوره:** در دوره به‌صورت مفهومی برای فهم precision و recall به کار می‌رود.

همچنین ببینید: Precision، Recall، F1 score

3.20 Classification · دسته‌بندی

مسئله‌ای که در آن مدل باید یکی از چند دسته را انتخاب کند. اگر فقط دو دسته داشته باشیم (بله/خیر) به آن دسته‌بندی دودویی می‌گوییم.

> **در این دوره:** پیش‌بینی ریزش مشتری یک مسئله‌ی دسته‌بندی دودویی است (جلسه‌ی ۱).

همچنین ببینید: Churn، Logistic regression، Label / Target

3.21 Churn · ریزش مشتری (چرن)

Churn یعنی مشتری اشتراک یا خدمتش را قطع کند. پیش‌بینی آن به کسب‌وکار کمک می‌کند قبل از رفتن مشتری با او تماس بگیرد.

> **در این دوره:** پروژه‌ی تجمعی دوره یک مدل churn روی داده‌ی جدولی است که در ۱۴ جلسه از Git تا Kubernetes می‌رسد (جلسه‌ی ۱ تا ۱۴).

همچنین ببینید: Classification، Label / Target، Dataset

3.22 Logistic regression · رگرسیون لجستیک (لجستیک رگرشن)

Logistic regression یک مدل ساده و سریع برای دسته‌بندی دودویی است که هر ویژگی را در یک وزن ضرب می‌کند و مجموع را به احتمالی بین ۰ و ۱ تبدیل می‌کند. با وجود نام «regression»، برای دسته‌بندی به کار می‌رود.

> **در این دوره:** مدل دوره `LogisticRegression(C=1.0, max_iter=500)` از scikit-learn است و روی CPU در چند ثانیه آموزش می‌بیند (جلسه‌ی ۱).

همچنین ببینید: Baseline، scikit-learn، Hyperparameter

3.23 scikit-learn · — (سایکیت‌لرن)

scikit-learn کتابخانه‌ی Python برای ML کلاسیک است: مدل‌های ساده، پیش‌پردازش داده و سنجش متریک. برای داده‌ی جدولی استاندارد صنعتی است.

> **در این دوره:** نسخه‌ی پین‌شده `1.9.1` است و `train.py` از آن LogisticRegression، `train_test_split`، `roc_auc_score` و `f1_score` را می‌گیرد (جلسه‌ی ۱).

همچنین ببینید: Logistic regression، Pipeline (scikit-learn)، Metric

3.24 Pipeline (scikit-learn) · خط لوله‌ی scikit-learn

در scikit-learn، Pipeline مراحل پیش‌پردازش و مدل را در یک شیء واحد زنجیر می‌کند تا همان مراحل در آموزش و پیش‌بینی تکرار شوند. این غیر از «pipeline در DVC» و «pipeline در CI» است که زنجیره‌ی مرحله‌های کاری‌اند.

> **در این دوره:** در `train.py` یک `Pipeline` ساخته شده که مرحله‌ی `pre` (مقیاس‌دهی و one-hot) و مرحله‌ی `clf` (مدل) را دارد (جلسه‌ی ۱).

همچنین ببینید: Preprocessing (scaling, one-hot)، Train/serve skew، DVC pipeline / stage، CI pipeline

3.25 Preprocessing (scaling, one-hot) · پیش‌پردازش (مقیاس‌دهی، one-hot)

مدل فقط عدد می‌فهمد و به مقیاس اعداد حساس است. مقیاس‌دهی (StandardScaler) ستون‌های عددی را هم‌اندازه می‌کند و one-hot ستون متنی را به چند ستون صفر/یک تبدیل می‌کند.

> **در این دوره:** در `build_model` از `StandardScaler` برای ستون‌های عددی و `OneHotEncoder` برای ستون‌های دسته‌ای استفاده شده است (جلسه‌ی ۱).

همچنین ببینید: Pipeline (scikit-learn)، Feature

3.26 Serialization (joblib / pickle) · سریال‌سازی مدل

برای اینکه مدلِ آموزش‌دیده را بعداً دوباره بارگذاری کنیم، آن را به یک فایل تبدیل می‌کنیم؛ به این کار serialization می‌گویند و `joblib` و `pickle` ابزارهای Python برای آن‌اند. فایل pickle را هرگز از منبع نامطمئن باز نکنید، چون می‌تواند کد اجرا کند.

> **در این دوره:** `train.py` مدل را با `joblib.dump` در `models/model.joblib` می‌نویسد؛ از جلسه‌ی ۶ فایل مدل در MLflow و registry نگهداری می‌شود.

همچنین ببینید: Model، Artifact، Model registry

3.27 Train/serve skew · ناهماهنگی آموزش و سرویس‌دهی

وقتی داده‌ای که هنگام سرویس‌دهی به مدل می‌رسد با آنچه در آموزش دیده (از نظر ستون‌ها، نوع یا پیش‌پردازش) فرق دارد، پیش‌بینی خراب می‌شود بی‌آنکه خطا بدهد. راه جلوگیری این است که شکل ورودی یک قرارداد مشخص داشته باشد.

> **در این دوره:** در جلسه‌ی ۷ و ۸ شکل ورودی هم در signature مدل در MLflow و هم در schema مدل pydantic در API تعریف می‌شود تا این دو با هم بخوانند.

همچنین ببینید: Model signature، Request / Response schema، Pipeline (scikit-learn)

3.28 SHAP · — (شپ)

SHAP روشی برای توضیح پیش‌بینی مدل است که نشان می‌دهد هر feature چقدر در یک پیش‌بینی نقش داشته. به این حوزه «هوش مصنوعی قابل‌توضیح» (explainable AI) می‌گویند.

> **در این دوره:** اختیاری: در جلسه‌ی ۷ می‌توان نمودار خلاصه‌ی SHAP را به‌عنوان artifact در MLflow ثبت کرد.

همچنین ببینید: Artifact، Model card / Release note

4 مفاهیم MLOps

MLOps Concepts

این فصل «چرا» پشت ابزارهاست: چه مشکلی باعث پیدایش MLOps شد، هدف‌ها چیست و دوره از چه سطحی شروع و به کجا ختم می‌شود. ابزارها عوض می‌شوند ولی این مفاهیم ثابت می‌مانند.

جلسه‌ها: جلسه ۱، ۴، ۵، ۱۴ · **تعداد واژه‌ها:** ۲۰

4.1 MLOps · عملیات یادگیری ماشین (ام‌ال‌آپس)

MLOps مجموعه‌ی روش‌ها و ابزارهایی است که ساختن مدل را به یک مسیر قابل‌اعتماد از داده تا تولید تبدیل می‌کند: نسخه‌بندی، آزمایش خودکار، استقرار، پایش و آموزش دوباره. هدفش «ساختن مدل بهتر» نیست؛ هدف این است که مدل هر روز در محیط واقعی درست کار کند.

> **در این دوره:** قول دوره: در پایان هر نفر یک Repo دارد که مسیر «داده ← آموزش ← ثبت مدل ← سرویس ← Kubernetes ← پایش» را اجرا می‌کند (جلسه‌ی ۱).

همچنین ببینید: ML lifecycle، MLOps maturity levels، DevOps

4.2 DevOps · دواپس (دواپس)

DevOps فرهنگ و روشی است که توسعه (Dev) و عملیات (Ops) نرم‌افزار را به هم نزدیک می‌کند تا انتشار سریع، مکرر و امن شود. MLOps همین ایده را به پروژه‌های ML می‌برد و داده و مدل را هم به کد اضافه می‌کند.

> **در این دوره:** در نقش‌ها، «DevOps/Platform» مسئول زیرساخت مشترک (شبکه، Kubernetes، امنیت) و «MLOps Engineer» مسئول مسیر ML است (جلسه‌ی ۱).

همچنین ببینید: MLOps، Data Scientist / ML Engineer / MLOps Engineer، CI (Continuous Integration)

4.3 ML lifecycle · چرخه‌ی عمر یادگیری ماشین

مراحل یک پروژه‌ی ML به‌ترتیب: جمع‌آوری داده، آماده‌سازی، آموزش، ارزیابی، استقرار، پایش و بازگشت به آموزش با داده‌ی جدید. نکته‌ی کلیدی این است که چرخه است نه یک خط مستقیم؛ استقرار پایان کار نیست.

> **در این دوره:** جلسه‌ی ۱ این چرخه را معرفی می‌کند و جلسه‌های بعدی هر قطعه را با یک ابزار پر می‌کنند.

همچنین ببینید: MLOps، Retraining، The 7 MLOps components

4.4 Data Scientist / ML Engineer / MLOps Engineer · نقش‌ها

Data Scientist الگو را کشف و مدل را می‌سازد (معمولاً در Notebook)، ML Engineer کد را به کد تولیدی تمیز و قابل‌تست تبدیل می‌کند، و MLOps Engineer مسیر پایدار از داده تا تولید را می‌سازد و نگه می‌دارد. در شرکت‌های کوچک اغلب یک نفر چند نقش را دارد.

> **در این دوره:** تمرکز این دوره نقش MLOps Engineer است (جلسه‌ی ۱).

همچنین ببینید: DevOps، MLOps

4.5 The 7 MLOps components · هفت جزء MLOps

چارچوبی که دوره برای مرور استفاده می‌کند: ۱) Version Control، ۲) CI/CD، ۳) Orchestration، ۴) Experiment Tracking، ۵) Data Lineage، ۶) Serving & Training، ۷) Observability & Monitoring. هر جلسه یکی از این‌ها را می‌سازد.

> **در این دوره:** جدول «هفت جزء ← جلسه‌ها» در جلسه‌ی ۱ و جمع‌بندی نهایی جلسه‌ی ۱۴ آمده است و دانشجو خودارزیابی هم می‌کند.

همچنین ببینید: MLOps maturity levels، Data lineage، Orchestration

4.6 Reproducibility · تکرارپذیری

یعنی هر کس، هر زمان و روی هر ماشین با همان کد، داده، پارامتر و محیط، همان نتیجه را بگیرد. بدون آن نمی‌توان خطا را یافت، نتیجه را ثابت کرد یا به نسخه‌ی قبلی برگشت.

> **در این دوره:** سه پایه‌ی آن در دوره: Git برای کد، DVC برای داده و pipeline، و `uv.lock` برای محیط؛ آزمون نهایی «clean clone» است (جلسه‌ی ۱، ۴ و ۱۴).

همچنین ببینید: Lockfile (uv.lock)، Seed، Clean clone، Data lineage

4.7 Data lineage · شجره‌ی داده (لینیج)

ثبت اینکه هر خروجی (مثل یک مدل) از کدام داده، با کدام کد و کدام پارامتر ساخته شده؛ مثل شجره‌نامه. در ممیزی می‌توان پرسید «این مدل بر چه داده‌ای تصمیم گرفت؟».

> **در این دوره:** فایل‌های `dvc.lock` و `dvc dag` در جلسه‌ی ۵ و برچسب نسخه‌ی داده روی مدل در جلسه‌ی ۷ شجره‌ی مدل را می‌سازند (جلسه‌ی ۴ تا ۷).

همچنین ببینید: Traceability، Reproducibility، dvc.lock

4.8 Traceability · ردیابی‌پذیری

توانایی برگشتن از یک چیز در تولید (مثلاً مدلی که الان جواب می‌دهد) به همه‌ی منشأهایش: commit، نسخه‌ی داده، run آزمایش و image. lineage نقشه‌ی منشأهاست و traceability توانایی دنبال کردن آن.

> **در این دوره:** از `models:/churn@champion` می‌توان به run در MLflow، commit در Git و نسخه‌ی داده در DVC رسید (جلسه‌ی ۷).

همچنین ببینید: Data lineage، Model registry

4.9 MLOps maturity levels · سطوح بلوغ MLOps

سه سطح برای سنجیدن وضعیت یک تیم: سطح ۰ دستی (Notebook ← فایل pickle ← استقرار دستی)، سطح ۱ خودکارسازی pipeline (داده نسخه‌دار، pipeline تکرارپذیر، ردیابی، registry، CI و پایش) و سطح ۲ خودکارسازی CI/CD کامل (GitOps، promotion خودکار و آموزش دوباره با drift). از روی این سطح‌ها می‌توان فهمید تیم یا پروژه الان کجاست و قدم بعدی چیست.

> **در این دوره:** دانشجو در شروع جلسه‌ی ۱ در سطح ۰ است و با پایان این دوره به سطح ۱ می‌رسد؛ سطح ۲ در دوره‌ی بعدی است.

همچنین ببینید: The 7 MLOps components، Technical debt (in ML)

4.10 Technical debt (in ML) · بدهی فنی

هر میان‌بُر امروز، مثل مدل بدون نسخه یا اسکریپت دستی، در آینده هزینه‌ی نگه‌داری می‌شود؛ مثل وام که سودش با زمان زیاد می‌شود. سیستم‌های ML به‌علاوه‌ی کد، به داده و پیکربندی هم وابسته‌اند و بدهی پنهان بیشتری دارند.

در این دوره: مصداق‌ها: آموزش بدون seed، داده بدون نسخه، رمز داخل کد و استقرار دستی؛ هر جلسه یکی از این‌ها را می‌بندد.

همچنین ببینید: MLOps maturity levels، Reproducibility

4.11 On-premises (on-prem) · درون‌سازمانی (آن‌پرِم)

On-prem یعنی نرم‌افزار و داده روی سرورهای خود سازمان اجرا شود و به سرویس ابری بیرونی وابسته نباشد. دلیلش معمولاً امنیت، قانون یا محدودیت شبکه است.

در این دوره: همه‌ی دوره on-prem است: لپ‌تاپ هر نفر نقش یک «دیتاسنتر کوچک» دارد و به‌جای S3 آمازون از SeaweedFS استفاده می‌کنیم (جلسه‌ی ۱).

همچنین ببینید: Air-gapped / Offline، Object storage

4.12 Air-gapped / Offline · آفلاین (جداشده از اینترنت)

محیط air-gapped کاملاً از اینترنت جداست. هر چیزی که لازم است (برنامه، image، بسته‌ی Python، پایگاه‌داده‌ی امنیتی) باید قبلاً جمع‌آوری و با فلش یا شبکه‌ی داخلی منتقل شده باشد.

در این دوره: «بسته‌ی آفلاین» (`bundle-amd64` یا `bundle-arm64`، حدود ۵ گیگابایت) شامل ابزارها، wheelhouse، imageها و داده است؛ نصب با `./scripts/install-from-bundle.sh` و تأیید با `preflight.sh` انجام می‌شود (جلسه‌ی ۰).

همچنین ببینید: On-premises (on-prem)، Wheelhouse، Container registry

4.13 Orchestration · ارکستراسیون

اجرای مراحل کار به ترتیب درست، با زمان‌بندی، تکرار و کنترل خطا. مثل رهبر ارکستر که مشخص می‌کند هر ساز کی وارد شود. ابزارهای شناخته‌شده: Airflow، Kubeflow و Argo.

در این دوره: دوره از DVC (برای ترتیب مرحله‌ها) و یک زمان‌بندی cron در Gitea Actions (برای آموزش دوباره) شروع می‌کند، یعنی «orchestration سبک» (جلسه‌ی ۵ و ۱۳).

همچنین ببینید: DAG، DVC pipeline / stage، Scheduled workflow (cron)

4.14 Release · انتشار / نسخه‌ی منتشرشده

یک نسخه‌ی مشخص و نام‌دار از نرم‌افزار یا مدل که آماده‌ی استفاده است. در دوره هم کد و هم مدل «نسخه‌ی منتشرشده» دارند.

در این دوره: تگ `v0.1.0` اولین release کد است؛ مدل با نسخه و alias در registry منتشر می‌شود (جلسه‌ی ۲ و ۷).

همچنین ببینید: SemVer، Tag، Rollback

4.15 Rollback · بازگشت به نسخه‌ی قبل

برگشتن سریع به آخرین نسخه‌ی سالم وقتی نسخه‌ی جدید مشکل دارد. هر استقرار خوب باید یک راه برگشت ساده و از قبل تمرین‌شده داشته باشد.

```
kubectl rollout undo deployment/churn-api
```

در این دوره: چهار بار تمرین می‌شود: promotion مدل (جلسه‌ی ۷)، اجرای دوباره‌ی Ansible با نسخه‌ی قبلی (جلسه‌ی ۱۰)، `kubectl rollout undo` (جلسه‌ی ۱۱) و `helm rollback` (جلسه‌ی ۱۲).

همچنین ببینید: Rollout، Release (Helm)، Ansible، Alias (@champion / @challenger)

4.16 Definition of Done (DoD) · تعریف انجام‌شده

فهرست شرایطی که اگر همه برقرار باشند، آن کار «تمام‌شده» حساب می‌شود. به‌جای احساس، با یک چک‌لیست قابل‌بررسی می‌سنجیم.

در این دوره: هر جلسه یک DoD دارد و `make verify-sNN` (مثلاً `dvc status` تمیز باشد یا `/health` کد ۲۰۰ بدهد) آن را خودکار بررسی می‌کند.

همچنین ببینید: Clean clone، Golden repo / Checkpoint branch

4.17 Clean clone · کلون تمیز

Repo را در یک پوشه‌ی کاملاً جدید clone کنید و فقط با دستورهای README اجرا کنید. اگر کار کرد، پروژه به فایل‌های جاافتاده روی لپ‌تاپ سازنده وابسته نیست.

در این دوره: ۱۵ نمره‌ی پروژه‌ی نهایی «اجرا از clean clone» است؛ آزمون نهایی جلسه‌ی ۱۴ همین است.

همچنین ببینید: Reproducibility، Definition of Done (DoD)، clone

4.18 Golden repo / Checkpoint branch · مخزن مرجع / شاخه‌ی نقطه‌ی بازگشت

Golden repo نسخه‌ی مرجع مدرس از پروژه است که در پایان هر جلسه یک tag می‌خورد (`s01-end` تا `s14-end`). اگر دانشجویی عقب افتاد یا کارش خراب شد، از همان نقطه شروع می‌کند.

در این دوره: مثلاً `git checkout s05-end`؛ شاخه‌های `checkpoint/s05-end` هم برای همین‌اند (جلسه‌ی ۱ تا ۱۴).

همچنین ببینید: Tag، Definition of Done (DoD)

4.19 Batch vs Stream · دسته‌ای در برابر جریانی

در پردازش Batch داده در دسته‌های زمان‌بندی‌شده (مثلاً هر شب ساعت ۲) منتقل می‌شود؛ ساده و ارزان. در Stream هر رویداد در لحظه‌ی رخ دادن پردازش می‌شود که سریع‌تر ولی پیچیده‌تر است.

در این دوره: داده‌ی دوره یک snapshot ثابت است و پردازش دسته‌ای دارد؛ Stream فقط در یک بلوک مفهومی ۱۵ دقیقه‌ای معرفی می‌شود (جلسه‌ی ۴).

همچنین ببینید: Data lake / Data warehouse، Snapshot

4.20 Data lake / Data warehouse · دریاچه‌ی داده / انبار داده

Data lake انباری ارزان از فایل‌های خام با ساختار آزاد (CSV، Parquet، تصویر) روی ذخیره‌ساز شیء است. Data warehouse جدول‌های تمیز و ساخت‌یافته برای تحلیل و گزارش است. داده‌ی ML معمولاً از lake می‌آید.

در این دوره: باکت‌های `dvc` و `mlflow` در SeaweedFS نمونه‌ای کوچک از «lake» اند (جلسه‌ی ۴).

همچنین ببینید: Object storage، Batch vs Stream، Bucket

5 Git و کار تیمی

Git & Teamwork

Git تاریخچه‌ی کد را نگه می‌دارد و کار گروهی را ممکن می‌کند؛ Gitea همان Git را روی سرور خود سازمان ارائه می‌دهد. پایه‌ی بقیه‌ی دوره (DVC، CI، استقرار) روی همین مفاهیم ساخته می‌شود.

جلسه‌ها: جلسه ۲ (و ۴، ۱۳) · **تعداد واژه‌ها:** ۲۵

5.1 Git · گیت

Git سامانه‌ی کنترل نسخه است: تاریخچه‌ی کامل تغییرات یک پروژه را نگه می‌دارد، اجازه می‌دهد به هر نقطه برگردید و چند نفر هم‌زمان روی یک کد کار کنند. مثل قابلیت «تاریخچه‌ی نسخه‌ها» در یک سند، ولی برای کل پوشه‌ی پروژه.

> **در این دوره:** پروژه‌ی دوره با `git init -b main` شروع می‌شود و تا پایان همه‌چیز (کد، `dvc.yaml`، `Dockerfile`، chart) در همین Repo است (جلسه‌ی ۱ و ۲).

همچنین ببینید: Repository (Repo)، Commit، Gitea

5.2 Repository (Repo) · مخزن (ریپازیتوری)

Repo پوشه‌ی پروژه به‌همراه تمام تاریخچه‌ی Git آن است که در پوشه‌ی پنهان `.git` نگهداری می‌شود. هر Repo معمولاً یک پروژه است.

> **در این دوره:** Repo دوره ساختار `src/ configs/ pipelines/ infra/ tests/ scripts/` دارد و روی Gitea با نشانی `localhost:3000` هم نگهداری می‌شود (جلسه‌ی ۱ و ۲).

همچنین ببینید: Git، clone، Remote (git remote / origin)

5.3 Commit · کامیت (ثبت تغییر) (کامیت)

Commit یک «عکس» مشخص از وضعیت کل پروژه است که با پیام توضیحی و شناسه‌ی یکتا (hash) ثبت می‌شود. هر commit یک نقطه‌ی امن برای برگشتن است.

```
git commit -m "feat: track churn dataset with DVC"
```

> **در این دوره:** پیام‌ها با قالب Conventional Commits نوشته می‌شوند، مثل `chore: add ruff and pytest config` یا `feat: track churn dataset with DVC` (جلسه‌ی ۲ و ۴).

همچنین ببینید: Commit hash، Staging area (git add)، Conventional Commits

5.4 Commit hash · شناسه‌ی commit

رشته‌ای طولانی از حروف و ارقام که از محتوای commit ساخته می‌شود و هر commit را یکتا می‌شناسد. معمولاً هفت نویسه‌ی اول کافی است.

```
git rev-parse --short HEAD
```

در این دوره: برای گزارش دقیق نسخه، commit hash را در خروجی چک‌پوینت می‌گذاریم (`git rev-parse --short HEAD`) و در MLflow هم به run وصل می‌شود (جلسه‌ی ۱ و ۶).

همچنین ببینید: Commit، Traceability

5.5 Staging area (git add) · ناحیه‌ی آماده‌سازی (استیجینگ)

قبل از commit، با `git add` تعیین می‌کنید کدام تغییرات وارد commit بعدی شوند؛ این «ناحیه‌ی انتظار» staging نام دارد. مثل چیدن اقلام در سبد خرید قبل از پرداخت.

در این دوره: `git status --short` نشان می‌دهد چه چیزی stage شده است؛ بعد از `dvc add` باید فایل pointer را stage کنید، نه خود داده را (جلسه‌ی ۲ و ۴).

همچنین ببینید: Commit، Pointer file (.dvc)

5.6 Working tree · پوشه‌ی کاری

فایل‌هایی که همین الان در پوشه‌ی پروژه می‌بینید و ویرایش می‌کنید. تفاوتش با تاریخچه‌ی Git این است که ممکن است هنوز commit نشده باشد.

در این دوره: همین مفهوم در DVC هم هست: `dvc status` نشان می‌دهد داده‌ی پوشه‌ی کاری با فایل pointer می‌خواند یا نه (جلسه‌ی ۴).

همچنین ببینید: Staging area (git add)، Commit، dvc checkout

5.7 Branch · شاخه (برنچ)

Branch یک خط موازی از تاریخچه است که روی آن می‌توانید بدون خراب کردن شاخه‌ی اصلی تغییر بدهید. در واقع یک اشاره‌گر متحرک به آخرین commit آن خط است و با هر commit جلو می‌رود.

```
git switch -c feature/quality-gates
```

در این دوره: هر تغییر کوچک یک branch با نام `feature/...` می‌گیرد، مثل `feature/quality-gates` در Lab جلسه‌ی ۲.

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Tag (Git) و Branch

همچنین ببینید: Merge، main، Tag، Branching strategy (feature / release / hotfix)

5.8 main · شاخه‌ی اصلی

شاخه‌ی اصلی پروژه که همیشه باید سالم باشد، یعنی هرچه روی آن است قابل اجرا و تست‌شده است. تغییرات فقط از راه merge به آن می‌رسند.

در این دوره: در Gitea می‌توان با branch protection مستقیم push روی main را بست؛ در دوره اختیاری است (جلسه‌ی ۲).

همچنین ببینید: Branch، Branch protection، Pull Request (PR)

5.9 Merge · ادغام (مرج)

Merge تغییرات یک branch را به branch دیگر اضافه می‌کند. اگر هر دو یک خط را تغییر داده باشند، Git conflict (تعارض) گزارش می‌کند و شما باید دستی تصمیم بگیرید.

> **در این دوره:** در دوره merge معمولاً با دکمه‌ی Merge در Pull Request روی Gitea انجام می‌شود و rebase فقط برای عیب‌یابی اشاره می‌شود (جلسه‌ی ۲).

همچنین ببینید: Pull Request (PR)، Branch، Merge conflict

5.10 Merge conflict · تعارض ادغام

وقتی Git نمی‌تواند خودکار تصمیم بگیرد که از دو نسخه‌ی یک بخش از فایل کدام را نگه دارد، نشانه‌گذاری می‌کند و از شما می‌خواهد انتخاب کنید. ترسناک نیست: فقط یعنی دو نفر یک خط را تغییر داده‌اند.

> **در این دوره:** با branchهای کوتاه‌عمر و Pull Requestهای کوچک تعارض‌ها کم می‌شوند (جلسه‌ی ۲).

همچنین ببینید: Merge، Branch

5.11 Remote (git remote / origin) · مخزن راه‌دور (ریموت)

Remote نسخه‌ای از Repo روی سرور (مثل Gitea) است که تیم از طریق آن کد را به اشتراک می‌گذارد. نام قراردادی remote اصلی `origin` است. توجه: «remote» در DVC چیز دیگری است.

> **در این دوره:** `git remote add origin` سپس `git push -u origin main` اولین ارسال به Gitea است؛ ولی remote در DVC یک bucket روی SeaweedFS است (جلسه‌ی ۲ و ۴).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی remote (Git) و remote (DVC)

همچنین ببینید: push / pull / fetch، clone، Remote (DVC)

5.12 clone · کپی کامل Repo (کلون)

`git clone` یک کپی کامل از Repo را با کل تاریخچه روی ماشین شما می‌سازد. بعد از clone می‌توانید بدون اینترنت هم تاریخچه را ببینید و روی کد کار کنید.

> **در این دوره:** دانشجو Repo پروژه را از Gitea clone می‌کند؛ آزمون «clean clone» هم از همین است (جلسه‌ی ۲ و ۱۴).

همچنین ببینید: Repository (Repo)، Clean clone، Remote (git remote / origin)

5.13 push / pull / fetch · ارسال / دریافت

`git push` commitهای محلی را به remote می‌فرستد؛ `git fetch` فقط تغییرات remote را می‌گیرد و `git pull` هم fetch و هم merge را انجام می‌دهد. قبل از شروع کار روز، pull کنید.

> **در این دوره:** در `git push -u origin feature/quality-gates` گزینه‌ی u شاخه‌ی محلی را به شاخه‌ی راه‌دور وصل می‌کند؛ هر push تست‌ها را با hook اجرا می‌کند (جلسه‌ی ۲).

همچنین ببینید: Remote (git remote / origin)، Merge، Git hook

5.14 Pull Request (PR) · درخواست ادغام

PR درخواستی برای ادغام یک branch در main همراه با بازبینی همکاران است. هم جای گفت‌وگو درباره‌ی تغییر است و هم ثبت دائمی «چرا». در GitLab به آن Merge Request می‌گویند.

> **در این دوره:** در جلسه‌ی ۲ هر دانشجو یک PR با عنوان به قالب Conventional Commits روی Gitea باز می‌کند و بعد از merge، main را pull می‌کند.

همچنین ببینید: Code review، Merge، CI (Continuous Integration)

5.15 Code review · بازبینی کد

همکار تغییر را قبل از ادغام می‌خواند، اشتباه یا ابهام را گزارش می‌کند و یا تأیید (approve) می‌دهد. هدف فقط پیدا کردن خطا نیست؛ انتقال دانش تیم هم هست.

> **در این دوره:** در PR دوره reviewer و author با هم نقش‌ها را عوض می‌کنند و نتیجه یکی از approve یا request changes است (جلسه‌ی ۲).

همچنین ببینید: Pull Request (PR)

5.16 Tag · برچسب نسخه (تگ)

Tag یک نام ثابت برای یک commit مشخص است، مثل «v0.1.0»؛ برخلاف branch جلو نمی‌رود. برای علامت‌گذاری نسخه‌های منتشرشده به کار می‌رود. Annotated tag پیام، نویسنده و تاریخ هم دارد.

```
git tag -a v0.1.0 -m "Baseline: logistic regression, ROC-AUC 0.80"
```

> **در این دوره:** `v0.1.0` برای baseline، `data-v1` و `data-v2` برای نسخه‌های داده و `s01-end` تا `s14-end` برای نقطه‌های بازگشت جلسه‌ها (جلسه‌ی ۲ و ۴).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Tag و Branch (Git)

همچنین ببینید: SemVer، Branch، Release

5.17 SemVer · نسخه‌گذاری معنایی (سم‌ور)

قرارداد شماره‌گذاری نسخه به شکل `MAJOR.MINOR.PATCH`: ورژن MAJOR با تغییر ناسازگار، MINOR با قابلیت جدید سازگار و PATCH با رفع باگ بالا می‌رود. از روی عدد می‌فهمید ارتقا ریسک دارد یا نه.

> **در این دوره:** اولین release کد `v0.1.0` و نسخه‌های image و chart هم بعداً با همین شیوه برچسب می‌خورند (جلسه‌ی ۲، ۹ و ۱۲).

همچنین ببینید: Tag، Release، Image tag / digest

5.18 Conventional Commits · قرارداد پیام commit

قالبی برای پیام commit به شکل «نوع: توضیح»، که نوع آن یکی از `feat`، `fix`، `chore`، `docs`، `test` و مانند آن است. با این قالب تاریخچه خوانا می‌شود و می‌توان از روی آن خودکار فهرست تغییرات ساخت.

> **در این دوره:** نمونه‌ها: `feat: dataset v2 (8000 rows, drifted)` و `test: add unit tests` (جلسه‌ی ۲ و ۴).

همچنین ببینید: Commit، Pull Request (PR)

5.19 .gitignore · فایل نادیده‌گرفتن (گیت‌ایگنور)

فایلی که الگوی نام فایل‌هایی را می‌دهد که Git نباید ردیابی کند، مثل `.venv`، فایل‌های موقت یا فایل‌های حاوی رمز. با این فایل از اضافه شدن اتفاقی چیزهای بزرگ یا محرمانه به Repo جلوگیری می‌شود.

> **در این دوره:** `.env` در `.gitignore` است؛ از جلسه‌ی ۴ خود داده هم از Git خارج می‌شود، چون با DVC نسخه‌بندی می‌شود (جلسه‌ی ۲ و ۴).

همچنین ببینید: .env file، Pointer file (.dvc)، Secret

5.20 HEAD / detached HEAD · نقطه‌ی فعلی / HEAD جداشده (هد)

HEAD اشاره‌گر به commitی است که الان رویش هستید. اگر مستقیم روی یک tag یا commit بروید و روی هیچ branch نباشید، Git می‌گوید «detached HEAD»؛ این وضعیت خطرناک نیست و فقط یعنی کار شما به شاخه‌ای وصل نیست.

> **در این دوره:** با `git checkout data-v1` وارد این وضعیت می‌شوید و داده‌ی نسخه‌ی ۱ را با `dvc checkout` برمی‌گردانید (جلسه‌ی ۴).

همچنین ببینید: git checkout / git switch، Tag

5.21 git checkout / git switch · رفتن به شاخه یا نسخه

`git checkout` (و دستور جدیدتر `git switch` برای شاخه‌ها) پوشه‌ی کاری را روی یک branch، tag یا commit می‌برد. این فقط کد را عوض می‌کند و روی فایل‌های داده‌ی DVC اثری ندارد؛ برای آن‌ها `dvc checkout` لازم است.

```
git checkout data-v1
uv run dvc checkout
```

> **در این دوره:** `git checkout data-v1` و بعد `dvc checkout`: دو دستور برای برگشتن به نسخه‌ی ۱ داده (جلسه‌ی ۴).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی git checkout و dvc checkout

همچنین ببینید: dvc checkout، HEAD / detached HEAD، Branch

5.22 Branching strategy (feature / release / hotfix) · راهبرد شاخه‌بندی

قرارداد تیم برای نام و کاربرد شاخه‌ها: `feature/*` برای هر تغییر کوچک با عمر کوتاه، `release/*` برای آماده‌سازی نسخه، `hotfix/*` برای رفع فوری باگ از روی tag تولید. در تیم‌های کوچک ممکن است release حذف شود و مستقیم از main tag بزنند.

> **در این دوره:** دوره از `feature/*` و tag روی main استفاده می‌کند؛ روشی نزدیک به trunk-based development (جلسه‌ی ۲).

همچنین ببینید: Branch، Tag، main

5.23 Branch protection · حفاظت از شاخه

قابلیتی در Gitea که اجازه نمی‌دهد مستقیم روی شاخه‌ی مهم (مثلاً main) push شود و ادغام را فقط از راه PR و تأیید بازبین می‌پذیرد. مثل قفل کردن در اصلی ساختمان: فقط کسانی که از مسیر تعیین‌شده می‌آیند وارد می‌شوند.

> **در این دوره:** در Lab جلسه‌ی ۲ اختیاری است ولی در تیم واقعی بسیار توصیه می‌شود.

همچنین ببینید: main، Pull Request (PR)

5.24 Gitea · گیتی‌آ (گیتی)

Gitea سرور Git سبک و متن‌باز است که روی زیرساخت خود سازمان نصب می‌شود و ظاهری شبیه GitHub دارد: Repo، PR، Issue، Releases و CI. جایگزین‌های on-prem: GitLab CE، Forgejo.

> **در این دوره:** نسخه‌ی `gitea/gitea:1.27.3` با Docker Compose روی `localhost:3000` (وب) و `2222` (SSH) اجرا می‌شود (جلسه‌ی ۲).

همچنین ببینید: Gitea Actions، Pull Request (PR)، Remote (git remote / origin)

5.25 Secret · راز (اطلاعات محرمانه) (سیکرت)

هر چیز محرمانه: رمز، کلید API، توکن، کلید S3. قاعده‌ی طلایی: secret هرگز داخل Git و کد نمی‌رود، چون تاریخچه‌ی Git برای همیشه آن را نگه می‌دارد. (در Kubernetes شیء `Secret` هم داریم که برای نگهداری همین‌ها است.)

> **در این دوره:** کلید آزمایشگاه فقط در `.env` و `.dvc/config.local` می‌ماند؛ یک hook به نام `no-secrets` قبل از commit الگوی رمز را جستجو می‌کند (جلسه‌ی ۲ و ۴).

همچنین ببینید: .env file، Git hook، Secret (Kubernetes)

6 کیفیت کد
Code Quality

اگر هر نفر کد را به شکل خودش بنویسد و بدون تست commit کند، هیچ مرحله‌ی خودکاری بعداً قابل‌اعتماد نخواهد بود. ابزارهای این فصل در جلسه‌ی ۲ نصب می‌شوند و بعداً CI همان‌ها را اجرا می‌کند.

جلسه‌ها: جلسه ۲ (و ۱۳) · **تعداد واژه‌ها:** ۹

6.1 Lint / Static analysis · بررسی ایستای کد (لینت)

Lint ابزاری است که کد را بدون اجرا می‌خواند و خطاهای محتمل یا سبک نادرست را گزارش می‌کند، مثل import بلااستفاده. نام آن از «پرز لباس» آمده: کد را از پرز پاک می‌کند.

```
make lint
```

در این دوره: `make lint` دو دستور را اجرا می‌کند: `ruff check .` و `ruff format --check .` (جلسه‌ی ۲).

همچنین ببینید: ruff، Formatter، pre-commit

6.2 Formatter · قالب‌بند کد

ابزاری که ظاهر کد (فاصله‌ها، طول سطر، نقل‌قول‌ها) را خودکار یکدست می‌کند تا بحث سلیقه‌ای در بازبینی کد پیش نیاید. با قالب‌بند، شما فقط روی محتوای کد فکر می‌کنید و ظاهر را ابزار تنظیم می‌کند.

در این دوره: `ruff format` در دوره قالب‌بند است و حداکثر طول سطر در `pyproject.toml` برابر `100` تعیین شده است (جلسه‌ی ۲).

همچنین ببینید: ruff، Lint / Static analysis

6.3 ruff — (راف)

ruff ابزار سریع (نوشته‌شده با Rust) است که کار lint و قالب‌بندی را در Python انجام می‌دهد و جای چند ابزار قدیمی‌تر را می‌گیرد. چون خیلی سریع است، اجرای آن قبل از هر commit کاری حس نمی‌شود.

```
uv run ruff check .
uv run ruff format .
```

در این دوره: نسخه‌ی `0.16.7`؛ قواعد فعال در `[tool.ruff.lint]` با `select = ["E","F","I","B","UP"]` تعیین شده است (جلسه‌ی ۲).

همچنین ببینید: Lint / Static analysis، Formatter، pyproject.toml

6.4 Unit test · آزمون واحد

آزمون خودکار یک واحد کوچک کد (مثلاً یک تابع) که همیشه با یک ورودی ثابت یک خروجی مشخص را انتظار دارد. اگر بعداً کسی چیزی را خراب کند، تست قرمز می‌شود. خرابی چیزی که قبلاً کار می‌کرد را regression می‌گویند.

در این دوره: دو تست اصلی: ساخت داده با seed ثابت همیشه یک جدول می‌دهد، و مدل احتمال‌هایی بین ۰ و ۱ با طول درست برمی‌گرداند (جلسه‌ی ۲).

همچنین ببینید: pytest، pre-commit، CI (Continuous Integration)

6.5 pytest — (پای‌تست)

pytest چارچوب استاندارد تست در Python است: فایل‌هایی که نامشان با `test_` شروع می‌شود را پیدا و اجرا می‌کند و با `assert` درستی ادعاها را می‌سنجد. هر تابع تست با `assert` یک ادعا را می‌سنجد و اگر ادعا نادرست باشد، تست «قرمز» می‌شود.

```
make test
```

در این دوره: نسخه‌ی `9.1.1`؛ `make test` معادل `uv run pytest -q` است و در `pyproject.toml` مسیر `tests` و `pythonpath = ["scripts"]` تنظیم شده (جلسه‌ی ۲).

همچنین ببینید: Unit test، Makefile / make

6.6 Makefile / make — (میک‌فایل)

Makefile به دستورهای تکراری نام می‌دهد: `make lint` یعنی «دستورهای زیر هدف lint را اجرا کن». خط‌های دستور باید با TAB شروع شوند، نه فاصله؛ در غیر این‌صورت خطای `missing separator` می‌گیرید.

در این دوره: Makefile دوره هدف‌هایی مثل `data`، `data-v2`، `train`، `lint`، `test` و `s3-ls` دارد؛ بعداً `make verify-sNN` هم اضافه می‌شود (جلسه‌ی ۱ تا ۴).

همچنین ببینید: Shell script، Lint / Static analysis، pytest

6.7 pre-commit — (پری‌کامیت)

pre-commit ابزاری است که hookهای Git را از یک فایل پیکربندی نصب می‌کند و هر بار قبل از commit یا push کنترل‌های کیفیت را اجرا می‌کند. نتیجه این است که کد خراب اصلاً وارد تاریخچه نمی‌شود.

```
uv run pre-commit install
```

در این دوره: فایل `.pre-commit-config.yaml` را با `uv run pre-commit install` فعال می‌کنیم؛ hook قبل از commit، `ruff` و `no-secrets` را اجرا می‌کند و hook قبل از push، `pytest` را اجرا می‌کند (جلسه‌ی ۲).

همچنین ببینید: Git hook، ruff، Secret

6.8 Git hook · قلاب Git (هوک)

برنامه‌ای که Git در لحظه‌های مشخص (قبل از commit یا قبل از push) خودکار اجرا می‌کند. اگر کد خروج آن غیر صفر باشد، عملیات لغو می‌شود. مثل نگهبانی که قبل از ورود چیزها را وارسی می‌کند.

> **در این دوره:** hookها داخل پوشه‌ی `.git` نصب می‌شوند ولی فایل پیکربندی‌شان در Git است؛ به همین دلیل بعد از clone دوباره `pre-commit install` لازم است (جلسه‌ی ۲).

همچنین ببینید: pre-commit، Exit code

6.9 Config as code · پیکربندی به‌عنوان کد

یعنی تنظیمات پروژه (پارامترها، مسیرها، قواعد lint) در فایل‌های متنی نگهداری و با کد نسخه‌بندی شوند، به‌جای اینکه در ذهن افراد یا فقط روی یک سرور باشند. وقتی تنظیمات داخل Git است، می‌دانید چه کسی، کی و چرا آن را عوض کرده است.

> **در این دوره:** `configs/params.yaml` همه‌ی پارامترهای آموزش را دارد، پس تغییر آن‌ها یک commit قابل‌ردیابی است (جلسه‌ی ۲ و ۵).

همچنین ببینید: YAML، Params (params.yaml)، Infrastructure as Code (IaC)

7 Docker

Docker & Containers

Docker برنامه را همراه همه‌ی وابستگی‌هایش در یک بسته‌ی قابل‌حمل اجرا می‌کند تا «روی لپ‌تاپ من کار می‌کرد» دیگر دلیل نباشد. ابتدا برای اجرای سرویس‌های آزمایشگاه (جلسه‌ی ۳) و بعد برای بسته‌بندی API مدل (جلسه‌ی ۹) استفاده می‌شود.

جلسه‌ها: جلسه ۳، ۹ · **تعداد واژه‌ها:** ۱۵

7.1 Container · کانتینر (کانتینر)

Container یک فرایند ایزوله روی ماشین شماست که از روی یک image اجرا می‌شود و فایل‌ها، شبکه و فرایندهای خودش را می‌بیند. مثل یک اتاق دربسته در یک ساختمان مشترک: همه از یک هسته‌ی سیستم‌عامل استفاده می‌کنند ولی هر کانتینر فضای جدا دارد.

در این دوره: سرویس‌های آزمایشگاه (`gitea`، `seaweedfs`، `mlflow`) هر کدام یک کانتینر هستند؛ `docker ps` فهرستشان را نشان می‌دهد (جلسه‌ی ۳).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Image و Container

همچنین ببینید: Image، Virtual machine (VM)، Volume

7.2 Image · ایمیج (ایمیج)

Image بسته‌ی فقط‌خواندنی و آماده است که شامل برنامه و همه‌ی چیزهای لازم برای اجرای آن می‌شود. خودش اجرا نمی‌شود؛ از روی آن کانتینر ساخته می‌شود، مثل فایل نصب برنامه یا «کلاس» در برنامه‌نویسی که از آن نمونه می‌سازند.

در این دوره: imageهای آزمایشگاه از جمله `chrislusf/seaweedfs:4.45` و `gitea/gitea:1.27.3` از بسته‌ی آفلاین بارگذاری می‌شوند (جلسه‌ی ۳)؛ در جلسه‌ی ۹ image مدل خودمان را می‌سازیم.

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Image و Container

همچنین ببینید: Container، Dockerfile، Layer، Container registry

7.3 Docker Engine / Container runtime · موتور Docker / اجراکننده‌ی کانتینر

برنامه‌ای در پس‌زمینه که imageها را می‌گیرد و کانتینرها را می‌سازد و اجرا می‌کند. روی ویندوز و مک، چون کانتینرها به هسته‌ی لینوکس نیاز دارند، آن را داخل یک VM سبک اجرا می‌کنند.

در این دوره: روی لینوکس: Docker Engine؛ روی مک: Colima (رایگان) به‌جای Docker Desktop؛ روی ویندوز: Docker Engine داخل WSL2 (جلسه‌ی ۰ و ۳).

همچنین ببینید: Container، WSL2، Virtual machine (VM)

7.4 Virtual machine (VM) · ماشین مجازی

VM یک کامپیوتر کامل شبیه‌سازی‌شده با سیستم‌عامل مستقل است و از کانتینر سنگین‌تر و کندتر بالا می‌آید. کانتینر هسته را با میزبان به اشتراک می‌گذارد ولی VM هسته‌ی خودش را دارد.

در این دوره: در جلسه‌ی ۱۰ با Multipass یک VM اوبونتو `24.04` می‌سازیم و API را با Ansible روی آن مستقر می‌کنیم؛ این «خط پایه‌ی VM» قبل از Kubernetes است.

همچنین ببینید: Container، Multipass، Ansible

7.5 Layer · لایه

هر image از چند لایه‌ی فقط‌خواندنی ساخته می‌شود و هر دستور Dockerfile معمولاً یک لایه می‌سازد. لایه‌ها cache می‌شوند و بین imageها مشترکند، پس اگر فقط کد عوض شود لایه‌ی بسته‌های سنگین دوباره ساخته نمی‌شود.

در این دوره: به همین دلیل در Dockerfile ابتدا فایل‌های وابستگی کپی و نصب می‌شوند و بعد کد (جلسه‌ی ۹).

همچنین ببینید: Dockerfile، Image، Multi-stage build

7.6 Dockerfile · دستورالعمل ساخت image (داکرفایل)

فایل متنی با دستورهایی مثل `FROM`، `COPY`، `RUN` و `CMD` که مرحله به مرحله می‌گوید image چگونه ساخته شود؛ مثل دستور پخت یک غذا. از روی این فایل `docker build` یک image می‌سازد و هر سطر آن معمولاً یک لایه است.

```
docker build -t churn-api:0.1.0 .
```

در این دوره: جلسه‌ی ۳ فقط «خواندن» یک Dockerfile را تمرین می‌کند (فایل `mlflow/Dockerfile`)، و در جلسه‌ی ۹ دانشجو Dockerfile API را می‌نویسد. image پایه `python:3.12-slim-bookworm` است.

همچنین ببینید: Image، Layer، Base image، Multi-stage build

7.7 Base image · image پایه

imageای که ساخت image شما از آن شروع می‌شود (خط `FROM`). نسخه‌های `slim` کوچک‌ترند و سطح حمله و آسیب‌پذیری کمتری دارند.

در این دوره: `python:3.12-slim-bookworm` پایه‌ی image مدل است و `nginx:1.28-alpine` پایه‌ی Nginx (جلسه‌ی ۹).

همچنین ببینید: Dockerfile، Vulnerability scan، Multi-stage build

7.8 Image tag / digest · برچسب image

Tag نام قابل‌خواندن یک نسخه‌ی image است (مثل `seaweedfs:4.45`)، ولی می‌تواند بعداً به image دیگری اشاره کند. digest (`sha256:...`) اثر انگشت دقیق محتواست و تغییر نمی‌کند. تگ `latest` را در استقرار واقعی استفاده نکنید چون معلوم نیست کدام نسخه است.

در این دوره: نسخه‌ی گره‌ی kind در `versions.env` با digest پین شده است؛ imageهای دوره tag نسخه دارند (جلسه‌ی ۳ و ۹). (با tag در Git فرق دارد.)

همچنین ببینید: Image، SemVer، Tag

7.9 Container registry · مخزن image (رجیستری)

سرویسی که imageها را ذخیره و توزیع می‌کند. Docker Hub نمونه‌ی عمومی است؛ سازمان‌های بسته registry داخلی دارند. (با «model registry» در MLflow اشتباه نگیرید.)

```
docker push localhost:5001/churn-api:0.1.0
```

در این دوره: در کلاس آفلاین، imageها از بسته می‌آیند و در جلسه‌ی ۹ یک registry محلی `registry:3.1.0` روی `localhost:5001` برای push کردن image خودمان داریم.

همچنین ببینید: Image، Model registry، Air-gapped / Offline

7.10 Volume · ولوم (حجم ماندگار) (ولوم)

Volume مکانی است که Docker خودش مدیریت می‌کند و داده‌ی کانتینر را بیرون از کانتینر نگه می‌دارد تا با حذف کانتینر از بین نرود. اگر volume نداشته باشید، با حذف کانتینر تمام داده‌ی داخلش هم از بین می‌رود.

در این دوره: `gitea-data` و `seaweedfs-data` volume هستند؛ `docker compose down` آن‌ها را نگه می‌دارد ولی `docker compose down -v` همه را پاک می‌کند (جلسه‌ی ۳).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Volume و Bind mount

همچنین ببینید: Bind mount، Container

7.11 Bind mount · اتصال پوشه‌ی میزبان

در bind mount یک پوشه یا فایل مشخص از ماشین میزبان مستقیم داخل کانتینر قرار می‌گیرد و هر تغییر در یک طرف در طرف دیگر دیده می‌شود. برای پیکربندی و توسعه خوب است، ولی به مسیر ماشین وابسته است.

در این دوره: در Compose، فایل `./seaweedfs/s3.json` به‌صورت فقط‌خواندنی (`ro`) داخل کانتینر `seaweedfs` mount می‌شود (جلسه‌ی ۳).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Volume و Bind mount

همچنین ببینید: Volume، Docker Compose

7.12 Docker network · شبکه‌ی Docker

شبکه‌ی مجازی خصوصی که کانتینرها را به هم وصل می‌کند و اجازه می‌دهد با نام به هم دسترسی داشته باشند (مثلاً `seaweedfs:8333`)، بدون اینکه IP را بدانند. کانتینرهای یک شبکه می‌توانند با نام یکدیگر را پیدا کنند و کانتینرهای شبکه‌های دیگر آن‌ها را نمی‌بینند.

در این دوره: Compose دوره همه‌ی سرویس‌ها را در شبکه‌ی `mlops-lab` قرار می‌دهد؛ MLflow داخل آن با نام `seaweedfs` به S3 می‌رسد (جلسه‌ی ۳).

همچنین ببینید: Port mapping، DNS، Docker Compose

7.13 Port mapping · نگاشت پورت

اتصال یک پورت کانتینر به پورتی روی میزبان، به شکل «میزبان:کانتینر»، تا از بیرون کانتینر قابل دسترسی شود. آدرس `127.0.0.1` در ابتدای آن یعنی فقط از خود ماشین.

در این دوره: `127.0.0.1:8333:8333` در فایل Compose، S3 SeaweedFS را فقط روی لپ‌تاپ باز می‌کند (جلسه‌ی ۳).

همچنین ببینید: Port، localhost، Docker network

7.14 Docker Compose · داکر کامپوز

ابزاری برای تعریف و اجرای چند کانتینر مرتبط با یک فایل YAML. به‌جای تایپ چند دستور `docker run` طولانی، یک بار توصیف می‌کنید و با یک دستور همه را بالا می‌آورید.

```
docker compose up -d
docker compose ps
```

> **در این دوره:** `compose/docker-compose.yml` سرویس‌های `seaweedfs`، `mlflow`، `gitea` و `gitea-runner` را تعریف می‌کند؛ بالا آوردن با `docker compose up -d` (جلسه‌ی ۳).

همچنین ببینید: Volume، Docker network، YAML

7.15 .dockerignore · فایل نادیده‌گرفتن Docker

فایلی شبیه `.gitignore` که می‌گوید هنگام ساخت image چه فایل‌هایی (مثل `.venv`، `.git`، داده) نباید به context ساخت فرستاده شوند. image کوچک‌تر و ساخت سریع‌تر می‌شود و رمزها هم نشت نمی‌کنند.

> **در این دوره:** در جلسه‌ی ۹ برای image API نوشته می‌شود.

همچنین ببینید: .gitignore، Dockerfile

8 ذخیره‌ساز شیء و S3

Object Storage & S3

داده‌های بزرگ، نسخه‌های داده و فایل مدل‌ها در Git جا نمی‌گیرند. دوره آن‌ها را در یک ذخیره‌ساز شیء سازگار با S3 نگه می‌دارد که روی لپ‌تاپ خودتان اجرا می‌شود و همان رابطی را دارد که ابزارهای ابری هم استفاده می‌کنند.

جلسه‌ها: جلسه ۳ (و ۴، ۶) · **تعداد واژه‌ها:** ۱۱

8.1 Object storage · ذخیره‌ساز شیء

روشی برای نگهداری فایل‌ها که به‌جای درخت پوشه و فایل، هر فایل را یک «شیء» با یک کلید یکتا می‌داند و از راه HTTP می‌خواند و می‌نویسد. برای حجم زیاد، ارزان و مقیاس‌پذیر است و پایه‌ی data lake است.

در این دوره: SeaweedFS در دوره نقش ذخیره‌ساز شیء را دارد و داده‌ی DVC و artifactهای MLflow را نگه می‌دارد (جلسه‌ی ۳).

همچنین ببینید: Data lake / Data warehouse ،SeaweedFS ،Bucket ،S3 (and S3-compatible)

8.2 S3 (and S3-compatible) · — (اس‌تری)

S3 نام سرویس ذخیره‌ساز شیء آمازون و در عمل نام یک رابط استاندارد (API) است. «سازگار با S3» یعنی سرویس دیگری همان رابط را دارد و ابزارهایی مثل boto3، DVC و MLflow بدون تغییر با آن کار می‌کنند.

در این دوره: نشانی `s3://dvc/store` را به همین ترتیب به SeaweedFS می‌دهیم؛ فقط `endpoint` عوض می‌شود: `http://localhost:8333` (جلسه‌ی ۳ و ۴).

همچنین ببینید: SeaweedFS، Endpoint (S3)، Access key / Secret key، boto3 / aws CLI

8.3 SeaweedFS · — (سیدفایل‌اس)

SeaweedFS ذخیره‌ساز متن‌باز و سبک با دروازه‌ی سازگار با S3 است که با یک باینری یا یک کانتینر اجرا می‌شود. در دوره جایگزین MinIO است.

در این دوره: image `chrislusf/seaweedfs:4.45` با دستور `server -s3` و پورت‌های `8333` (S3)، `9333` (master) و `8888` (filer) بالا می‌آید (جلسه‌ی ۳).

همچنین ببینید: Object storage، S3 (and S3-compatible)، Volume

8.4 Bucket · سطل / ظرف (باکت)

Bucket ظرف سطح بالای اشیاء در ذخیره‌ساز شیء است و نامش در کل سرویس یکتاست. شبیه یک درایو یا پوشه‌ی ریشه است.

```
aws --endpoint-url http://localhost:8333 s3 ls
```

در این دوره: دو bucket داریم: `dvc` برای داده‌ی نسخه‌بندی‌شده و `mlflow` برای artifactهای آزمایش (جلسه‌ی ۳).

همچنین ببینید: Object key / Prefix، Data lake / Data warehouse

8.5 Object · شیء

یک «شیء» همان فایل ذخیره‌شده به‌همراه کلیدش و ویژگی‌های توصیفی (metadata) است. اشیاء تغییر داده نمی‌شوند؛ برای تغییر، شیء جدیدی جایگزین می‌شود.

در این دوره: نسخه‌ی ۱ داده یک شیء ۲۰۱۲۰۲ بایتی در bucket به نام `dvc` است (جلسه‌ی ۴).

همچنین ببینید: Bucket، Object key / Prefix، Hash (md5) / Checksum

8.6 Object key / Prefix · کلید شیء

Key نام کامل شیء داخل bucket است، مثل `store/files/md5/79/9213...`. در S3 پوشه‌ی واقعی وجود ندارد؛ بخش اول کلید (prefix) فقط شبیه پوشه نمایش داده می‌شود. (کلید اینجا با «access key» فرق دارد.)

در این دوره: DVC هر نسخه‌ی داده را زیر کلیدی مبتنی بر md5 محتوا می‌گذارد، تا دو فایل یکسان یک شیء شوند (جلسه‌ی ۴).

همچنین ببینید: Object، Bucket، Access key / Secret key، DVC cache

8.7 Endpoint (S3) · نشانی سرویس (اندپوینت)

آدرس و پورت سرویس ذخیره‌ساز. ابزارهایی که به‌طور پیش‌فرض به آمازون وصل می‌شوند، با تعیین endpoint به سرویس شما متصل می‌شوند.

در این دوره: `http://localhost:8333` از بیرون و `http://seaweedfs:8333` از داخل شبکه‌ی Docker؛ در DVC با `dvc remote modify ... endpointurl` تنظیم می‌شود (جلسه‌ی ۳ و ۴).

همچنین ببینید: S3 (and S3-compatible)، URL، Docker network، Endpoint (API)

8.8 Access key / Secret key · کلید دسترسی

جفتی که هویت شما را در S3 می‌سازد: access key مثل «نام کاربری» و secret key مثل «رمز». خطای `InvalidAccessKeyId` یعنی کلید اول اشتباه است و `SignatureDoesNotMatch` یعنی رمز اشتباه است.

در این دوره: کلید آزمایشگاه فقط در `.env` و `.dvc/config.local` است و هیچ‌وقت در Git یا این کتابچه نمی‌آید؛ آن را از مدرس بگیرید (جلسه‌ی ۳ و ۴).

همچنین ببینید: Authentication، Secret، .env file

8.9 Authentication · احراز هویت

پاسخ به «تو کیستی؟»: سیستم اثبات می‌کند که کاربر یا برنامه همان است که ادعا می‌کند، مثلاً با رمز یا کلید. مثل نشان‌دادن کارت شناسایی. بدون این مرحله مجوز دادن بی‌معناست.

در این دوره: S3 با جفت کلید دسترسی و Gitea با نام کاربری و رمز (یا SSH key) هویت را می‌سنجند (جلسه‌ی ۲ و ۳).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Authentication و Authorization

همچنین ببینید: Authorization، Access key / Secret key

8.10 Authorization · مجوزدهی

پاسخ به «تو مجازی چه کاری بکنی؟»: بعد از آنکه هویت مشخص شد، سیستم تعیین می‌کند چه اعمالی (خواندن، نوشتن، حذف) برای آن هویت آزاد است. مثل اینکه کارت شناسایی شما اجازه‌ی ورود به همه‌ی طبقات را نمی‌دهد.

> **در این دوره:** در `s3.json` سطح‌های دسترسی `Admin`، `Read`، `Write`، `List` و `Tagging` برای هویت آزمایشگاه تعیین شده است؛ در Kubernetes هم RBAC همین نقش را دارد (جلسه‌ی ۳).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Authentication و Authorization

همچنین ببینید: Authentication، Access key / Secret key

8.11 boto3 / aws CLI · — (بوتوتری)

`boto3` کتابخانه‌ی Python و `aws` ابزار خط فرمان رسمی آمازون برای کار با S3 هستند. چون SeaweedFS سازگار با S3 است، همین ابزارها را با تعیین `--endpoint-url` به آن وصل می‌کنیم.

```
make s3-ls
```

> **در این دوره:** نسخه‌های `boto3 1.43.95` و `awscli 1.46.1` پین شده‌اند؛ `make s3-ls` فهرست bucketها را می‌دهد (جلسه‌ی ۳).

همچنین ببینید: S3 (and S3-compatible)، Endpoint (S3)، Bucket

9 DVC: نسخه‌بندی داده و pipeline

DVC: Data Versioning & Pipelines

Git برای فایل‌های بزرگ داده مناسب نیست. DVC کنار Git کار می‌کند: خودِ داده را جای دیگری (cache و remote) نگه می‌دارد و فقط یک فایل کوچک اشاره‌گر در Git می‌گذارد. از جلسه‌ی ۵ همین ابزار مراحل آموزش را هم به‌صورت pipeline تکرارپذیر اجرا می‌کند.

جلسه‌ها: جلسه ۴، ۵ · **تعداد واژه‌ها:** ۱۸

9.1 DVC (Data Version Control) — (دی‌وی‌سی)

DVC ابزار خط فرمانی است که کنار Git، داده، مدل و pipeline را نسخه‌بندی می‌کند. شبیه Git به نظر می‌رسد (`add`، `push`، `pull`، `checkout`، `status`) ولی خودش مخزن مستقلی نیست و همیشه داخل یک Repo گیت کار می‌کند.

```
uv run dvc init
uv run dvc add data/raw/churn.csv
```

در این دوره: نسخه‌ی `3.67.1` با پشتیبانی S3؛ راه‌اندازی با `uv run dvc init` و آغاز ردیابی با `uv run dvc add data/raw/churn.csv` (جلسه‌ی ۴).

همچنین ببینید: Data versioning، Pointer file (.dvc)، DVC cache، Remote (DVC)

9.2 Data versioning · نسخه‌بندی داده

نگه‌داری نسخه‌های مختلف یک مجموعه‌داده و امکان برگشتن به هر کدام. Git برای فایل‌های بزرگ کند و حجیم می‌شود، و بدون نسخه‌ی داده، تکرار یک آزمایش یا فهمیدن اینکه مدل با چه داده‌ای ساخته شد ممکن نیست.

در این دوره: داده‌ی v1 (۵۰۰۰ سطر) و v2 (۸۰۰۰ سطر) هر کدام با یک tag در Git (`data-v1`، `data-v2`) نام‌گذاری شده‌اند (جلسه‌ی ۴).

همچنین ببینید: DVC (Data Version Control)، Snapshot، Git LFS، Data lineage

9.3 Snapshot · عکس لحظه‌ای (اسنپ‌شات)

عکس ثابت از داده در یک زمان مشخص. DVC نسخه‌ی یک snapshot را نگه می‌دارد، نه جریان زنده‌ی داده را.

در این دوره: `churn.csv` هر نسخه یک snapshot است؛ نسخه‌ی v2 در جلسه‌ی ۴ با `make data-v2` ساخته می‌شود.

همچنین ببینید: Data versioning، Batch vs Stream

9.4 Pointer file (.dvc) · فایل اشاره‌گر

فایل متنی کوچکی (با پسوند `.dvc`) که در Git ذخیره می‌شود و می‌گوید داده‌ی واقعی چه اثر انگشتی (md5) و چه حجمی دارد. داده‌ی واقعی در Git نیست؛ این فایل کوچک مثل «برگه‌ی تحویل» است که می‌گوید کدام نسخه از انبار برداشته شود.

در این دوره: فایل `data/raw/churn.csv.dvc` شامل `md5`، `size` و `hash: md5` و `path: churn.csv` است؛ تغییر داده یعنی تغییر دو خط md5 و size (جلسه‌ی ۴).

همچنین ببینید: Hash (md5) / Checksum، DVC cache، .gitignore، Staging area (git add)

9.5 Hash (md5) / Checksum · اثر انگشت فایل (هش)

Hash عددی ثابت‌طول است که از محتوای یک فایل ساخته می‌شود؛ اگر حتی یک بایت عوض شود، hash کاملاً فرق می‌کند. دو فایل یکسان همیشه یک hash دارند، پس hash مثل اثر انگشت فایل است. md5 رشته‌ای ۳۲ نویسه‌ای است.

در این دوره: DVC با md5 داده را شناسایی می‌کند؛ ذخیره‌سازی بر اساس محتوا (content-addressed) باعث می‌شود نسخه‌ی قدیمی هیچ‌وقت بازنویسی نشود (جلسه‌ی ۴).

همچنین ببینید: Object key / Prefix ،DVC cache ،Pointer file (.dvc)

9.6 DVC cache · حافظه‌ی نهان DVC

پوشه‌ی پنهان `.dvc/cache` در Repo شما که نسخه‌های داده را در آن نگه می‌دارد. با `dvc checkout` داده از cache به پوشه‌ی کاری برمی‌گردد و اگر در cache نبود، باید اول از remote گرفته شود.

در این دوره: پس از `dvc add` یک فایل در `.dvc/cache/files/md5/79/921313...` ساخته می‌شود (جلسه‌ی ۴).

همچنین ببینید: Hash (md5) / Checksum ،dvc push / pull / status ،Remote (DVC)

9.7 Remote (DVC) · ذخیره‌گاه مشترک DVC

مکان اشتراکی خارج از لپ‌تاپ که نسخه‌های داده در آن ذخیره می‌شوند. «remote» در DVC و «remote» در Git دو چیز مستقل‌اند: اولی بایت‌های داده را نگه می‌دارد و دومی کد و فایل‌های pointer را.

```
uv run dvc remote add -d storage s3://dvc/store
```

در این دوره: دو remote داریم: Gitea برای کد و pointerها و bucket `dvc` روی SeaweedFS (`s3://dvc/store`) برای داده؛ remote با گزینه‌ی d پیش‌فرض می‌شود (جلسه‌ی ۴).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی remote (Git) و remote (DVC)

همچنین ببینید: Endpoint (S3) ،DVC cache ،Remote (git remote / origin)

9.8 dvc push / pull / status · ارسال / دریافت / وضعیت

`dvc push` نسخه‌های داده‌ای را که روی remote نیست می‌فرستد، `dvc pull` آن‌ها را از remote می‌گیرد و `dvc status -c` cache را با remote مقایسه می‌کند و می‌گوید هم‌گام هستند یا نه. این سه دستور همان نقشی را دارند که `git push` و `git pull` برای کد دارند، ولی برای داده.

```
uv run dvc push
uv run dvc status -c
```

در این دوره: قبل از push، `dvc status -c` خط `new: data/raw/churn.csv` نشان می‌دهد و بعد از آن `Cache and remote 'storage' are in sync` (جلسه‌ی ۴).

همچنین ببینید: dvc checkout ،DVC cache ،Remote (DVC)

9.9 dvc checkout · بازگرداندن داده از cache

داده‌ی پوشه‌ی کاری را مطابق فایل‌های pointer فعلی به‌روز می‌کند. `git checkout` فقط کد و pointer را عوض می‌کند؛ بدون `dvc checkout` داده همان نسخه‌ی قبلی می‌ماند و ناهماهنگ می‌شود.

```
git checkout data-v1
uv run dvc checkout
```

> **در این دوره:** برای برگشتن به نسخه‌ی ۱: `git checkout data-v1` سپس `uv run dvc checkout` و `make train` دوباره `roc_auc 0.802` می‌دهد (جلسه‌ی ۴).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی git checkout و dvc checkout

همچنین ببینید: git checkout / git switch، DVC cache، Pointer file (.dvc)

9.10 Git LFS · —

افزونه‌ای برای Git که فایل بزرگ را در سرور جدا نگه می‌دارد و در Repo فقط یک اشاره‌گر می‌گذارد. ایده‌اش شبیه DVC است ولی به سرور Git وابسته است و pipeline و متریک ندارد.

> **در این دوره:** در جلسه‌ی ۴ فقط برای مقایسه معرفی می‌شود؛ DVC چون روی هر ذخیره‌ساز S3 کار می‌کند انتخاب دوره است.

همچنین ببینید: Data versioning، DVC (Data Version Control)

9.11 DVC pipeline / stage · خط لوله‌ی DVC

زنجیره‌ای از مراحل (stage) که هر کدام ورودی (deps)، خروجی (outs)، پارامتر و دستور دارد. DVC با این اطلاعات می‌فهمد کدام مرحله باید دوباره اجرا شود و کدام نتیجه‌ی قبلی را می‌تواند استفاده کند.

> **در این دوره:** مراحل `preprocess`، `train` و `evaluate` در `dvc.yaml` تعریف می‌شوند (جلسه‌ی ۵). (این غیر از pipeline در scikit-learn و CI است.)

همچنین ببینید: dvc.yaml (deps / outs)، DAG، dvc repro، Pipeline (scikit-learn)، CI pipeline

9.12 dvc.yaml (deps / outs) · فایل تعریف pipeline

فایلی که مرحله‌ها را با کلیدهای `cmd` (دستور)، `deps` (ورودی‌های وابسته)، `params` (پارامترهای مؤثر)، `outs` (خروجی‌ها) و `metrics` توصیف می‌کند. با آن، pipeline مثل کد در Git نسخه‌بندی می‌شود.

> **در این دوره:** در جلسه‌ی ۵ برای هر مرحله `deps` و `outs` نوشته می‌شود و `params.yaml` به‌عنوان پارامتر معرفی می‌شود.

همچنین ببینید: DVC pipeline / stage، Params (params.yaml)، dvc.lock

9.13 dvc.lock · فایل قفل DVC

فایلی که DVC بعد از اجرای pipeline می‌سازد و اثر انگشت دقیق همه‌ی ورودی‌ها و خروجی‌های هر مرحله را در آن ثبت می‌کند. بر اساس آن می‌فهمد چه چیزی عوض شده و برای lineage هم استفاده می‌شود.

> **در این دوره:** `dvc.lock` همراه کد در Git commit می‌شود و بخشی از شجره‌ی مدل است (جلسه‌ی ۵).

همچنین ببینید: Data lineage، dvc repro، Hash (md5) / Checksum

9.14 DAG · گراف جهت‌دار بدون دور (دَگ)

DAG (Directed Acyclic Graph) نمودار وابستگی مراحل است: فلش‌ها می‌گویند کدام مرحله به نتیجه‌ی کدام نیاز دارد و هیچ دوری وجود ندارد. با آن ترتیب درست اجرا مشخص می‌شود و مراحل مستقل می‌توانند موازی اجرا شوند.

```
uv run dvc dag
```

> **در این دوره:** `dvc dag` نمودار `preprocess`، `train` و `evaluate` را نشان می‌دهد؛ Airflow و Argo هم همین مدل DAG را دارند (جلسه‌ی ۵).

همچنین ببینید: DVC pipeline / stage، Orchestration

9.15 dvc repro · بازتولید pipeline

دستوری که pipeline را اجرا می‌کند ولی فقط مرحله‌هایی را که ورودی‌شان تغییر کرده؛ بقیه از cache می‌آیند. تغییر داده یا پارامتر بدون `dvc repro` یعنی نتیجه‌های قدیمی که با ورودی جدید نمی‌خوانند.

```
uv run dvc repro
```

> **در این دوره:** با تغییر `C` در `params.yaml` فقط مرحله‌ی `train` و بعد از آن اجرا می‌شود (جلسه‌ی ۵).

همچنین ببینید: DVC pipeline / stage، dvc.lock، Params (params.yaml)

9.16 Params (params.yaml) · پارامترهای pipeline

فایل YAML مقدارهای قابل‌تنظیم هر آزمایش (همان hyperparameterها و تنظیمات) که DVC مراقب تغییرشان است. `dvc params diff` تفاوت دو commit را نشان می‌دهد.

```
uv run dvc params diff
```

> **در این دوره:** `configs/params.yaml` شامل `data.test_size`، `data.seed`، `train.C`، `train.max_iter` و `eval.threshold` است (جلسه‌ی ۱ و ۵).

همچنین ببینید: Hyperparameter، Config as code، DVC metrics

9.17 DVC metrics · متریک‌های DVC

فایلی مثل `metrics.json` که DVC آن را به‌عنوان نتیجه‌ی مدل ردیابی می‌کند. `dvc metrics diff` نشان می‌دهد با تغییر کد یا داده، هر عدد چقدر عوض شده است.

```
uv run dvc metrics diff
```

> **در این دوره:** با دو نسخه‌ی داده، `roc_auc` از `0.802` (v1) به `0.7493` (v2) رفته و `f1` از `0.5969` به `0.6991` (جلسه‌ی ۴ و ۵).

همچنین ببینید: Metric، Params (params.yaml)، dvc exp

9.18 dvc exp · آزمایش‌های DVC

زیرمجموعه‌ی دستورهای DVC برای اجرای سریع چند آزمایش با پارامترهای مختلف بدون commit کردن هرکدام و مقایسه‌ی نتایجشان. بعداً نتیجه‌ی برتر را commit می‌کنید.

> **در این دوره:** در جلسه‌ی ۵ پیش از معرفی MLflow دیده می‌شود و در جلسه‌ی ۶ همین ایده با ردیابی کامل‌تر ادامه می‌یابد.

همچنین ببینید: Experiment tracking، Params (params.yaml)

10 MLflow و مخزن مدل

MLflow & the Model Registry

وقتی ده‌ها آزمایش اجرا می‌کنید، باید بدانید هرکدام با چه پارامتر و داده‌ای چه نتیجه‌ای داد و کدام مدل واقعاً در تولید است. MLflow این دو کار را می‌کند: ردیابی آزمایش‌ها (جلسه‌ی ۶) و registry مدل با alias (جلسه‌ی ۷).

جلسه‌ها: جلسه ۶، ۷ · **تعداد واژه‌ها:** ۱۷

10.1 MLflow — (ام‌ال‌فلو)

MLflow پلتفرم متن‌باز برای ردیابی آزمایش‌های ML و نگهداری مدل‌هاست. دو بخش مهم آن Tracking (ثبت آزمایش) و Model Registry (مدیریت نسخه‌های مدل) است.

در این دوره: سرور MLflow نسخه‌ی `3.16.0` در کانتینر روی `localhost:5050` اجرا می‌شود، با SQLite برای متادیتا و SeaweedFS برای artifactها از همان روز اول (جلسه‌ی ۶).

همچنین ببینید: Experiment tracking، Model registry، Tracking server

10.2 Experiment tracking — ردیابی آزمایش

ثبت سیستماتیک هر آزمایش (پارامترها، متریک‌ها، فایل‌ها، نسخه‌ی کد) تا بتوانید آن‌ها را مقایسه و هر کدام را بازتولید کنید. جایگزین «اکسل و نام فایل‌های model_final_v3_NEW».

در این دوره: در جلسه‌ی ۶ هر اجرای pipeline یک run در MLflow ثبت می‌کند و در رابط وب `localhost:5050` مقایسه می‌شود.

همچنین ببینید: Run، Experiment (MLflow)، dvc exp

10.3 Experiment (MLflow) — آزمایش (گروه runها)

در MLflow، Experiment یک پوشه‌ی منطقی است که runهای مربوط به یک مسئله را کنار هم نگه می‌دارد. (با «آزمایش» در معنای هر اجرای تکی فرق دارد؛ آن را run می‌گویند.) هر بار آموزش مدل، یک run داخل همان experiment می‌شود.

در این دوره: مثلاً یک experiment به نام `churn` همه‌ی runهای آموزش مدل churn را دارد (جلسه‌ی ۶).

همچنین ببینید: Run، Experiment tracking

10.4 Run — اجرا (ران)

Run یک بار اجرای آموزش (یا ارزیابی) است که MLflow پارامترها، متریک‌ها، artifactها و commit کد را برایش ثبت می‌کند. هر run یک شناسه‌ی یکتا دارد.

در این دوره: از کد با `mlflow.log_param`، `mlflow.log_metric` و `mlflow.log_artifact` به run اطلاعات اضافه می‌شود؛ مقدارهایی چون `roc_auc` و `f1` ثبت می‌شوند (جلسه‌ی ۶).

همچنین ببینید: Experiment (MLflow)، Artifact، Metric

10.5 Tracking server · سرور ردیابی

سرویس مرکزی MLflow که runها را می‌گیرد و با رابط وب نشان می‌دهد. سه بخش دارد: کلاینت (کد شما)، backend store (پایگاه داده‌ی متادیتا) و artifact store (فضای فایل‌های بزرگ).

در این دوره: backend store یک فایل SQLite است و artifact store همان bucket `mlflow` در SeaweedFS (جلسه‌ی ۶). از لپ‌تاپ با `localhost:5050` و از داخل کانتینرها با `mlflow:5000` به آن می‌رسیم.

همچنین ببینید: MLflow، S3 (and S3-compatible)، Bucket

10.6 Artifact · فرآورده (آرتیفکت)

هر فایل خروجی فرایند: فایل مدل، گزارش متریک، نمودار یا image Docker. در MLflow artifactها در artifact store نگه می‌دارند و به run وصل‌اند.

در این دوره: مدل train و نمودار اختیاری SHAP به‌عنوان artifact ثبت می‌شوند (جلسه‌ی ۶ و ۷).

همچنین ببینید: Run، Tracking server، SHAP

10.7 Model registry · مخزن مدل

فهرست مرکزی مدل‌ها با نام، نسخه‌ها و وضعیت هر نسخه؛ نقطه‌ی هماهنگی بین «مدل‌ساز» و «استقراردهنده». به‌جای پرسیدن «کدام فایل را بردارم؟»، نام و alias را می‌خوانید. (غیر از registry image.)

در این دوره: مدل با نام `churn` در MLflow registry ثبت می‌شود و API در جلسه‌ی ۸ آن را با `models:/churn@champion` بارگذاری می‌کند (جلسه‌ی ۷ و ۸).

همچنین ببینید: Registered model / Model version، Alias (@champion / @challenger)، Promotion، Container registry

10.8 Registered model / Model version · نسخه‌ی مدل ثبت‌شده

هر بار که مدلی با نام مشخص ثبت می‌شود یک نسخه‌ی شماره‌دار جدید (۱، ۲، ...) می‌گیرد. نسخه‌ها تغییرناپذیرند؛ به‌جای ویرایش یک نسخه، نسخه‌ی تازه می‌سازیم.

در این دوره: `churn` نسخه‌های ۱ و ۲ دارد و alias مشخص می‌کند کدام در تولید است (جلسه‌ی ۷).

همچنین ببینید: Model registry، Alias (@champion / @challenger)

10.9 Alias (@champion / @challenger) · نام مستعار نسخه (الیاس)

alias یک برچسب متحرک است که به یک نسخه‌ی مشخص مدل اشاره می‌کند و می‌توان آن را به نسخه‌ی دیگر منتقل کرد. مثل شناسه‌ی «قهرمان فعلی» که دست‌به‌دست می‌شود. جایگزین «Stages» (Staging/Production) است که در MLflow منسوخ شده.

در این دوره: `@champion` مدل تولید و `@challenger` نامزد جایگزینی است؛ promotion یعنی جابه‌جا کردن `@champion` به نسخه‌ی جدید و rollback یعنی برگرداندنش (جلسه‌ی ۷).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Alias (MLflow) و Tag (MLflow)

همچنین ببینید: Champion / Challenger، Promotion، Tags (MLflow)، Model URI (models:/...)

10.10 Tags (MLflow) · برچسب‌های توصیفی (تگ)

جفت‌های کلید/مقدار دلخواه که روی run یا نسخه‌ی مدل می‌گذارید تا اطلاعات توصیفی ثبت شود؛ مثلاً وضعیت تأیید. برخلاف alias به نسخه‌ی خاص «اشاره» نمی‌کنند و فقط توضیح‌اند.

در این دوره: `validation_status=approved` نمونه‌ای است که برای نسخه‌ی تأییدشده‌ی مدل می‌گذاریم (جلسه‌ی ۷).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Alias (MLflow) و Tag (MLflow)

همچنین ببینید: Alias (@champion / @challenger)، Registered model / Model version، Tag

10.11 Champion / Challenger · قهرمان / رقیب

Champion مدلی است که الان در تولید است و challenger مدل جدیدی که می‌خواهد جایش را بگیرد. تا وقتی challenger روی معیارها از champion بهتر نباشد، جابه‌جا نمی‌شود.

در این دوره: اسکریپت promotion متریک challenger را با آستانه و با champion مقایسه می‌کند (جلسه‌ی ۷).

همچنین ببینید: Alias (@champion / @challenger)، Promotion، Evaluation gate

10.12 Promotion · ارتقا

یعنی تبدیل یک نسخه‌ی مدل از «نامزد» به «مدل تولید». باید کنترل‌شده، با شواهد (متریک) و قابل برگشت باشد، نه یک کپی دستی فایل.

در این دوره: اسکریپت promotion در جلسه‌ی ۷ فقط اگر متریک از آستانه رد شود `@champion` را عوض می‌کند.

همچنین ببینید: Alias (@champion / @challenger)، Evaluation gate، Rollback

10.13 Evaluation gate · دروازه‌ی ارزیابی

شرطی خودکار که مدل را فقط وقتی از آن عبور می‌دهد که متریکش از حد تعیین‌شده بهتر باشد. مثل دروازه‌ی کنترل کیفیت در کارخانه: محصول معیوب از آن رد نمی‌شود.

در این دوره: اگر `roc_auc` مدل پایین‌تر از آستانه‌ی انتخابی باشد، promotion متوقف می‌شود (جلسه‌ی ۷).

همچنین ببینید: Threshold، Promotion، Metric

10.14 Model signature · امضای مدل

توصیف دقیق ورودی و خروجی مدل (نام ستون‌ها و نوع آن‌ها) که همراه مدل ذخیره می‌شود. با آن می‌توان جلوی ورودی ناسازگار را گرفت و train/serve skew را کم کرد.

در این دوره: signature مدل در MLflow با schema مدل pydantic در API هماهنگ می‌شود (جلسه‌ی ۷ و ۸).

همچنین ببینید: Train/serve skew، Request / Response schema، Registered model / Model version

10.15 Model card / Release note · کارت مدل

سند کوتاهی درباره‌ی مدل: چه کاری می‌کند، با چه داده‌ای ساخته شده، متریک‌ها، محدودیت‌ها و نسخه. هر کسی که مدل را می‌خواهد استفاده کند، از روی آن می‌فهمد به چه چیزی می‌تواند اعتماد کند.

در این دوره: در جلسه‌ی ۷ همراه نسخه‌ی مدل ثبت می‌شود.

همچنین ببینید: Registered model / Model version، Data lineage

10.16 Model URI (models:/...) · نشانی مدل

آدرس یکتای یک مدل در registry. شکل `models:/<name>@<alias>` به آخرین نسخه‌ی با آن alias اشاره می‌کند و شکل `models:/<name>/<version>` به نسخه‌ی مشخص.

> **در این دوره:** `models:/churn@champion` در API جلسه‌ی ۸ به‌کار می‌رود تا بدون تغییر کد، با عوض شدن alias مدل عوض شود.

همچنین ببینید: Alias (@champion / @challenger)، Model registry

10.17 Stages (legacy) · مرحله‌های قدیمی

در نسخه‌های قبلی MLflow، نسخه‌ی مدل را به مرحله‌های `Staging` و `Production` می‌بردند. از نسخه‌ی ۲٫۹ منسوخ شده و در MLflow 3.x هم منسوخ است؛ به‌جای آن alias و tag استفاده می‌شود.

> **در این دوره:** دوره فقط برای فهم مستندات قدیمی از آن نام می‌برد و عملی کار نمی‌کند (جلسه‌ی ۷).

همچنین ببینید: Alias (@champion / @challenger)، Tags (MLflow)

11 سرویس‌دهی مدل با FastAPI

Model Serving with FastAPI

مدلی که فقط روی لپ‌تاپ سازنده کار می‌کند به هیچ‌کس سود نمی‌رساند. در جلسه‌ی ۸ مدل را پشت یک API می‌گذاریم تا هر برنامه‌ای با یک درخواست HTTP از آن پیش‌بینی بگیرد.

جلسه‌ها: جلسه ۸ (و ۱۱، ۱۳) · **تعداد واژه‌ها:** ۱۱

11.1 Model serving · سرویس‌دهی مدل

در دسترس گذاشتن مدل آموزش‌دیده به‌صورت یک سرویس دائمی که ورودی می‌گیرد و پیش‌بینی برمی‌گرداند. مدل یک‌بار هنگام شروع سرویس بارگذاری می‌شود نه در هر درخواست.

در این دوره: سرویس `churn-api` مدل را از `models:/churn@champion` می‌خواند و از سه مسیر `/health`، `/predict` و `/metrics` استفاده می‌کند (جلسه‌ی ۸).

همچنین ببینید: Inference / Prediction، FastAPI، Batch inference، Model URI (models:/…)

11.2 FastAPI · — (فست‌ای‌پی‌آی)

FastAPI چارچوب Python برای ساخت API است؛ سریع، با ورودی‌های دارای نوع و مستندات خودکار. هر تابع به یک مسیر HTTP وصل می‌شود.

در این دوره: نسخه‌ی `0.141.1`؛ API دوره با `@app.get("/health")` و `@app.post("/predict")` تعریف می‌شود (جلسه‌ی ۸).

همچنین ببینید: uvicorn، pydantic، Endpoint (API)، OpenAPI / Swagger docs

11.3 uvicorn · — (یووی‌کورن)

uvicorn وب‌سرور سبکی است که برنامه‌ی FastAPI را اجرا می‌کند و درخواست‌های HTTP را به آن می‌رساند. FastAPI خودش سرور نیست و برای اجرا به چیزی مثل uvicorn نیاز دارد.

در این دوره: نسخه‌ی `0.53.0`؛ دستور اجرا شبیه `uvicorn churn.api:app --host 0.0.0.0 --port 8000` است و بعداً در Dockerfile و واحد systemd هم همین به کار می‌رود (جلسه‌ی ۸ تا ۱۰).

همچنین ببینید: FastAPI، systemd، Dockerfile

11.4 pydantic · — (پایدانتیک)

pydantic کتابخانه‌ای است که با تعریف یک کلاس، شکل و نوع داده‌ی ورودی را مشخص و خودکار اعتبارسنجی می‌کند. اگر ورودی با شکل نخواند، قبل از رسیدن به مدل رد می‌شود.

در این دوره: نسخه‌ی `2.13.5`؛ ورودی `/predict` یک کلاس pydantic با ستون‌های مدل است و خطای ورودی نامعتبر کد `422` می‌دهد (جلسه‌ی ۸).

همچنین ببینید: Request / Response schema، FastAPI، Model signature

11.5 Endpoint (API) · نقطه‌ی پایانی API (اندپوینت)

یک مسیر مشخص در API که کار مشخصی انجام می‌دهد، مثلاً `/predict`. هر endpoint با یک متد (GET/POST) و یک آدرس شناخته می‌شود. (با endpoint در S3 که آدرس کل سرویس است فرق دارد.)

در این دوره: سه endpoint API دوره: `GET /health`، `POST /predict` و `GET /metrics` (جلسه‌ی ۸).

همچنین ببینید: REST، HTTP، Endpoint (S3)

11.6 Request / Response schema · طرح درخواست و پاسخ

توصیف رسمی اینکه یک درخواست چه فیلدهایی با چه نوعی دارد و پاسخ چه شکلی دارد. مثل فرم کاغذی با کادرهای مشخص: هر کس باید همان کادرها را پر کند.

در این دوره: schema ورودی `/predict` همان ستون‌های آموزش (مثل `tenure` و `monthly_charges`) را دارد و پاسخ شامل احتمال ریزش است؛ تست‌های pytest با httpx آن را می‌آزمایند (جلسه‌ی ۸).

همچنین ببینید: pydantic، Model signature، Train/serve skew، JSON

11.7 Health check · بررسی سلامت

یک endpoint ساده (معمولاً `/health`) که به‌سرعت می‌گوید سرویس زنده و آماده است. سیستم‌های بیرونی (Docker، Kubernetes، load balancer) از روی آن تصمیم می‌گیرند به سرویس ترافیک بدهند یا آن را دوباره راه بیندازند.

در این دوره: `curl localhost:8000/health` باید `200` برگرداند؛ بعداً به probeهای readiness و liveness در Kubernetes وصل می‌شود (جلسه‌ی ۸ و ۱۱).

همچنین ببینید: Probes: liveness / readiness، HTTP status code، Endpoint (API)

11.8 Logging (structured) · ثبت رویداد (لاگ) (لاگ)

نوشتن پیام‌های رویدادها در خروجی برنامه تا بعداً بتوان فهمید چه شده است. لاگ «ساخت‌یافته» (structured) مثل JSON است و برای جستجو و تحلیل ماشینی مناسب‌تر است.

در این دوره: لاگ‌های سرویس روی stdout می‌روند و با `docker logs` یا `kubectl logs` دیده می‌شوند (جلسه‌ی ۸ و ۱۱).

همچنین ببینید: stdout / stderr، Monitoring / Observability

11.9 httpx / TestClient · — (اچ‌تی‌تی‌پی‌اکس)

httpx کتابخانه‌ی Python برای فرستادن درخواست HTTP است و در تست API کاربرد دارد: می‌توان بدون بالا آوردن سرور واقعی درخواست آزمایشی به FastAPI فرستاد. به این ترتیب تست‌ها سریع‌اند و به شبکه و پورت وابسته نیستند.

در این دوره: نسخه‌ی `0.28.1`؛ تست‌های `/health` و `/predict` با pytest و httpx نوشته می‌شوند (جلسه‌ی ۸).

همچنین ببینید: pytest، Endpoint (API)

11.10 OpenAPI / Swagger docs · مستند خودکار API

FastAPI از روی کد، یک توصیف استاندارد OpenAPI می‌سازد و صفحه‌ی تعاملی مستندات (مسیر `/docs`) را خودکار فراهم می‌کند که می‌توان درخواست آزمایشی هم فرستاد. این صفحه به کسی که API را نمی‌شناسد نشان می‌دهد چه مسیرهایی هست و چه ورودی‌ای می‌خواهد.

در این دوره: پس از اجرای API، `localhost:8000/docs` را در مرورگر باز کنید (جلسه‌ی ۸).

همچنین ببینید: FastAPI، REST

11.11 Batch inference · پیش‌بینی دسته‌ای

به‌جای پاسخ آنی به هر درخواست، یک فایل بزرگ از ورودی‌ها را یک‌جا و زمان‌بندی‌شده (مثلاً شبانه) پیش‌بینی می‌کنند. وقتی جواب فوری لازم نیست، ساده‌تر و ارزان‌تر است.

در این دوره: تمرین اختیاری (تکلیف) در جلسه‌ی ۸.

همچنین ببینید: Inference / Prediction، Model serving، Batch vs Stream

12 بسته‌بندی و امنیت image

Containerization & Image Security

image یک مدل یا API فقط وقتی قابل اعتماد است که کوچک، با حداقل دسترسی و بدون آسیب‌پذیری شناخته‌شده‌ی جدی باشد. جلسه‌ی ۹ این سه هدف را با چند ابزار ساده عملی می‌کند.

جلسه‌ها: جلسه ۹ · **تعداد واژه‌ها:** ۸

12.1 Multi-stage build · ساخت چندمرحله‌ای

در یک Dockerfile چند مرحله (`FROM` ... `FROM`) تعریف می‌شود: در مرحله‌ی اول ابزار سنگین ساخت نصب و برنامه ساخته می‌شود و در مرحله‌ی آخر فقط نتیجه‌ی لازم در image نهایی می‌ماند. مثل آشپزخانه‌ای که ظرف‌های آشپزی را سالن غذاخوری نمی‌برد.

در این دوره: در جلسه‌ی ۹ image API با این روش کوچک‌تر و امن‌تر ساخته می‌شود.

همچنین ببینید: Dockerfile، Base image، Layer

12.2 Non-root user · کاربر بدون اختیار مدیر

اجرای برنامه در کانتینر با یک کاربر معمولی به‌جای root. اگر برنامه آسیب‌پذیر باشد و نفوذی رخ دهد، مهاجم اختیار کمتری دارد. اصل کلی‌اش least privilege (حداقل دسترسی لازم) است.

در این دوره: در Dockerfile جلسه‌ی ۹ با دستور `USER` یک کاربر عادی برای اجرای `uvicorn` تعریف می‌شود.

همچنین ببینید: File permission، sudo، Vulnerability scan

12.3 Vulnerability scan · اسکن آسیب‌پذیری

بررسی خودکار image برای پیدا کردن بسته‌های دارای ضعف امنیتی شناخته‌شده، با مقایسه‌ی فهرست بسته‌های داخل آن با یک پایگاه داده‌ی آسیب‌پذیری‌ها. خروجی فهرستی با شدت (severity) هر مورد است.

در این دوره: در جلسه‌ی ۹ حداقل دو یافته را با به‌روزرسانی بسته یا تغییر base image برطرف می‌کنیم و دوباره اسکن می‌کنیم.

همچنین ببینید: Trivy، CVE، SBOM

12.4 Trivy · — (تریوی)

Trivy اسکنر متن‌باز آسیب‌پذیری برای image، فایل‌ها و Repo است. در محیط آفلاین با یک پایگاه داده‌ی از پیش دانلودشده کار می‌کند.

```
TRIVY_SKIP_DB_UPDATE=true trivy image --offline-scan churn-api:0.1.0
```

در این دوره: نسخه‌ی `0.74.0` با DB آفلاین در `trivy-cache`؛ با `TRIVY_SKIP_DB_UPDATE=true` و `--offline-scan` از رفتن به اینترنت جلوگیری می‌شود (جلسه‌ی ۹).

همچنین ببینید: Vulnerability scan، CVE، Air-gapped / Offline

12.5 CVE · شناسه‌ی آسیب‌پذیری شناخته‌شده (سی‌وی‌ای)

CVE (Common Vulnerabilities and Exposures) شناسه‌ی عمومی یک آسیب‌پذیری امنیتی شناخته‌شده است، با قالب «CVE-سال-عدد». مثل «شماره‌ی پرونده» که همه‌ی ابزارها و سازمان‌ها برای یک ضعف یکسان به کار می‌برند.

در این دوره: Trivy هر یافته را با شناسه‌ی CVE، بسته‌ی آسیب‌پذیر و نسخه‌ی رفع‌شده نشان می‌دهد (جلسه‌ی ۹).

همچنین ببینید: Trivy ،Vulnerability scan

12.6 SBOM · فهرست اجزای نرم‌افزار (اِس‌بام)

SBOM (Software Bill of Materials) فهرست دقیق همه‌ی بسته‌ها و نسخه‌های داخل یک نرم‌افزار یا image است، مثل لیست مواد روی بسته‌ی غذا. وقتی آسیب‌پذیری تازه‌ای اعلام شود با آن سریع می‌فهمید آیا درگیر هستید.

در این دوره: با ابزار Syft برای image ساخته می‌شود (اختیاری در جلسه‌ی ۹).

همچنین ببینید: Syft ،CVE ،Software supply chain

12.7 Syft · — (سیفت)

Syft ابزار تولید SBOM از image یا پوشه است. خروجی آن فهرست بسته‌های داخل image است که می‌توان آن را با ابزار اسکن مقایسه کرد.

```
syft churn-api:0.1.0
```

در این دوره: نسخه‌ی `1.51.1` (جلسه‌ی ۹، اختیاری).

همچنین ببینید: Trivy ،SBOM

12.8 Software supply chain · زنجیره‌ی تأمین نرم‌افزار

مجموعه‌ی همه‌ی چیزهایی که وارد محصول می‌شود: بسته‌های Python، image پایه، ابزارهای ساخت. هر حلقه‌اش می‌تواند آسیب‌پذیر یا مخرب باشد، پس باید منبع و نسخه‌ی آن‌ها شناخته و کنترل‌شده باشد.

در این دوره: به همین دلیل از wheelhouse و registry داخلی و نسخه‌های پین‌شده استفاده می‌کنیم (جلسه‌ی ۰ و ۹).

همچنین ببینید: SBOM ،Wheelhouse ،Container registry ،Image tag / digest

13 استقرار روی VM و Infrastructure as Code

VM Deployment & IaC

قبل از Kubernetes، یک مسیر ساده و واقعی را می‌رویم: مستقر کردن API روی یک ماشین مجازی لینوکس. بعد همان کار را با Ansible به‌صورت کد و تکرارپذیر درمی‌آوریم تا هر استقرار مثل قبلی باشد.

جلسه‌ها: جلسه ۱۰ · **تعداد واژه‌ها:** ۱۱

13.1 systemd — (سیستم‌دی)

systemd مدیر سرویس‌های لینوکس است: برنامه را در پس‌زمینه اجرا می‌کند، بعد از خرابی دوباره بالا می‌آورد و هنگام روشن شدن ماشین خودکار شروع می‌کند. هر سرویس با یک فایل «unit» تعریف می‌شود.

```
systemctl status churn-api
```

در این دوره: در جلسه‌ی ۱۰ API با یک unit به نام مثل `churn-api.service` اجرا می‌شود و با `systemctl status` بررسی می‌شود.

همچنین ببینید: Virtual machine (VM)، uvicorn، Reverse proxy (Nginx)

13.2 Reverse proxy (Nginx) · پراکسی معکوس (ریورس پراکسی)

سرویسی در جلوی برنامه که درخواست‌های کاربران را می‌گیرد و به برنامه‌ی پشت خود می‌فرستد. می‌تواند HTTPS، لاگ‌گیری و محدودیت درخواست را به عهده بگیرد تا خود برنامه ساده بماند. Nginx رایج‌ترین نمونه است.

در این دوره: روی VM در جلسه‌ی ۱۰ Nginx پورت ۸۰ را به پورت داخلی `uvicorn` می‌فرستد. همین نقش در Kubernetes را Ingress انجام می‌دهد.

همچنین ببینید: systemd، Ingress، uvicorn

13.3 Multipass — (مولتی‌پس)

Multipass ابزاری از Canonical برای ساخت سریع ماشین مجازی اوبونتو روی لپ‌تاپ است. با یک دستور یک ماشین اوبونتو بالا می‌آید و با یک دستور دیگر حذف می‌شود، پس برای آزمایش مناسب است.

در این دوره: نسخه‌ی `1.16.4` و VM اوبونتو `24.04` با حدود ۲ گیگابایت RAM (جلسه‌ی ۱۰). نباید VM و kind را همزمان اجرا کنید.

همچنین ببینید: Virtual machine (VM)، Ansible

13.4 Infrastructure as Code (IaC) · زیرساخت به‌عنوان کد

توصیف سرورها و تنظیمات آن‌ها در فایل‌های متنی که اجرا می‌شوند، به‌جای تایپ دستی دستورها. نتیجه: تکرارپذیر، قابل بازبینی در Git و قابل برگشت.

در این دوره: Ansible (جلسه‌ی ۱۰)، فایل‌های Compose و manifestهای Kubernetes و Helm (جلسه‌ی ۱۱ و ۱۲) همه نوعی IaC هستند.

همچنین ببینید: Ansible، Terraform، Idempotent، Config as code

13.5 Ansible — (آنسیبل)

Ansible ابزار خودکارسازی پیکربندی سرورهاست که از راه SSH و بدون نصب عامل (agent) روی سرور کار می‌کند. کارها را در فایل‌های YAML می‌نویسید و روی یک یا چند ماشین اجرا می‌کنید.

```
ansible-playbook -i inventory.ini site.yml
```

در این دوره: `ansible-core 2.21.4` روی لپ‌تاپ اجرا می‌شود و API را روی VM مستقر می‌کند؛ با اجرای دوباره‌ی playbook و نسخه‌ی قبلی، rollback هم انجام می‌شود (جلسه‌ی ۱۰).

همچنین ببینید: Playbook، Inventory، Idempotent، SSH

13.6 Playbook — دفترچه‌ی عملیات (پلی‌بوک)

فایل YAML که فهرستی از «وظیفه»ها (task) را برای اجرا روی گروهی از ماشین‌ها توصیف می‌کند؛ مثلاً: بسته را نصب کن، فایل را کپی کن، سرویس را راه بینداز. چون playbook متن است، در Git نگه‌داری می‌شود و هر استقرار همان مراحل را دقیقاً تکرار می‌کند.

در این دوره: playbook جلسه‌ی ۱۰ نسخه‌ی API را از یک متغیر می‌گیرد؛ برای rollback همان را با شماره‌ی نسخه‌ی قبلی دوباره اجرا می‌کنید.

همچنین ببینید: Ansible، Inventory، Idempotent، YAML

13.7 Inventory — فهرست ماشین‌ها

فایلی که می‌گوید Ansible باید به کدام ماشین‌ها (با چه نشانی و کاربری) وصل شود و آن‌ها را گروه‌بندی می‌کند. مثل دفترچه‌ی آدرس برای عملیات.

در این دوره: در جلسه‌ی ۱۰ شامل یک ماشین است: VM اوبونتو که با Multipass ساخته شده.

همچنین ببینید: Ansible، Playbook، Multipass

13.8 Idempotent — خودتوان / تکرارپذیر بدون اثر اضافه (ایدِمپوتِنت)

عملی idempotent است که اگر چند بار پشت سر هم اجرا شود، نتیجه‌اش مثل یک بار اجرا باشد. مثل دکمه‌ی «روشن» روی چراغ: بار دوم فشردنش چیز جدیدی اضافه نمی‌کند. در Ansible یعنی اگر چیزی از قبل درست است، دست نمی‌زند.

در این دوره: اجرای دوباره‌ی playbook جلسه‌ی ۱۰ نباید چیزی عوض کند مگر اینکه نسخه یا تنظیمی تغییر کرده باشد.

همچنین ببینید: Playbook، Infrastructure as Code (IaC)، Rollback

13.9 Terraform — (ترافرم)

Terraform ابزار IaC برای ساخت و مدیریت منابع زیرساخت (سرور، شبکه، سرویس ابری) به‌صورت توصیفی است. Ansible بیشتر «پیکربندی داخل سرور» و Terraform بیشتر «ساختن خود زیرساخت» است.

در این دوره: در این دوره فقط در سطح آشنایی معرفی می‌شود؛ در دوره‌ی بعدی استفاده‌ی عملی دارد (جلسه‌ی ۱۰).

همچنین ببینید: Infrastructure as Code (IaC)، Ansible

13.10 Runbook · دفترچه‌ی راهنمای عملیات (ران‌بوک)

سندی کوتاه و گام‌به‌گام برای کسی که باید سیستم را راه بیندازد، بررسی کند یا مشکلش را رفع کند، حتی اگر سازنده‌ی آن نباشد. در شرایط اضطرار از روی آن عمل می‌کنند.

> **در این دوره:** در جلسه‌ی ۱۰ runbook استقرار و rollback، و در جلسه‌ی ۱۴ runbook نهایی پروژه نوشته می‌شود.

همچنین ببینید: Rollback، Definition of Done (DoD)

13.11 Log rotation · چرخش لاگ

خودکار بایگانی یا حذف فایل‌های لاگ قدیمی تا دیسک پر نشود. بدون آن، فایل لاگ یک سرویس شلوغ می‌تواند ظرف چند هفته تمام دیسک ماشین را پر کند.

> **در این دوره:** از موضوع‌های جلسه‌ی ۱۰ برای سرویس روی VM.

همچنین ببینید: Logging (structured)، systemd

14 Kubernetes

Kubernetes

Kubernetes چند ماشین را یک «کلاستر» می‌کند و کانتینرها را روی آن‌ها اجرا، پایش و در صورت خرابی جایگزین می‌کند. شما به‌جای «این کانتینر را اجرا کن» می‌گویید «می‌خواهم دو نمونه از این سرویس همیشه زنده باشد» و Kubernetes آن را حفظ می‌کند.

جلسه‌ها: جلسه ۱۱ · **تعداد واژه‌ها:** ۲۰

14.1 Kubernetes (K8s) — (کوبِرنتیز)

Kubernetes سامانه‌ی متن‌باز مدیریت کانتینرهاست: زمان‌بندی روی ماشین‌ها، جایگزینی خودکار کانتینر خراب، مقیاس‌دهی و به‌روزرسانی تدریجی. گاهی به‌اختصار K8s نوشته می‌شود (۸ حرف بین K و s).

> **در این دوره:** در جلسه‌ی ۱۱ API را با دستورهای `kubectl` روی کلاستر kind مستقر می‌کنیم؛ نسخه‌ی گره‌ها `v1.36.4` است.

همچنین ببینید: Cluster، Pod، Deployment، kind

14.2 Cluster · خوشه (کلاستر)

مجموعه‌ای از ماشین‌ها (گره‌ها) که با Kubernetes مانند یک سیستم واحد دیده می‌شوند. یک بخش «مغز» (control plane) تصمیم می‌گیرد و گره‌های کارگر کانتینرها را اجرا می‌کنند.

> **در این دوره:** کلاستر دوره با نام `mlops` و تنها یک گره‌ی control-plane داخل Docker ساخته می‌شود (`kind-up.sh`).

همچنین ببینید: Node، kind، kubectl

14.3 Node · گره

یک ماشین (فیزیکی یا مجازی) در کلاستر که پادها روی آن اجرا می‌شوند. کلاستر از چند گره ساخته می‌شود و هر گره منابع خودش (CPU و RAM) را در اختیار پادها می‌گذارد.

```
kubectl get nodes
```

> **در این دوره:** در kind هر «گره» خودش یک کانتینر Docker است؛ `kubectl get nodes` فهرستشان را می‌دهد.

همچنین ببینید: Cluster، Pod، kind

14.4 kind — (کایند)

kind (Kubernetes IN Docker) یک کلاستر Kubernetes کامل را داخل کانتینرهای Docker روی لپ‌تاپ می‌سازد. برای آموزش و آزمایش مناسب است و نیاز به ماشین مجازی ندارد.

```
kind create cluster --name mlops
```

> **در این دوره:** نسخه‌ی `v0.33.0` با فایل `lab-kit/k8s/kind-config.yaml`؛ پورت‌های `8080` و `8443` لپ‌تاپ را به Traefik وصل می‌کند و registry محلی روی `5001` هست (جلسه‌ی ۱۱ و ۱۲).

همچنین ببینید: Cluster، kubectl، Container registry

14.5 kubectl — (کیوب‌کتل / کیوب‌کنترل)

kubectl ابزار خط فرمان برای گفت‌وگو با کلاستر است: دیدن منابع، ساخت، حذف و عیب‌یابی. هر چه در Kubernetes می‌بینید یا تغییر می‌دهید، از همین ابزار می‌گذرد.

```
kubectl get pods
kubectl apply -f deployment.yaml
```

در این دوره: نسخه‌ی `v1.36.4`؛ دستورهای پرکاربرد: `kubectl get pods`، `kubectl logs`، `kubectl describe`، `kubectl apply -f` (جلسه‌ی ۱۱).

همچنین ببینید: Manifest، Pod، port-forward

14.6 Pod · پاد (پاد)

کوچک‌ترین واحد اجرا در Kubernetes: یک یا چند کانتینر که با هم و روی یک گره اجرا می‌شوند و یک IP مشترک دارند. پاد موقتی است و در صورت خرابی با یک پاد جدید و IP جدید جایگزین می‌شود، به همین دلیل مستقیم به آن وصل نمی‌شویم.

در این دوره: `kubectl get pods` پادهای `churn-api` را با وضعیت `Running` یا خطایی مثل `ImagePullBackOff` نشان می‌دهد (جلسه‌ی ۱۱).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Pod و Deployment

همچنین ببینید: Deployment، Service، Replica

14.7 Deployment · استقرار (شیء Deployment)

شیءای که می‌گوید «چند نمونه از این پاد با این image همیشه اجرا شود» و Kubernetes این وضعیت را حفظ می‌کند. بالاترین سطحی است که معمولاً با آن کار می‌کنید و به‌روزرسانی تدریجی و برگشت را مدیریت می‌کند.

در این دوره: `churn-api` یک Deployment با چند replica است؛ تغییر image نسخه‌ی جدید را تدریجی بالا می‌آورد (جلسه‌ی ۱۱).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Pod و Deployment

همچنین ببینید: Pod، Replica، Rollout، Service

14.8 Replica · نمونه‌ی هم‌زمان (ریپلیکا)

تعداد نمونه‌های هم‌زمانِ یک پاد. چند replica هم تحمل خطا را بالا می‌برد و هم بار را بین نمونه‌ها تقسیم می‌کند.

در این دوره: `kubectl scale deployment/churn-api --replicas=2` تعداد را عوض می‌کند و Service ترافیک را بین پادها پخش می‌کند (جلسه‌ی ۱۱).

همچنین ببینید: Deployment، Service، Pod

14.9 Service · سرویس (شیء Service)

آدرس و نام ثابت برای گروهی از پادها. چون پادها می‌آیند و می‌روند، Service یک نشانی پایدار می‌دهد که ترافیک را بین پادهای سالم پخش می‌کند. مثل شماره‌ی مرکزی یک شرکت که هر بار به یک کارمند وصل می‌شود.

در این دوره: Service مدل را با نام داخلی در کلاستر در دسترس می‌گذارد و Ingress در جلسه‌ی ۱۲ به آن می‌رسد (جلسه‌ی ۱۱).

همچنین ببینید: Pod، Ingress، port-forward، Label / Selector

14.10 Namespace · فضای نام

بخش‌بندی منطقی یک کلاستر؛ مثل پوشه‌ای برای جدا کردن منابع پروژه‌ها یا محیط‌ها. نام یک منبع فقط در namespace خودش یکتا باید باشد.

در این دوره: سرویس‌های دوره می‌توانند در namespace جدا مثل `monitoring` برای Prometheus و Grafana باشند (جلسه‌ی ۱۱ و ۱۳).

همچنین ببینید: Cluster، Pod

14.11 ConfigMap · — (کانفیگ‌مپ)

شیءای برای نگه‌داری تنظیمات غیرمحرمانه (آدرس‌ها، پرچم‌ها) بیرون از image، تا بتوان بدون ساخت دوباره‌ی image، تنظیمات را عوض کرد. ConfigMap محرمانه نیست و برای رمز و کلید نباید استفاده شود؛ برای آن‌ها شیء Secret هست.

در این دوره: مثلاً آدرس سرور MLflow و نام alias مدل می‌توانند از ConfigMap بیایند (جلسه‌ی ۱۱).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی ConfigMap و Secret (Kubernetes)

همچنین ببینید: Secret (Kubernetes)، Environment variable، Manifest

14.12 Secret (Kubernetes) · شیء Secret

شیء مخصوص نگه‌داری اطلاعات محرمانه (رمز، کلید) که جدا از image و manifest به پاد داده می‌شود. توجه: به‌طور پیش‌فرض فقط کدگذاری base64 می‌شود، رمزنگاری قوی نیست و باید دسترسی به آن را محدود کرد.

در این دوره: کلید S3 آزمایشگاه به‌جای نوشته‌شدن در manifest، از یک Secret به پاد می‌رسد (جلسه‌ی ۱۱).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی ConfigMap و Secret (Kubernetes)

همچنین ببینید: ConfigMap، Secret، Environment variable

14.13 Manifest · فایل توصیف منبع (مانیفست)

فایل YAMLای که یک منبع Kubernetes (مثل Deployment یا Service) را توصیف می‌کند: نوع، نام، و مشخصات. با `kubectl apply -f` به کلاستر داده می‌شود.

در این دوره: در جلسه‌ی ۱۱ manifestهای `Deployment` و `Service` را می‌نویسیم و در جلسه‌ی ۱۲ آن‌ها را در chart هلم قالب‌بندی می‌کنیم.

همچنین ببینید: Declarative / Desired state، kubectl، YAML، Chart

14.14 Declarative / Desired state · توصیفی / وضعیت مطلوب

در روش توصیفی نمی‌گویید «چه کار کن»، بلکه می‌گویید «نتیجه چه باشد» (مثلاً ۲ replica) و سیستم خودش اختلاف وضعیت فعلی با مطلوب را برطرف می‌کند. روش امری (imperative) برعکس، دستور قدم‌به‌قدم است.

در این دوره: manifest، playbook و `docker-compose.yml` همه توصیفی‌اند.

همچنین ببینید: Manifest، Infrastructure as Code (IaC)، Idempotent

14.15 Label / Selector · برچسب و انتخابگر

Label جفت کلید/مقدار روی یک منبع است و Selector منابعی را که برچسب مشخص دارند انتخاب می‌کند. Service از همین راه می‌فهمد ترافیک را به کدام پادها بدهد.

> **در این دوره:** مثلاً `app: churn-api` هم روی پادها و هم در selector سرویس می‌آید؛ اگر یکی تایپی داشته باشد، Service به هیچ پادی نمی‌رسد (جلسه‌ی ۱۱).

همچنین ببینید: Service، Pod

14.16 Probes: liveness / readiness · بررسی‌های زنده بودن و آماده بودن

Liveness probe می‌پرسد «برنامه هنوز زنده است؟» و اگر نه، کانتینر را دوباره راه می‌اندازد. Readiness probe می‌پرسد «آماده‌ی دریافت ترافیک است؟» و اگر نه، موقتاً پاد را از Service بیرون می‌گذارد. هر دو معمولاً از endpoint سلامت استفاده می‌کنند.

> **در این دوره:** probeهای Deployment `churn-api` به `/health` وصل می‌شوند؛ نبودن probe یکی از «خرابی‌های عمدی» تمرین جلسه‌ی ۱۱ است.

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Liveness probe و Readiness probe

همچنین ببینید: Health check، Pod

14.17 Requests / Limits · درخواست و سقف منابع

Request حداقل CPU/RAM است که پاد برای زمان‌بندی به آن نیاز دارد و Limit سقفی است که نباید از آن بگذرد. اگر limit حافظه رد شود، کانتینر کشته می‌شود (OOMKilled).

> **در این دوره:** در لپ‌تاپ‌های ۸ گیگابایتی مقدارها کوچک انتخاب می‌شوند (جلسه‌ی ۱۱).

همچنین ببینید: Pod، Deployment

14.18 Rollout · به‌روزرسانی تدریجی (رول‌اوت)

مرحله‌به‌مرحله جایگزین کردن نسخه‌ی قدیمی با جدید: چند پاد جدید بالا می‌آید و بعد قدیمی‌ها پایین می‌آیند تا سرویس قطع نشود. اگر نسخه‌ی جدید مشکل داشت، می‌توان برگشت.

```
kubectl rollout status deployment/churn-api
```

> **در این دوره:** `kubectl rollout status deployment/churn-api` و برای برگشت `kubectl rollout undo deployment/churn-api` (جلسه‌ی ۱۱).

همچنین ببینید: Rollback، Deployment

14.19 port-forward · هدایت پورت

دستوری که یک پورت لپ‌تاپ را موقت به یک سرویس یا پاد داخل کلاستر وصل می‌کند تا بدون Ingress آن را از مرورگر ببینید. برای آزمایش و عیب‌یابی است، نه استفاده‌ی دائمی.

```
kubectl port-forward svc/churn-api 8000:8000
```

> **در این دوره:** Grafana دوره با `kubectl port-forward` روی `localhost:3001` باز می‌شود (جلسه‌ی ۱۱ و ۱۳).

همچنین ببینید: Service، Ingress، Port

14.20 ImagePullBackOff · خطای دریافت image

وضعیتی که یعنی کلاستر نتوانسته image را بگیرد: نام یا tag اشتباه، image در registry نیست یا بارگذاری نشده.
رایج‌ترین خطای آزمایشگاه‌های Kubernetes است.

```
kind load docker-image churn-api:0.1.0 --name mlops
```

در این دوره: در kind باید image را به registry محلی push یا با `kind load docker-image` وارد کلاستر کرد (جلسه‌ی ۱۱).

همچنین ببینید: Container registry، kind، Image tag / digest

15 Ingress و Helm

Ingress & Helm

سرویسی که فقط داخل کلاستر دیده می‌شود، فایده‌ای ندارد. Ingress ترافیک بیرون را به سرویس درست می‌رساند و Helm کل مجموعه‌ی manifestها را به یک بسته‌ی قابل‌نصب با تنظیمات جدا برای هر محیط تبدیل می‌کند.

جلسه‌ها: جلسه ۱۲ · **تعداد واژه‌ها:** ۷

15.1 Ingress — (اینگرس)

قاعده‌ای در Kubernetes که می‌گوید ترافیک HTTP ورودی بر اساس نام میزبان یا مسیر به کدام Service برود. مثل پذیرش ساختمان که بر اساس نام شرکت، مراجعه‌کننده را به طبقه‌ی درست راهنمایی می‌کند.

در این دوره: با Ingress، `http://churn.localhost:8080` به سرویس `churn-api` می‌رسد (جلسه‌ی ۱۲).

همچنین ببینید: Ingress controller (Traefik)، Service، Reverse proxy (Nginx)، Gateway API

15.2 Ingress controller (Traefik) — کنترل‌کننده‌ی Ingress (ترِی‌فیک)

خود شیء Ingress فقط یک قاعده است و برای اجرا به برنامه‌ای نیاز دارد که قاعده‌ها را بخواند و ترافیک را واقعاً هدایت کند؛ این برنامه ingress controller است. Traefik نمونه‌ی آن است.

در این دوره: Traefik با chart `traefik/traefik 41.5.0` نصب می‌شود (Traefik v3.7.13) و به‌جای ingress-nginx که بازنشسته شده انتخاب شده است؛ پورت‌های `8080` و `8443` لپ‌تاپ را می‌گیرد (جلسه‌ی ۱۲).

همچنین ببینید: Ingress، Chart، Gateway API

15.3 Gateway API — (گیت‌وی ای‌پی‌آی)

استاندارد جدیدتر Kubernetes برای مسیریابی ترافیک که قرار است جانشین Ingress شود و امکانات بیشتری مثل جدایی نقش‌ها دارد. با آن می‌توان مسیریابی پیچیده‌تری تعریف کرد و مدیر کلاستر و تیم برنامه نقش‌های جدا دارند.

در این دوره: در جلسه‌ی ۱۲ فقط به‌صورت پیش‌نمایش معرفی می‌شود؛ در تنظیمات Traefik دوره غیرفعال است.

همچنین ببینید: Ingress، Ingress controller (Traefik)

15.4 Helm — (هلم)

Helm مدیر بسته‌ی Kubernetes است: چند manifest را در یک بسته به نام chart جمع می‌کند و نصب، ارتقا و برگشت آن را با یک دستور ممکن می‌کند. مثل `apt` یا `pip` برای کلاستر.

```
helm upgrade --install churn ./charts/churn -f values-dev.yaml
```

در این دوره: نسخه‌ی `v4.2.4`؛ chart مدل چرن با `helm install` نصب و با `helm upgrade` و `helm rollback` به‌روز و برگردانده می‌شود (جلسه‌ی ۱۲).

همچنین ببینید: Chart، Values، Release (Helm)

15.5 Chart · بسته‌ی Helm (چارت)

پوشه‌ای از قالب‌های manifest (`templates/`)، فایل مقدارهای پیش‌فرض (`values.yaml`) و فایل توصیف (`Chart.yaml`). قالب‌ها جاهای خالی دارند که با values پر می‌شوند.

> **در این دوره:** Traefik و Prometheus/Grafana هم با chart نصب می‌شوند (`kube-prometheus-stack 91.3.0`)، و chart `churn` را خودمان می‌نویسیم (جلسه‌ی ۱۲ و ۱۳).

همچنین ببینید: Helm، Values، Manifest

15.6 Values · مقدارهای تنظیم

فایل YAMLای که جاهای خالی قالب‌های chart را پر می‌کند (تعداد replica، نسخه‌ی image، منابع). با فایل values جدا برای هر محیط، یک chart برای dev و prod کافی است.

> **در این دوره:** `values-dev.yaml` و `values-prod.yaml` تنها تفاوت‌های محیط را دارند؛ با گزینه‌ی f انتخاب می‌شوند (جلسه‌ی ۱۲).

همچنین ببینید: Chart، Release (Helm)، ConfigMap

15.7 Release (Helm) · نمونه‌ی نصب‌شده‌ی chart

هر بار نصب یک chart با نام مشخص، یک release ساخته می‌شود که تاریخچه‌ی revisionها را دارد، پس می‌توان به revision قبلی برگشت. (با «release» به معنای نسخه‌ی منتشرشده‌ی کد فرق دارد.) پس هر نصب را می‌توان جدا از نصب‌های دیگر ارتقا داد یا برگرداند.

```
helm history churn
helm rollback churn 1
```

> **در این دوره:** `helm rollback churn 1` به نسخه‌ی قبلی برمی‌گردد و `helm history churn` فهرست revisionها را می‌دهد (جلسه‌ی ۱۲).

همچنین ببینید: Helm، Rollback، Release

16 CI/CD

CI/CD

همان کنترل‌های کیفیتی که تا اینجا دستی (یا با hook) اجرا کرده‌اید، در CI روی یک سرور مشترک و برای هر تغییر خودکار اجرا می‌شود تا هیچ تغییر معیوبی بی‌خبر وارد main نشود. در جلسه‌ی ۱۳ این زنجیره ML-aware می‌شود.

جلسه‌ها: جلسه ۱۳ · **تعداد واژه‌ها:** ۸

16.1 CI (Continuous Integration) · یکپارچه‌سازی پیوسته

روشی که در آن با هر تغییر، ساخت و آزمون خودکار روی یک سرور مشترک اجرا می‌شود تا اگر چیزی خراب شد، همان لحظه دیده شود. هدف: تغییرات کوچک و مکرر را زود و امن به هم یکپارچه کنیم.

در این دوره: workflow جلسه‌ی ۱۳ با هر push مراحل `lint`، `test`، `build`، `scan` و `push` را اجرا می‌کند.

جفت اشتباه‌برانگیز: جدول مقایسه‌ی CI و CD

همچنین ببینید: CD (Continuous Delivery / Deployment)، CI pipeline، Runner، Gitea Actions

16.2 CD (Continuous Delivery / Deployment) · تحویل / استقرار پیوسته

CD ادامه‌ی CI است. در Continuous Delivery هر تغییرِ تأییدشده خودکار آماده‌ی انتشار می‌شود و انتشار با یک تصمیم انسانی انجام می‌گیرد؛ در Continuous Deployment حتی انتشار هم خودکار است.

در این دوره: دوره از سطح ۱ بلوغ است: CI کامل و promotion دستی با gate؛ promotion خودکار در دوره‌ی بعدی است (جلسه‌ی ۱۳).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی CI و CD

همچنین ببینید: CI (Continuous Integration)، Promotion، MLOps maturity levels

16.3 CI pipeline · خط لوله‌ی CI

زنجیره‌ی مرحله‌هایی که CI اجرا می‌کند و اگر هر مرحله شکست بخورد، بقیه اجرا نمی‌شوند. (با DVC در pipeline و scikit-learn فرق دارد؛ اینجا مراحل مهندسی نرم‌افزار است.) ساده‌ترین شکل آن یک زنجیره‌ی خطی از چند مرحله است که از چپ به راست اجرا می‌شود.

در این دوره: `lint`، `test`، `build`، `scan` و `push`، و بعد از آن بخش ML-aware: `dvc repro` و مقایسه‌ی متریک‌ها (جلسه‌ی ۱۳).

همچنین ببینید: CI (Continuous Integration)، DVC pipeline / stage، Pipeline (scikit-learn)

16.4 Gitea Actions · —

سامانه‌ی CI داخلی Gitea با سینتکسی سازگار با GitHub Actions: فایل‌های workflow در Repo نوشته می‌شوند و runner آن‌ها را اجرا می‌کند. هر چه یاد بگیرید در بازار کار هم کاربرد دارد.

در این دوره: workflowها در پوشه‌ی `.gitea/workflows/` قرار می‌گیرند؛ runner با image `gitea/act_runner:0.6.1` اجرا می‌شود (جلسه‌ی ۱۳).

همچنین ببینید: Workflow / Job / Step، Runner، Gitea

16.5 Workflow / Job / Step · گردش کار، کار، گام

workflow فایل YAML کل فرایند CI است که با یک رویداد (مثل push یا زمان‌بندی) شروع می‌شود. هر workflow چند job دارد و هر job از چند step (دستور یا action) ساخته شده است.

در این دوره: workflow جلسه‌ی ۱۳ رویداد push و یک رویداد زمان‌بندی (برای آموزش دوباره) دارد.

همچنین ببینید: Gitea Actions، Runner، Scheduled workflow (cron)، YAML

16.6 Runner · اجراکننده‌ی CI (رانر)

برنامه‌ای که jobهای CI را برمی‌دارد و واقعاً اجرا می‌کند، معمولاً هر job داخل یک کانتینر. سرور Gitea فقط مدیریت می‌کند و کار را runner انجام می‌دهد.

در این دوره: سرویس `gitea-runner` در Compose با تنظیم آفلاین کار می‌کند و job image دوره `mlops/ci-job:1.0` شامل Python، uv، docker CLI و Trivy است (جلسه‌ی ۱۳).

همچنین ببینید: Gitea Actions، CI (Continuous Integration)، Container

16.7 ML-aware CI (CML idea) · CI آگاه از ML

CI که علاوه بر کد، داده و مدل را هم می‌سنجد: pipeline را اجرا می‌کند و تفاوت متریک‌ها با شاخه‌ی اصلی را مثل یک نظر داخل PR می‌گذارد تا بازبین ببیند تغییر مدل را بهتر کرده یا بدتر. این ایده از ابزاری به نام CML گرفته شده است که همین کار را برای سامانه‌هایی مثل GitHub و GitLab انجام می‌دهد.

در این دوره: در جلسه‌ی ۱۳ خروجی `dvc metrics diff` به‌صورت نظر روی PR در Gitea گذاشته می‌شود.

همچنین ببینید: DVC metrics، Pull Request (PR)، CI (Continuous Integration)

16.8 Scheduled workflow (cron) · گردش کار زمان‌بندی‌شده (کرون)

workflow که به‌جای push، طبق یک زمان‌بندی (cron) اجرا می‌شود؛ مثلاً هر هفته. cron ابزار قدیمی زمان‌بندی لینوکس است که ساعت اجرا را با پنج عدد (دقیقه، ساعت، روز، ماه، روز هفته) تعریف می‌کند.

در این دوره: در جلسه‌ی ۱۳ یک workflow برای آموزش دوباره‌ی دوره‌ای مدل با `dvc repro` می‌نویسیم؛ نوعی «orchestration سبک».

همچنین ببینید: Retraining، Orchestration، Workflow / Job / Step

17 پایش و Drift

Monitoring & Drift

استقرار پایان کار نیست. باید بدانید سرویس زنده، سریع و بدون خطاست، و مدل هنوز روی داده‌ی امروز درست پیش‌بینی می‌کند؛ چون داده‌ی دنیای واقعی با زمان تغییر می‌کند.

جلسه‌ها: جلسه ۱۳، ۱۴ · **تعداد واژه‌ها:** ۱۵

17.1 Monitoring / Observability · پایش / مشاهده‌پذیری

جمع‌آوری و دیدن اطلاعات سلامت سیستم در حال کار: متریک‌ها، لاگ‌ها و رخدادها. پایش می‌گوید «چیزی خراب است» و observability کمک می‌کند بفهمید «چرا».

در این دوره: در جلسه‌ی ۱۳ متریک‌های API با Prometheus جمع و با Grafana نمایش داده می‌شوند و گزارش drift با Evidently ساخته می‌شود.

همچنین ببینید: Prometheus، Grafana، Data drift، Logging (structured)

17.2 Metric (Prometheus) · متریک پایش

یک عدد اندازه‌گیری‌شده‌ی سیستم در گذر زمان، مثل تعداد درخواست یا زمان پاسخ. (با «metric» به معنای معیار کیفیت مدل مثل ROC-AUC فرق دارد؛ اینجا درباره‌ی سلامت سرویس است.) متریک‌ها معمولاً عدد و برچسب دارند و با زمان ذخیره می‌شوند تا بتوان روند آن‌ها را دید.

در این دوره: `prometheus-fastapi-instrumentator 8.1.0` متریک‌های درخواست را از مسیر `/metrics` در اختیار Prometheus می‌گذارد (جلسه‌ی ۸ و ۱۳).

همچنین ببینید: Metric، Scrape، /metrics endpoint

17.3 Prometheus · — (پرومتئوس)

Prometheus پایگاه داده و سامانه‌ی جمع‌آوری متریک است که به‌صورت دوره‌ای از سرویس‌ها متریک می‌گیرد، آن‌ها را با زمان ذخیره می‌کند و امکان پرس‌وجو و هشدار می‌دهد. چون خودش از سرویس‌ها می‌خواند، سرویس نیازی ندارد بداند Prometheus کجاست.

در این دوره: نصب سبک Prometheus در کلاستر kind با chart `kube-prometheus-stack` (نسخه‌ی `91.3.0`، حدود ۱ گیگابایت RAM) انجام می‌شود (جلسه‌ی ۱۳).

همچنین ببینید: Scrape، Grafana، Alert

17.4 Scrape · برداشت متریک (اسکریپ)

در Prometheus، خود Prometheus در فاصله‌های منظم (مثلاً هر ۱۵ ثانیه) به آدرس `/metrics` سرویس‌ها سر می‌زند و مقدارها را می‌خواند؛ سرویس‌ها متریک را «نمی‌فرستند»، بلکه Prometheus «برمی‌دارد». اگر سرویس خاموش باشد، scrape شکست می‌خورد و همین نشانه‌ای از مشکل است.

در این دوره: Prometheus در جلسه‌ی ۱۳ به `/metrics` سرویس `churn-api` متصل می‌شود.

همچنین ببینید: Prometheus، /metrics endpoint

17.5 /metrics endpoint · نقطه‌ی متریک

مسیری از API که متریک‌های سرویس را به‌صورت متن ساده و با قالب استاندارد Prometheus نشان می‌دهد. خواندنش برای انسان ساده است ولی هدف اصلی آن خوراک دادن به Prometheus است.

در این دوره: در جلسه‌ی ۸ به API اضافه می‌شود تا در جلسه‌ی ۱۳ «از قبل آماده» باشد.

همچنین ببینید: Endpoint (API)، Scrape، Metric (Prometheus)

17.6 Grafana · — (گرافانا)

Grafana ابزار نمایش داده با نمودارهای زنده و داشبورد است که از Prometheus و منابع دیگر می‌خواند. Grafana خودش داده ندارد؛ فقط آن را از منبع می‌خواند و زیبا و قابل‌فهم نشان می‌دهد.

در این دوره: نسخه‌ی `13.2` از chart؛ با `kubectl port-forward` روی `localhost:3001` باز می‌شود و نمودار تاخیر، خطا و تعداد درخواست دارد (جلسه‌ی ۱۳).

همچنین ببینید: Dashboard، Prometheus، port-forward

17.7 Dashboard · داشبورد

صفحه‌ای که چند نمودار و عدد مهم را کنار هم نشان می‌دهد تا وضعیت سرویس در یک نگاه فهمیده شود؛ مثل صفحه‌ی ابزار ماشین. داشبورد خوب فقط چند عدد مهم دارد، نه صدها نمودار.

در این دوره: داشبورد دوره بر پایه‌ی روش RED است: Rate، Errors و Duration (جلسه‌ی ۱۳).

همچنین ببینید: Grafana، RED method (latency)

17.8 RED method (latency) · روش RED (رد)

سه عدد پایه برای سنجش هر سرویس: Rate (چند درخواست در ثانیه)، Errors (چه نسبتی خطا شد) و Duration (چقدر طول کشید؛ همان latency یا تاخیر). با همین سه عدد بیشتر مشکل‌ها دیده می‌شوند.

در این دوره: در جلسه‌ی ۱۳ سه نمودار Grafana برای `churn-api` ساخته می‌شود.

همچنین ببینید: Dashboard، SLO / SLI، Metric (Prometheus)

17.9 Alert · هشدار (الرت)

قاعده‌ای که وقتی متریکی از حد تعیین‌شده بگذرد، به کسی خبر می‌دهد. هشدار خوب نادر و قابل‌اقدام است؛ هشدارهای زیاد باعث نادیده گرفتن همه‌شان می‌شود.

در این دوره: در دوره به‌صورت مفهوم و یک قاعده‌ی نمونه معرفی می‌شود (جلسه‌ی ۱۳).

همچنین ببینید: Prometheus، SLO / SLI

17.10 SLO / SLI · هدف سطح خدمت

SLI اندازه‌ای است که کیفیت را می‌سنجد (مثلاً نسبت پاسخ‌های سریع) و SLO هدفی که برایش تعیین می‌کنید (مثلاً ۹۹٪ پاسخ‌ها زیر نیم‌ثانیه). با آن می‌دانید کِی «کافی خوب» است و کِی باید اقدام کرد.

در این دوره: در جلسه‌ی ۱۳ و ۱۴ در چک‌لیست آمادگی تولید مثال زده می‌شود.

همچنین ببینید: RED method (latency)، Alert، Production-readiness checklist

17.11 Data drift · انحراف داده (دیتا دریفت)

وقتی توزیع داده‌ی ورودی در تولید با داده‌ی آموزش فرق کند؛ مثلاً مشتریان جدید با مدت اشتراک کوتاه‌تر. مدل ممکن است همان رابطه را یاد گرفته باشد، ولی روی نمونه‌هایی که ندیده بد عمل کند.

در این دوره: داده‌ی v2 (۸۰۰۰ سطر) عمداً با جمعیتی تغییریافته ساخته شده و ROC-AUC از `0.802` به `0.7493` می‌رسد؛ Evidently تغییر توزیع v1 و v2 را گزارش می‌کند (جلسه‌ی ۴ و ۱۳).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Data drift و Concept drift

همچنین ببینید: Concept drift، Evidently، Retraining

17.12 Concept drift · انحراف مفهوم (کانسپت دریفت)

وقتی خودِ رابطه‌ی بین ورودی و خروجی عوض می‌شود: همان داده‌ی ورودی حالا جواب متفاوتی دارد. مثلاً بعد از تغییر قیمت‌ها، همان رفتار مشتری دیگر نشانه‌ی ریزش نیست. فقط با دیدن برچسب‌های واقعی جدید قابل تشخیص است.

در این دوره: در دوره به‌صورت مفهومی گفته می‌شود؛ آنچه در آزمایش می‌سازیم گزارش data drift است (جلسه‌ی ۱ و ۱۳).

جفت اشتباه‌برانگیز: جدول مقایسه‌ی Data drift و Concept drift

همچنین ببینید: Data drift، Retraining

17.13 Evidently · — (اویدنتلی)

Evidently کتابخانه‌ی Python برای ساخت گزارش درباره‌ی کیفیت داده و drift است: داده‌ی مرجع (آموزش) را با داده‌ی جدید مقایسه می‌کند و یک گزارش HTML تولید می‌کند. خروجی را می‌توان در مرورگر باز کرد و از روی آن دید کدام ستون‌ها تغییر کرده‌اند.

در این دوره: نسخه‌ی `0.7.23` با API جدید 0.7 پین شده؛ گزارش v1 در برابر v2 در جلسه‌ی ۱۳ ساخته می‌شود.

همچنین ببینید: Data drift، Monitoring / Observability

17.14 Retraining · آموزش دوباره

آموزش مدل با داده‌ی تازه‌تر برای جبران drift. می‌تواند زمان‌بندی‌شده (مثلاً هفتگی) یا تحریک‌شده با هشدار باشد؛ مدل جدید قبل از جایگزینی باید از evaluation gate بگذرد.

در این دوره: در جلسه‌ی ۱۳ یک workflow زمان‌بندی‌شده `dvc repro` را اجرا می‌کند؛ آموزش خودکار مبتنی بر drift در دوره‌ی بعدی است.

همچنین ببینید: Scheduled workflow (cron)، Evaluation gate، MLOps maturity levels

17.15 Production-readiness checklist · چک‌لیست آمادگی تولید

فهرست امتیازدار از چیزهایی که یک سرویس ML باید داشته باشد تا بتوان به آن اعتماد کرد: تست، پایش، rollback، امنیت، مستندات و runbook. قبل از اینکه بگویید پروژه تمام شد، با این چک‌لیست کمبودها را پیدا می‌کنید.

در این دوره: مبنای نمره‌دهی پروژه‌ی نهایی است: اجرا از clean clone (۱۵)، pipeline تکرارپذیر (۱۰)، registry و alias (۵)، image امن (۵)، K8s+Helm+Ingress (۵)، پایش و drift (۵)، runbook و demo (۵) (جلسه‌ی ۱۳ و ۱۴).

همچنین ببینید: Clean clone، Runbook، SLO / SLI

جفت‌های اشتباه‌برانگیز

Commonly confused pairs

این ۱۶ جفت بیشترین سردرگمی را برای تازه‌واردها ایجاد می‌کنند. اول جدول را بخوانید، بعد جمله‌ی آخر را برای به‌خاطر سپردن ببینید.

Parameter (weight) در برابر Hyperparameter

جنبه	Parameter (weight)	Hyperparameter
چه کسی مقدارش را تعیین می‌کند؟	الگوریتم، هنگام آموزش، از روی داده	شما، قبل از آموزش
کجا ذخیره می‌شود؟	داخل فایل مدل	در `configs/params.yaml`
مثال در دوره	وزن‌های logistic regression	`C: 1.0` و `max_iter: 500`

یادآوری: پارامتر را مدل «یاد می‌گیرد»؛ ابرپارامتر را شما «تنظیم می‌کنید».

تعریف‌ها: Parameter (weight)، Hyperparameter

Precision در برابر Recall

جنبه	Precision	Recall
سؤالی که جواب می‌دهد	از هشدارهای مدل چند درصد درست بود؟	از ریزش‌های واقعی چند درصد را پیدا کردیم؟
اگر پایین باشد	هشدار بی‌جای زیاد	ریزش‌های از دست‌رفته‌ی زیاد
با بالا بردن threshold	معمولاً بالا می‌رود	معمولاً پایین می‌آید

یادآوری: F1 این دو را در یک عدد ترکیب می‌کند.

تعریف‌ها: Precision، Recall

Tag (Git) در برابر Branch

جنبه	Tag (Git)	Branch
حرکت می‌کند؟	نه، به یک commit ثابت اشاره دارد	بله، با هر commit جلو می‌رود
کاربرد	علامت‌گذاری نسخه‌ی منتشرشده	خط کار در حال انجام
مثال در دوره	`v0.1.0`، `data-v2`	`feature/quality-gates`

یادآوری: tag عکس یادگاری است؛ branch جاده‌ای است که ادامه دارد.

تعریف‌ها: Tag، Branch

git checkout در برابر dvc checkout

جنبه	git checkout	dvc checkout
چه چیزی را برمی‌گرداند؟	کد و فایل‌های pointer	فایل‌های داده‌ی واقعی مطابق pointer
داده‌ی بزرگ را عوض می‌کند؟	نه	بله (از cache)
ترتیب برگشت به نسخه‌ی ۱	اول `git checkout data-v1`	بعد `uv run dvc checkout`

یادآوری: اگر فقط git checkout بزنید، کد عوض شده ولی داده همان نسخه‌ی قبلی است.

تعریف‌ها: git checkout / git switch، dvc checkout

remote (Git) در برابر remote (DVC)

جنبه	remote (Git)	remote (DVC)
چه چیزی در آن نگه‌داری می‌شود؟	کد، تاریخچه، فایل‌های pointer	بایت‌های واقعی داده
مکان در دوره	Gitea (`localhost:3000`)	bucket `dvc` روی SeaweedFS
دستور ارسال	`git push`	`dvc push`

یادآوری: دو remote مستقل داریم و هر دو باید push شوند.

تعریف‌ها: Remote (git remote / origin)، Remote (DVC)

Image در برابر Container

جنبه	Image	Container
چیست؟	بسته‌ی فقط‌خواندنی و ثابت	نمونه‌ی در حال اجرا از یک image
تشبیه	دستور پخت یا «کلاس»	غذای پخته‌شده یا «شیء»
چند تا؟	یک image می‌تواند چندین کانتینر بسازد	هر کانتینر از یک image می‌آید

یادآوری: با `docker run` از image کانتینر ساخته می‌شود؛ با حذف کانتینر image می‌ماند.

تعریف‌ها: Image، Container

Volume در برابر Bind mount

جنبه	Volume	Bind mount
چه کسی مدیریت می‌کند؟	Docker	شما، با یک مسیر مشخص روی میزبان
بهترین کاربرد	داده‌ی ماندگار سرویس (مثل `seaweedfs-data`)	فایل تنظیمات مثل `s3.json`
وابستگی به ماشین	کم	زیاد (به مسیر میزبان)

یادآوری: داده‌ی سرویس را در volume بگذارید و تنظیمات را با bind mount بدهید.

تعریف‌ها: Volume، Bind mount

CI در برابر CD

جنبه	CI	CD
مخفف	Continuous Integration	Deployment / Continuous Delivery
پاسخ می‌دهد	آیا این تغییر خراب نمی‌کند؟ (ساخت و تست)	آیا آماده‌ی انتشار است و منتشر شد؟
نتیجه	image یا بسته‌ی تأییدشده	نسخه‌ی در حال اجرا در محیط هدف

یادآوری: CI کیفیت را می‌سنجد؛ CD آن را به دست کاربر می‌رساند.

تعریف‌ها: CI (Continuous Integration)، CD (Continuous Delivery / Deployment)

Data drift در برابر Concept drift

جنبه	Data drift	Concept drift
چه چیزی عوض می‌شود؟	توزیع ورودی‌ها	رابطه‌ی ورودی و خروجی
بدون برچسب تشخیص‌پذیر؟	بله	معمولاً نه؛ به برچسب جدید نیاز دارد
مثال	مشتریان جدید با tenure کوتاه‌تر	بعد از تغییر قیمت، رفتار قبلی دیگر نشانه‌ی ریزش نیست

یادآوری: data drift یعنی «ورودی‌ها فرق کرده‌اند»، concept drift یعنی «معنای جواب فرق کرده است».

تعریف‌ها: Data drift، Concept drift

Authentication در برابر Authorization

جنبه	Authentication	Authorization
سؤال	تو کیستی؟	چه کاری مجازی بکنی؟
ترتیب	اول	بعد از احراز هویت
مثال در دوره	کلید دسترسی S3	سطح‌های `Read`، `Write`، `List` در `s3.json`

یادآوری: کارت شناسایی را نشان می‌دهید (authentication)، بعد معلوم می‌شود به کدام طبقه می‌توانید بروید (authorization).

تعریف‌ها: Authentication، Authorization

Alias (MLflow) در برابر Tag (MLflow)

جنبه	Alias (MLflow)	Tag (MLflow)
کارکرد	اشاره‌ی متحرک به یک نسخه‌ی مدل	توضیح کلید/مقدار روی نسخه یا run
برای بارگذاری استفاده می‌شود؟	بله: `models:/churn@champion`	نه
مثال	`@champion`، `@challenger`	`validation_status=approved`

یادآوری: در دنیای Git و Docker نیز واژه‌ی tag معنای دیگری دارد؛ همیشه بپرسید «tag چه چیزی؟».

تعریف‌ها: Alias (@champion / @challenger)، Tags (MLflow)

Pod در برابر Deployment

جنبه	Pod	Deployment
نقش	واحد اجرای کانتینر	مدیر یک گروه از پادها
اگر خراب شود	جایگزین نمی‌شود مگر مدیری داشته باشد	پاد جدید می‌سازد تا تعداد replica حفظ شود
به‌روزرسانی	ندارد	rollout تدریجی و rollback

یادآوری: معمولاً پاد مستقیم نمی‌سازید؛ Deployment می‌سازید.

تعریف‌ها: Pod، Deployment

ConfigMap در برابر Secret (Kubernetes)

جنبه	ConfigMap	Secret (Kubernetes)
محتوا	تنظیمات غیرمحرمانه	رمز، کلید، توکن
پیش‌فرض ذخیره	متن ساده	کدگذاری base64 (رمزنگاری قوی نیست)
مثال	نام alias مدل	کلید S3 آزمایشگاه

یادآوری: هر دو بیرون از imageاند؛ تفاوت در حساسیت داده و محدود کردن دسترسی است.

تعریف‌ها: ConfigMap، Secret (Kubernetes)

Liveness probe در برابر Readiness probe

جنبه	Liveness probe	Readiness probe
می‌پرسد	آیا برنامه هنوز زنده است؟	آیا آماده‌ی دریافت ترافیک است؟
اگر ناموفق باشد	کانتینر دوباره راه‌اندازی می‌شود	پاد موقتاً از Service کنار گذاشته می‌شود
مثال	برنامه قفل کرده است	هنوز مدل بارگذاری نشده است

یادآوری: liveness برای «نجات»، readiness برای «صبر کردن».

تعریف‌ها: Probes: liveness / readiness، Probes: liveness / readiness

scikit-learn Pipeline در برابر DVC / CI pipeline

جنبه	scikit-learn Pipeline	DVC / CI pipeline
چه چیزی را زنجیر می‌کند؟	پیش‌پردازش و مدل داخل یک شیء Python	مراحل کار روی فایل‌ها (DVC) یا مراحل مهندسی (CI)
کجا تعریف می‌شود؟	در کد `train.py`	در `dvc.yaml` یا workflow در `.gitea/workflows/`
مثال	`pre` ← `clf`	`preprocess` ← `train` ← `evaluate` یا `lint` ← `test` ← `build`

یادآوری: وقتی کسی می‌گوید «pipeline»، بپرسید کدام یک.

تعریف‌ها: Pipeline (scikit-learn)، DVC pipeline / stage

Container registry در برابر Model registry

جنبه	Container registry	Model registry
چه چیزی نگه می‌دارد؟	imageهای Docker	نسخه‌های مدل و aliasها
ابزار در دوره	`registry:3.1.0` روی `localhost:5001`	MLflow روی `localhost:5050`
شناسه	`localhost:5001/churn-api:0.1.0`	`models:/churn@champion`

یادآوری: image بسته‌ی برنامه است؛ مدل یک artifact داخل آن یا کنار آن است.

تعریف‌ها: Container registry، Model registry

اختصارها

Abbreviations

اختصار	شکل کامل	معنی فارسی	تعریف
API	Application Programming Interface	رابط برنامه‌نویسی	1.26
AUC	Area Under the Curve	سطح زیر منحنی	3.14
CD	Continuous Delivery / Deployment	تحویل / استقرار پیوسته	16.2
CI	Continuous Integration	یکپارچه‌سازی پیوسته	16.1
CLI	Command-Line Interface	رابط خط فرمان	1.4
CPU	Central Processing Unit	واحد پردازش مرکزی	—
CSV	Comma-Separated Values	مقدارهای جداشده با ویرگول	1.31
CVE	Common Vulnerabilities and Exposures	شناسه‌ی آسیب‌پذیری‌های شناخته‌شده	12.5
DAG	Directed Acyclic Graph	گراف جهت‌دار بدون دور	9.14
DNS	Domain Name System	سامانه‌ی نام دامنه	1.22
DoD	Definition of Done	تعریف انجام‌شده	4.16
DVC	Data Version Control	نسخه‌بندی داده	9.1
ETL / ELT	Extract, Transform, Load	استخراج، تبدیل، بارگذاری	—
F1	F1 score (harmonic mean of precision and recall)	میانگین هارمونیک precision و recall	3.17
HTTP	HyperText Transfer Protocol	پروتکل انتقال ابرمتن	1.23
IaC	Infrastructure as Code	زیرساخت به‌عنوان کد	13.4
IP	Internet Protocol (address)	نشانی پروتکل اینترنت	1.21
JSON	JavaScript Object Notation	قالب متنی داده با کلید و مقدار	1.28
K8s	Kubernetes	کوبرنتیز	14.1
LFS	Large File Storage (Git LFS)	ذخیره‌سازی فایل‌های بزرگ	9.10
MD5	Message-Digest algorithm 5	تابع هش md5	9.5
ML	Machine Learning	یادگیری ماشین	—
MLOps	Machine Learning Operations	عملیات یادگیری ماشین	4.1
OOM	Out Of Memory	تمام شدن حافظه	14.17

اختصار	شکل کامل	معنی فارسی	تعریف
PR	Pull Request	درخواست ادغام	5.14
RAM	Random Access Memory	حافظه‌ی دسترسی تصادفی	—
RBAC	Role-Based Access Control	کنترل دسترسی بر پایه‌ی نقش	8.10
RED	Rate, Errors, Duration	نرخ، خطا، مدت	17.8
REST	Representational State Transfer	—	1.27
ROC	Receiver Operating Characteristic	منحنی مشخصه‌ی عملکرد	3.14
S3	Simple Storage Service	سرویس ذخیره‌سازی ساده (رابط ذخیره‌ساز شیء)	8.2
SBOM	Software Bill of Materials	فهرست اجزای نرم‌افزار	12.6
SemVer	Semantic Versioning	نسخه‌گذاری معنایی	5.17
SHAP	SHapley Additive exPlanations	توضیح‌پذیری پیش‌بینی مدل	3.28
SLI	Service Level Indicator	شاخص سطح خدمت	17.10
SLO	Service Level Objective	هدف سطح خدمت	17.10
SSH	Secure Shell	پوسته‌ی امن	1.17
TLS	Transport Layer Security	امنیت لایه‌ی انتقال (رمزنگاری HTTPS)	—
TOML	Tom's Obvious Minimal Language	قالب فایل تنظیمات	1.30
URL	Uniform Resource Locator	نشانی وب	1.25
VM	Virtual Machine	ماشین مجازی	7.4
WSL2	Windows Subsystem for Linux 2	زیرسیستم ویندوز برای لینوکس	1.32
YAML	YAML Ain't Markup Language	قالب فایل تنظیمات با تورفتگی	1.29

نمایه‌ی انگلیسی

English Index

فهرست الفبایی همه‌ی ۲۶۷ واژه‌ی کتابچه. عدد کنار هر واژه شماره‌ی بخش و شماره‌ی صفحه است؛ روی واژه کلیک کنید تا به تعریفش بروید.

A

8.8 · 40 ..... Access key
4.12 · 23 ..... Air-gapped
17.9 · 68 ..... Alert
10.9 · 47 ..... Alias (@champion / @challenger)
13.5 · 56 ..... Ansible
1.26 · 10 ..... API
1.5 · 6 ..... Option / Flag / Argument ← Argument
10.6 · 47 ..... Artifact
8.9 · 40 ..... Authentication
8.10 · 41 ..... Authorization
8.11 · 41 ..... boto3 / aws CLI ← aws CLI

B

7.7 · 36 ..... Base image
3.12 · 17 ..... Baseline
1.3 · 5 ..... Bash
11.11 · 52 ..... Batch inference
4.19 · 24 ..... Batch vs Stream
7.11 · 37 ..... Bind mount
8.11 · 41 ..... boto3
5.7 · 27 ..... Branch
5.23 · 30 ..... Branch protection
5.22 · 30 ..... Branching strategy (feature / release / hotfix)
8.4 · 39 ..... Bucket

C

16.2 · 65 ..... CD (Continuous Delivery / Deployment)
10.11 · 48 ..... Champion / Challenger ← Challenger
10.11 · 48 ..... Champion
15.5 · 64 ..... Chart
4.18 · 24 ..... Golden repo / Checkpoint branch ← Checkpoint branch
9.5 · 43 ..... Hash (md5) / Checksum ← Checksum
3.21 · 19 ..... Churn
16.1 · 65 ..... CI (Continuous Integration)
16.3 · 65 ..... CI pipeline
3.20 · 19 ..... Classification
4.17 · 24 ..... Clean clone
5.12 · 28 ..... clone
14.2 · 58 ..... Cluster
5.15 · 29 ..... Code review
1.4 · 5 ..... Command
5.3 · 26 ..... Commit
5.4 · 27 ..... Commit hash
17.12 · 69 ..... Concept drift
6.9 · 34 ..... Config as code
14.11 · 60 ..... ConfigMap
3.19 · 18 ..... Confusion matrix
7.1 · 35 ..... Container
7.9 · 37 ..... Container registry
7.3 · 35 ..... Docker Engine / Container runtime ← Container runtime
5.18 · 29 ..... Conventional Commits
1.31 · 11 ..... CSV
1.18 · 8 ..... curl
12.5 · 54 ..... CVE

D

9.14 · 45 ..... DAG
17.7 · 68 ..... Dashboard
17.11 · 69 ..... Data drift
4.20 · 25 ..... Data lake
4.7 · 22 ..... Data lineage
4.4 · 21 ..... Data Scientist
9.2 · 42 ..... Data versioning
4.20 · 25 ..... Data lake / Data warehouse ← Data warehouse
3.6 · 16 ..... Dataset
14.14 · 60 ..... Declarative
4.16 · 24 ..... Definition of Done (DoD)
2.3 · 12 ..... Dependency
14.7 · 59 ..... Deployment
14.14 · 60 ..... Declarative / Desired state ← Desired state
5.20 · 30 ..... HEAD / detached HEAD ← detached HEAD
4.2 · 21 ..... DevOps
7.8 · 36 ..... Image tag / digest ← digest
1.22 · 9 ..... DNS
7.14 · 38 ..... Docker Compose
7.3 · 35 ..... Docker Engine
7.12 · 37 ..... Docker network
7.6 · 36 ..... Dockerfile
7.15 · 38 ..... .dockerignore
9.1 · 42 ..... DVC (Data Version Control)
9.6 · 43 ..... DVC cache
9.9 · 44 ..... dvc checkout
9.18 · 45 ..... dvc exp
9.17 · 45 ..... DVC metrics
9.11 · 44 ..... DVC pipeline
9.8 · 43 ..... dvc push
9.15 · 45 ..... dvc repro
9.13 · 44 ..... dvc.lock
9.12 · 44 ..... dvc.yaml (deps / outs)

E

11.5 · 51 ..... Endpoint (API)
8.7 · 40 ..... Endpoint (S3)
1.8 · 6 ..... env file
1.7 · 6 ..... Environment variable

K

14.4 · 58 ..... kind
14.5 · 59 ..... kubectl
14.1 · 58 ..... Kubernetes (K8s)

L

3.5 · 16 ..... Label
14.15 · 61 ..... Label
7.5 · 36 ..... Layer
14.17 · 61 ..... Requests / Limits ← Limits
6.1 · 32 ..... Lint
1.19 · 9 ..... localhost
2.8 · 13 ..... Lockfile (uv.lock)
13.11 · 57 ..... Log rotation
11.8 · 51 ..... Logging (structured)
3.22 · 19 ..... Logistic regression

M

5.8 · 27 ..... main
6.6 · 33 ..... Makefile / make ← make
6.6 · 33 ..... Makefile
14.13 · 60 ..... Manifest
5.9 · 28 ..... Merge
5.10 · 28 ..... Merge conflict
3.13 · 17 ..... Metric
17.2 · 67 ..... Metric (Prometheus)
17.5 · 68 ..... /metrics endpoint
4.4 · 21 ..... Data Scientist / ML Engineer / M… ← ML Engineer
4.3 · 21 ..... ML lifecycle
16.7 · 66 ..... ML-aware CI (CML idea)
10.1 · 46 ..... MLflow
4.1 · 21 ..... MLOps
4.4 · 21 ..... Data Scientist / ML Engineer / M… ← MLOps Engineer
4.9 · 22 ..... MLOps maturity levels
3.1 · 15 ..... Model
10.15 · 48 ..... Model card
10.7 · 47 ..... Model registry
11.1 · 50 ..... Model serving
10.14 · 48 ..... Model signature
10.16 · 49 ..... Model URI (models:/…)
10.8 · 47 ..... Registered model / Model version ← Model version
2.2 · 12 ..... Module
17.1 · 67 ..... Monitoring
12.1 · 53 ..... Multi-stage build
13.3 · 55 ..... Multipass

N

14.10 · 60 ..... Namespace
14.3 · 58 ..... Node
12.2 · 53 ..... Non-root user

O

8.5 · 40 ..... Object
8.6 · 40 ..... Object key
8.1 · 39 ..... Object storage
17.1 · 67 ..... Monitoring / Observability ← Observability
10.13 · 48 ..... Evaluation gate
17.13 · 69 ..... Evidently
1.15 · 8 ..... Exit code
10.3 · 46 ..... Experiment (MLflow)
10.2 · 46 ..... Experiment tracking

F

3.17 · 18 ..... F1 score
11.2 · 50 ..... FastAPI
3.4 · 15 ..... Feature
5.13 · 28 ..... push / pull / fetch ← fetch
1.11 · 7 ..... File permission
1.5 · 6 ..... Option / Flag / Argument ← Flag
6.2 · 32 ..... Formatter

G

15.3 · 63 ..... Gateway API
5.1 · 26 ..... Git
5.21 · 30 ..... git checkout
6.8 · 34 ..... Git hook
9.10 · 44 ..... Git LFS
5.21 · 30 ..... git checkout / git switch ← git switch
5.24 · 31 ..... Gitea
16.4 · 65 ..... Gitea Actions
5.19 · 30 ..... .gitignore
4.18 · 24 ..... Golden repo
17.6 · 68 ..... Grafana

H

9.5 · 43 ..... Hash (md5)
5.20 · 30 ..... HEAD
11.7 · 51 ..... Health check
15.4 · 63 ..... Helm
1.23 · 9 ..... HTTP
1.24 · 10 ..... HTTP status code
11.9 · 51 ..... httpx
3.10 · 17 ..... Hyperparameter

I

13.8 · 56 ..... Idempotent
7.2 · 35 ..... Image
7.8 · 36 ..... Image tag
14.20 · 62 ..... ImagePullBackOff
3.3 · 15 ..... Inference
13.4 · 55 ..... Infrastructure as Code (IaC)
15.1 · 63 ..... Ingress
15.2 · 63 ..... Ingress controller (Traefik)
13.7 · 56 ..... Inventory
1.21 · 9 ..... IP address

J

16.5 · 66 ..... Workflow / Job / Step ← Job
1.28 · 10 ..... JSON
2.12 · 14 ..... Jupyter Notebook

17.14 · 69 ..... Retraining
13.2 · 55 ..... Reverse proxy (Nginx)
3.14 · 17 ..... ROC-AUC
4.15 · 24 ..... Rollback
14.18 · 61 ..... Rollout
6.3 · 32 ..... ruff
10.4 · 46 ..... Run
13.10 · 57 ..... Runbook
16.6 · 66 ..... Runner

S

8.2 · 39 ..... S3 (and S3-compatible)
12.6 · 54 ..... SBOM
16.8 · 66 ..... Scheduled workflow (cron)
3.23 · 19 ..... scikit-learn
17.4 · 67 ..... Scrape
8.3 · 39 ..... SeaweedFS
5.25 · 31 ..... Secret
14.12 · 60 ..... Secret (Kubernetes)
8.8 · 40 ..... Access key / Secret key ← Secret key
3.11 · 17 ..... Seed
14.15 · 61 ..... Label / Selector ← Selector
5.17 · 29 ..... SemVer
3.26 · 20 ..... Serialization (joblib / pickle)
14.9 · 59 ..... Service
3.28 · 20 ..... SHAP
1.2 · 5 ..... Shell
1.16 · 8 ..... Shell script
17.10 · 68 ..... SLO / SLI ← SLI
17.10 · 68 ..... SLO
9.3 · 42 ..... Snapshot
12.8 · 54 ..... Software supply chain
1.17 · 8 ..... SSH
9.11 · 44 ..... DVC pipeline / stage ← stage
10.17 · 49 ..... Stages (legacy)
5.5 · 27 ..... Staging area (git add)
6.1 · 32 ..... Lint / Static analysis ← Static analysis
9.8 · 43 ..... dvc push / pull / status ← status
1.14 · 8 ..... stdout / stderr ← stderr
1.14 · 8 ..... stdout
16.5 · 66 ..... Workflow / Job / Step ← Step
1.10 · 7 ..... sudo
11.10 · 52 ..... OpenAPI / Swagger docs ← Swagger docs
12.7 · 54 ..... Syft
13.1 · 55 ..... systemd

T

5.16 · 29 ..... Tag
10.10 · 48 ..... Tags (MLflow)
3.5 · 16 ..... Label / Target ← Target
4.10 · 23 ..... Technical debt (in ML)
1.1 · 5 ..... Terminal
13.9 · 56 ..... Terraform
11.9 · 51 ..... httpx / TestClient ← TestClient
4.5 · 22 ..... The 7 MLOps components
3.18 · 18 ..... Threshold
1.30 · 11 ..... TOML
4.8 · 22 ..... Traceability
10.5 · 47 ..... Tracking server
4.12 · 23 ..... Air-gapped / Offline ← Offline
4.11 · 23 ..... On-premises (on-prem)
11.10 · 52 ..... OpenAPI
1.5 · 6 ..... Option
4.13 · 23 ..... Orchestration
3.8 · 16 ..... Overfitting

P

2.2 · 12 ..... Module / Package ← Package
3.9 · 16 ..... Parameter (weight)
9.16 · 45 ..... Params (params.yaml)
1.6 · 6 ..... Path
1.9 · 7 ..... PATH
2.5 · 13 ..... pip
1.12 · 7 ..... Pipe
3.24 · 19 ..... Pipeline (scikit-learn)
13.6 · 56 ..... Playbook
14.6 · 59 ..... Pod
9.4 · 42 ..... Pointer file (.dvc)
1.20 · 9 ..... Port
7.13 · 37 ..... Port mapping
14.19 · 61 ..... port-forward
6.7 · 33 ..... pre-commit
3.15 · 18 ..... Precision
3.3 · 15 ..... Inference / Prediction ← Prediction
8.6 · 40 ..... Object key / Prefix ← Prefix
3.25 · 20 ..... Preprocessing (scaling, one-hot)
14.16 · 61 ..... Probes: liveness
17.15 · 69 ..... Production-readiness checklist
17.3 · 67 ..... Prometheus
10.12 · 48 ..... Promotion
9.8 · 43 ..... dvc push / pull / status ← pull
5.13 · 28 ..... push / pull / fetch ← pull
5.14 · 29 ..... Pull Request (PR)
5.13 · 28 ..... push
11.4 · 50 ..... pydantic
2.11 · 14 ..... PyPI
2.7 · 13 ..... pyproject.toml
6.5 · 33 ..... pytest
2.1 · 12 ..... Python

R

14.16 · 61 ..... Probes: liveness / readiness ← readiness
3.16 · 18 ..... Recall
17.8 · 68 ..... RED method (latency)
1.13 · 7 ..... Redirect
10.8 · 47 ..... Registered model
4.14 · 23 ..... Release
15.7 · 64 ..... Release (Helm)
10.15 · 48 ..... Model card / Release note ← Release note
9.7 · 43 ..... Remote (DVC)
5.11 · 28 ..... Remote (git remote / origin)
14.8 · 59 ..... Replica
5.2 · 26 ..... Repository (Repo)
4.6 · 22 ..... Reproducibility
11.6 · 51 ..... Request
14.17 · 61 ..... Requests
11.6 · 51 ..... Request / Response schema ← Response schema
1.27 · 10 ..... REST

3.27 · 20 ..... Train/serve skew
3.7 · 16 ..... Train/test split
3.2 · 15 ..... Training
12.4 · 53 ..... Trivy

U

6.4 · 33 ..... Unit test
1.25 · 10 ..... URL
2.6 · 13 ..... uv
11.3 · 50 ..... uvicorn

V

15.6 · 64 ..... Values
2.4 · 13 ..... Virtual environment
7.4 · 36 ..... Virtual machine (VM)
7.10 · 37 ..... Volume
12.3 · 53 ..... Vulnerability scan

W

2.9 · 14 ..... Wheel
2.10 · 14 ..... Wheelhouse
16.5 · 66 ..... Workflow
5.6 · 27 ..... Working tree
1.32 · 11 ..... WSL2

Y

1.29 · 11 ..... YAML